

Capstone Project — thirty-four briefs

Dr. Kanthi Kiran Sirra · datatrends.tech · kk@datatrends.tech

The pipeline is the same for everybody. The *question* is yours. Choose one topic, build it end to end on your own, and submit by the announced date. Each topic gets four pages: the problem and the data, the generator to run, the layers you build, then the questions and the checks.

This is not Mini Project 2. Mini Project 2 is the Olist pipeline released in August — it is unchanged, and it is due **Sunday 13 September, 11:59 PM**. The Capstone is separate, comes after that, and is where you choose your own topic. **The Capstone deadline will be announced in the group.**

1 · BRONZE Land the raw file. Change nothing.	2 · SILVER Cast, dedupe, drop orphans. Write down what you decided.	3 · GOLD Aggregate to the answers. Small enough to travel.
4 · EXPORT Gold to one CSV in your Volume.	5 · STAGE Upload to a Snowflake stage.	6 · TABLE COPY INTO, then answer in SQL.

Six things that are true for every topic

Follow the generator exactly.

Every brief gives a seed and exact row counts. Use them without changing them. Your counts must match the ones printed on your page — that is how your work is checked, and it is why the seed is not optional.

Bronze does not clean.

Land every column as a string, add your provenance columns, and stop. Every judgement call belongs in Silver, and every judgement call must be written down.

Use `try_cast`, not `cast`.

ANSI mode is on in Free Edition. `cast` throws and kills the cell; `try_cast` gives you a NULL you can count. Counting those NULLs is the point of the exercise.

Namespace everything on Databricks.

You share one workspace, so your schema, your Volume and your table names all carry your ID. Snowflake is your own account — no prefix is needed there.

Run the load twice.

The second COPY INTO of the same file must report zero files. If it loads the rows again, you have not understood load metadata, and that is worth marks.

Schedule it.

Wrap your notebooks in a Databricks Job with a schedule and let it run once on its own. A pipeline you have to start by hand is not a pipeline.

Two conventions the code on every page assumes

VOL is your Volume path. Every code snippet writes to a variable called **VOL**. Define it once, at the top of your first notebook, and never type the path again:

```
MY_ID = "yourname"
VOL =
f"/Volumes/workspace/capstone_{MY_ID}/raw"
```

Create the catalog, schema and Volume first, then create the folders under it that your page names.

Sometimes two tables travel, not one.

Most pages give you a Snowflake query that reads your **GOLD_** table. A few questions need row-level detail that Gold has already summarised away, so their query reads a **SILVER_** table instead. When your page does that, send that Silver table to Snowflake by the same route — one CSV, one stage, one COPY INTO. Two tables travel instead of one. That is normal. Gold is not always a single table.

Where a public dataset is named on your page, check its header row before you write a line of code — public files get re-uploaded by many people and the column names drift. Where no reliable public file exists, the page says so plainly and the generator is the only path. That is not a shortcut: a generated dataset with a published seed is more reproducible than a download.

The thirty-four topics

RETAIL & E-COMMERCE

01	Marketplace Seller Scorecard	COMFORTABLE	p4
02	Return & Refund Leakage	COMFORTABLE	p8
03	Basket Affinity — What Sells Together	STRETCH	p12
04	Price Change Audit Trail	COMFORTABLE	p16
05	Store Footfall vs Sales Conversion	STARTER	p20

FINANCE & FINTECH

06	UPI Transaction Fraud Signals	STRETCH	p24
07	Loan Default Early Warning	STRETCH	p28
08	Stock Daily Returns & Volatility	COMFORTABLE	p32
09	Credit Card Spend Segmentation	COMFORTABLE	p36
10	Reconciliation — Two Systems, One Truth	STRETCH	p40

TRANSPORT & MOBILITY

11	NYC Taxi — Trip Economics	STARTER	p44
12	Bus Fleet Punctuality	COMFORTABLE	p48
13	Ride Cancellation Analysis	COMFORTABLE	p52
14	EV Charging Station Utilisation	COMFORTABLE	p56

HEALTH & PUBLIC SERVICES

15	Hospital Appointment No-Shows	COMFORTABLE	p60
16	Pharmacy Stock-Out Watch	COMFORTABLE	p64
17	Public Grievance Resolution SLA	STARTER	p68
18	Vaccination Coverage by District	STARTER	p72

ENERGY, CLIMATE & AGRI

19	Household Electricity Load Profile	COMFORTABLE	p76
20	Solar Plant Underperformance	COMFORTABLE	p80
21	Rainfall & Crop Yield	COMFORTABLE	p84
22	Air Quality Threshold Breaches	STARTER	p88

MEDIA, EDUCATION & SPORT

23	Streaming Watch-Time & Drop-Off	COMFORTABLE	p92
24	Student Performance & Attendance	STARTER	p96
25	IPL Match & Player Impact	COMFORTABLE	p100
26	Job Postings Skill Demand	STRETCH	p104

IT OPERATIONS & TELEMETRY

27	Web Server Log Error Analysis	COMFORTABLE	p108
28	API Latency SLO Tracking	STRETCH	p112
29	Warehouse Cost Attribution	STRETCH	p116
30	Pipeline Health & Data Quality Dashboard	STRETCH	p120

NETWORKS & TELECOM

31	Cell Tower Health & Dropped Calls	COMFORTABLE	p124
32	The Referral Graph — Who Connects Whom	STRETCH	p128
33	Port Scan Detection in Firewall Logs	STRETCH	p132
34	Interface Bandwidth & the Counter That Wraps	COMFORTABLE	p136

Starter · Comfortable · Stretch describe how much of the work is judgement rather than typing. All three are marked the same way. Choose the theme you would actually enjoy for a week, not the one that sounds hardest.

THE SITUATION

An online marketplace pays sellers monthly and wants to rank them, but no single column says who is good. A seller with high revenue and a 30% late-delivery rate is a liability; a small seller who never misses is an asset. You have to build the judgement the business cannot express in one column.

WHERE THE DATA COMES FROM

Olist Brazilian E-Commerce Public Dataset (Kaggle, ~100k orders). You need four files: `olist_orders_dataset.csv`, `olist_order_items_dataset.csv`, `olist_sellers_dataset.csv`, `olist_order_reviews_dataset.csv`. Column names in the real files may differ slightly from the notes below — check the header row before you write any code, and do not assume.

GENERATE IT — THE EXACT SPECIFICATION

Seed 42; every count below is exact. sellers: 40, S001-S040, one of 8 states by id mod 8. Each seller carries a hidden $\text{late_rate} = 0.02 + 0.33 * (\text{index} / 39)$, so S001 runs 2 percent late, S040 35. orders: 6,000, 0000000-0005999, seller = id mod 40 so 150 orders each, purchase date = $2025-01-01 + \text{floor}(\text{floor}(\text{id}/40) * 181 / 150)$, which gives every seller the same 26, 23, 26, 25, 26, 24 orders from January to June. `estimated_delivery_at` = purchase + 7 days; a late order arrives 8-11 days after purchase, an on-time one 2-5. The ten sellers numbered 1 mod 4 (S001, S005 ... S037) have that rate tripled from April on, capped at 0.9 - that escalation is what the review question asks about. `order_items`: 1 + id mod 4 lines per order, 15,000 in all, quantity 1-3, `unit_price` lognormal near 120. `order_reviews`: one per order, score 1-5, a point lower when the order was late. Inject: the last 300 orders (id 5700-5999, the last nine days of June) are still moving: `order_status` 'shipped', `delivered_at` empty; 120 duplicate order rows (id 1000-1119), 6,120 on disk; 90 `order_items` carrying seller_id 'S999', in no master (id 3000-3089, item 1); 200 `order_items` with `unit_price` 'NA' (id 4000-4199, item 1).

GENERATOR CODE – RUN THIS FIRST, EXACTLY AS WRITTEN

This assumes MY_ID and VOL are already set, as on the second page of this document. Do not change the seed or any of the counts — your checks later depend on them.

```
import pyspark.sql.functions as F # generator seed 42; every count below is exact
def W(d, n): d.write.mode('overwrite').option('header', True).csv(f'{VOL}/raw/{n}')
ST = "array('KA','MH','TN','DL','WB','GJ','TG','KL')"
LR = "round(0.02 + 0.33 * (id % 40) / 39, 3)" # hidden: S001 2% late .. S040 35%
W(spark.range(40).selectExpr("format_string('S%03d', id + 1) as seller_id",
    f"element_at({ST}, cast(id % 8 as int) + 1) as seller_state",
    "format_string('Hub%02d', id % 8 + 1) as seller_city"), 'sellers')
o = spark.range(6000).selectExpr("id", "format_string('O%06d', id) as order_id",
    "format_string('S%03d', id % 40 + 1) as seller_id",
    "format_string('C%05d', pmod(hash(id, 1), 4000)) as customer_id",
    "date_add(date'2025-01-01', cast(cast(id / 40 as int) * 181 / 150 as int)) as dy",
    f"{LR} as lr")
o = o.selectExpr("id", "id >= 5700 as undel", "cast(dy as timestamp) as ts",
    "if(month(dy) >= 4 and id % 4 = 0, least(0.9, lr * 3), lr) as lr2")
o = o.selectExpr("id", "pmod(hash(id, 2), 1000) < lr2 * 1000 as is_late")
o = o.selectExpr("id", "order_id", "seller_id", "customer_id", "is_late", "ts",
    "if(undel, 'shipped', 'delivered') as order_status",
    "timestampadd(day, 7, ts) as estimated_delivery_at",
    "if(undel, null, timestampadd(day, if(is_late, 8, 2)"
    " + pmod(hash(id, 3), 4), ts)) as delivered_at")
od = o.selectExpr("id", "order_id", "customer_id", "order_status",
    "ts as order_purchase_ts", "estimated_delivery_at", "delivered_at")
W(od.unionByName(od.filter('id between 1000 and 1119')).drop('id'), 'orders')
it = o.withColumn('item_no', F.expr('explode(sequence(1, cast(id % 4 + 1 as int))))')
W(it.selectExpr("order_id", "item_no",
    "if(id between 3000 and 3089 and item_no = 1, 'S999', seller_id) as seller_id",
    "format_string('PR%04d', pmod(hash(id, item_no, 4), 800)) as product_id",
    "pmod(hash(id, item_no, 5), 3) + 1 as quantity",
    "if(id between 4000 and 4199 and item_no = 1, 'NA',"
    " string(round(exp(4.79 + randn(42) * 0.5), 2))) as unit_price",
    "round(15 + pmod(hash(id, item_no, 6), 4000) / 100, 2) as freight_value"), 'order_items')
W(o.selectExpr("format_string('RV%06d', id) as review_id", "order_id",
    "if(is_late, greatest(1, pmod(hash(id, 7), 5)), pmod(hash(id, 7), 5) + 1) as review_score",
    "timestampadd(day, 12, ts) as review_ts"), 'order_reviews')
```

BRONZE – LAND IT, CHANGE NOTHING

Land all four files exactly as they arrive. Every column stays a string. Add three provenance columns to each table: `_source_file` (`input_file_name()`), `_ingested_at` (`current_timestamp()`), `_row_hash` (a hash of the concatenated raw columns — you will use it to find duplicates in Silver). Do not cast, trim, or filter anything here. If a row is garbage, it is garbage in Bronze too.

SILVER – THE DECISIONS YOU MUST MAKE, AND RECORD

Each decision below has a number attached to it. If your number does not match, stop and find out why before you go on — that gap is the finding, not a nuisance.

- 1. CAST:** quantity to INT, unit_price and freight_value to DOUBLE, all four timestamps to TIMESTAMP — every one with `try_cast`, never cast. Then count the NULLs each cast produced and record the number. The 200 'NA' prices become 200 NULLs, and that count is a finding, not a nuisance.
- 2. DEDUPE:** order_id is the key. Keep the row with the latest `_ingested_at` using `ROW_NUMBER` over a window; when the timestamps tie, keep the first. Orders fall from 6,120 to exactly 6,000 - any other number and your key is wrong. Record the before and after counts.
- 3. ORPHANS:** exactly 90 order_items carry a seller_id that matches no master row, so they cannot be attributed to anyone. Move those 90 into a `silver_rejects` table with a reason column rather than deleting them: 15,000 items on disk leaves 14,910.
- 4. LATE:** define late once, in Silver, as `delivered_at > estimated_delivery_at`. Not in Gold, and not three times in three queries. The 300 orders whose `delivered_at` is NULL are not late - they are undelivered, which is a different column and a different conversation.

GOLD – THE SHAPE THAT TRAVELS

One row per seller per month. Columns: seller_id, seller_state, month (first day of the month), orders, revenue, late_orders, late_rate, avg_review_score. Exactly $40 \times 6 = 240$ rows, and orders reads 26, 23, 26, 25, 26, 24 from January to June for every seller. Small enough to travel as a CSV, which is the whole point.

THE TRAP MOST PEOPLE FALL INTO

Almost everyone computes late_rate as `AVG(late_flag)` in Gold and then averages those averages across months to get a seller total. That is wrong — it weights a 3-order month the same as a 300-order month. Sum the numerators and denominators separately.

THE THREE QUESTIONS YOUR GOLD LAYER MUST ANSWER

1. Top 10 sellers by revenue per month, and how each one's rank moved from the previous month
2. Share of orders delivered late, per seller per state
3. Which sellers' average review score falls in the months their late rate rises?

IN SNOWFLAKE – THE HARD ONE, WORKED

Tables this query needs in Snowflake: **GOLD_SELLER_MONTH**

```
-- Q1: rank inside each month, then LAG the rank to see the movement.
-- Rank in a CTE first: LAG must see the months a seller was outside the top 10.
WITH r AS (
  SELECT month, seller_id, revenue,
         RANK() OVER (PARTITION BY month ORDER BY revenue DESC) AS rank_now
  FROM   GOLD_SELLER_MONTH
)
SELECT month, seller_id, revenue, rank_now,
       LAG(rank_now) OVER (PARTITION BY seller_id ORDER BY month) AS rank_prev,
       LAG(rank_now) OVER (PARTITION BY seller_id ORDER BY month)
       - rank_now AS places_gained
FROM   r
QUALIFY rank_now <= 10
ORDER BY month, rank_now;
```

YOU ARE DONE WHEN

- ☐ Bronze: orders 6,120, order_items 15,000, sellers 40, reviews 6,000. Compare each with its file's line count minus one header line and print both numbers.
- ☐ Silver orders = 6,120 - 120 duplicates = 6,000. Silver items = 15,000 - 90 orphans = 14,910, of which exactly 200 have a NULL unit_price. Both subtractions must close exactly.
- ☐ Gold has exactly 240 rows, one per seller per month, no NULL revenue, and orders reads 26, 23, 26, 25, 26, 24 across January to June on every seller.
- ☐ SUM(revenue) in Gold equals SUM(quantity * unit_price) over Silver items that are neither rejected nor NULL-priced, to a rounding cent. Roughly double on 120 orders means you joined items to orders before deduping.
- ☐ S001, S005 ... S037 - the ten sellers numbered 1 mod 4 - show a higher late_rate and a lower avg_review_score from April on while the other 30 stay flat. If all 40 move, your late flag is counting a NULL delivered_at as late.

IF YOU FINISH EARLY

Add a two-month trailing late_rate using a window frame, and flag the month a seller first crosses 20%. That is the month the business would have wanted an alert.

THE SITUATION

Returns destroy margin quietly. Finance sees one refund number a month and argues; nobody can say which product line, month or buyer produced it. Turn that number into a table naming the cause. The hard part is deciding what a negative quantity in a sales file means, and writing it down once so every later query agrees.

WHERE THE DATA COMES FROM

No free retail dataset ships a refund flag, refund value and category together. Closest is Online Retail II (UCI): cancellations are invoices starting 'C' with negative quantities - cancellations, not refunds, no category column, names differing between the workbook's two sheets, so confirm the header before coding. Only the generator gives a refund value you can check against.

GENERATE IT – THE EXACT SPECIFICATION

Seed 7; every count below is exact. products: 900, P0000-P0899, category = id mod 12 (75 each), unit_price = round(exp(4.6 + randn*0.55), 2). orders: 12,000. Ids 0-7,999 are each customer's first order, dated 2025-01-01 + ((29*id) mod 150); ids 8,000-11,999 are the same customer's repeat order, exactly 30 days later - so 8,000 customers, 4,000 of whom come back, and a buyer_type you can check your own logic against. sales_lines: 1 + id mod 4 lines per order, 3,000 orders of each size, 30,000 lines; order_id, line_no, order_date, customer_id, product_id, quantity 1-5, unit_price from the master. returns: line 1 of every first order whose id is a multiple of 4 (2,000) and of every repeat order whose id is a multiple of 10 (400), so 2,400 in all - quantity negative, return_date 1-30 days after the order, reason code. Two thirds of them, 1,600, land on P0000-P0029, which is where the leakage lives. Inject: 300 duplicate lines (disk 30,300); 180 returns whose order_id is in no sales line (disk 2,580); 500 unit_price 'NA'; 150 sales rows with a negative quantity and no return - till keying errors.

GENERATOR CODE – RUN THIS FIRST, EXACTLY AS WRITTEN

This assumes MY_ID and VOL are already set, as on the second page of this document. Do not change the seed or any of the counts — your checks later depend on them.

```
import pyspark.sql.functions as F # generator seed 7; every count below is exact
def W(d, n): d.write.mode('overwrite').option('header', True).csv(f'{VOL}/raw/{n}')
p = spark.range(900).selectExpr("id as pid", "format_string('P%04d', id) as product_id",
    "format_string('CAT%02d', id % 12) as category",
    "round(exp(4.6 + randn(7) * 0.55), 2) as up")
W(p.selectExpr("product_id", "category", "up as unit_price"), 'products')
a = spark.range(8000).selectExpr("id", "pmod(29 * id, 150) as dy") # first orders
b = spark.range(8000, 12000).selectExpr("id", "pmod(29 * (id - 8000), 150) + 30 as dy")
o = a.unionByName(b).selectExpr("id", "format_string('O%06d', id) as order_id",
    "format_string('C%05d', if(id < 8000, id, id - 8000)) as customer_id",
    "date_add(date'2025-01-01', cast(dy as int)) as order_date",
    "if(id < 8000, id % 4 = 0, id % 10 = 0) as ret") # 2,000 + 400 returned orders
ln = o.withColumn('line_no', F.expr('explode(sequence(1, cast(id % 4 + 1 as int)))'))
ln = ln.selectExpr("*, 'if(ret and line_no = 1 and id % 5 < 3, "
    "pmod(hash(id, 8), 30), pmod(hash(id, line_no, 3), 900)) as pid")
sl = ln.join(p.drop('category'), 'pid').selectExpr("id", "order_id", "line_no",
    "order_date", "customer_id", "product_id", "ret",
    "if(id % 4 = 1 and id < 600 and line_no = 1, -1, 1)"
    " * (pmod(hash(id, line_no, 5), 5) + 1) as quantity",
    "if(id % 4 = 3 and id < 2000 and line_no = 4, 'NA', string(up)) as unit_price")
dup = sl.filter('id % 4 >= 2 and id < 600 and line_no = 3')
W(sl.unionByName(dup).drop('id', 'ret'), 'sales_lines')
RS = "array('damaged','wrong size','not as described','late delivery','changed mind')"
r = sl.filter('ret and line_no = 1').selectExpr("id", "order_id", "line_no",
    "format_string('R%06d', id) as return_id", "-quantity as quantity",
    "date_add(order_date, 1 + pmod(hash(id, 6), 30)) as return_date",
    f"element_at({RS}, pmod(hash(id, 7), 5) + 1) as reason_code")
orp = spark.range(180).selectExpr("id", "format_string('O9%05d', id) as order_id",
    "cast(1 as int) as line_no", "format_string('R9%05d', id) as return_id",
    "-(pmod(hash(id, 5), 5) + 1) as quantity",
    "date_add(date'2025-03-01', cast(id % 60 as int)) as return_date",
    f"element_at({RS}, pmod(hash(id, 7), 5) + 1) as reason_code")
W(r.unionByName(orp).drop('id'), 'returns')
```

BRONZE – LAND IT, CHANGE NOTHING

Land all three files as they arrive, every column a string - 500 unit_price values are the word 'NA', and a typed read would fail the job or drop rows. Add _source_file (input_file_name()), _ingested_at (current_timestamp()) and _row_hash (a hash of the raw columns, which finds the 300 duplicates).

SILVER – THE DECISIONS YOU MUST MAKE, AND RECORD

Each decision below has a number attached to it. If your number does not match, stop and find out why before you go on — that gap is the finding, not a nuisance.

1. CAST quantity to INT and unit_price to DOUBLE with try_cast, never cast: ANSI mode is on and a plain cast kills the job. Exactly 500 unit_price values become NULL. Write that 500 into a data quality table - it is a finding you report, not a nuisance you hide.
2. DEDUPE on (order_id, line_no) with ROW_NUMBER over a window ordered by _ingested_at desc: sales lines fall 30,300 -> 30,000; any other number means your key is wrong. ORPHANS: the 180 returns whose order_id has no sales line go to silver_rejects, reason='orphan_return'.
3. NEGATIVE QUANTITY, the decision this project exists for: negative in returns is a genuine refund, so take abs(quantity); negative in sales with no matching return is a till keying error, so send exactly those 150 rows to silver_rejects, reason='negative_qty_no_return'.
4. REFUND VALUE, once: abs(returned_quantity) x the unit_price on the parent sales line, not the price on the return row - prices move and 500 prices are NULL. Define buyer_type here too: a customer is 'repeat' on an order if they have an earlier order_date, else 'first_time'.

GOLD – THE SHAPE THAT TRAVELS

One row per product per month per buyer_type: product_id, category, month (month start), buyer_type ('first_time'/'repeat'), units_sold, gross_revenue, units_returned, refund_value, refund_share (refund_value/gross_revenue). No first order falls in June, so $900 \times 11 = 9,900$ cells are possible; expect a little under 9,000 rows and print the number.

THE TRAP MOST PEOPLE FALL INTO

Almost everyone leaves the 150 negative-quantity sales rows in the revenue calculation. Gross revenue is then already net of them, so subtracting refund_value counts the same money twice; the tell is a low refund_share in the categories with the most negative rows. Sum gross revenue over positive quantities only.

THE THREE QUESTIONS YOUR GOLD LAYER MUST ANSWER

1. Refund value as a share of gross revenue, per category per month
2. Which products make up the first 50% of total refund value?
3. Refund rate for first-time buyers compared with repeat buyers

IN SNOWFLAKE – THE HARD ONE, WORKED

Tables this query needs in Snowflake: **GOLD_PRODUCT_MONTH**

```
-- Q2 Pareto: running total over products by refund value, cut at 50% with QUALIFY.
WITH p AS (SELECT product_id, category, SUM(refund_value) rv
            FROM GOLD_PRODUCT_MONTH GROUP BY 1, 2)
SELECT product_id, category, rv,
       SUM(rv) OVER (ORDER BY rv DESC
                     ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS cum_refund,
       RATIO_TO_REPORT(rv) OVER () AS share_of_total,
       ROW_NUMBER() OVER (ORDER BY rv DESC) AS rn FROM p
QUALIFY cum_refund - rv < 0.5 * SUM(rv) OVER ()
ORDER BY rn;
```

YOU ARE DONE WHEN

- ☐ Bronze sales_lines = 30,300 and Bronze returns = 2,580; compare each with the file line count minus one header.
- ☐ Silver sales_lines = 30,300 - 300 duplicates - 150 keying errors = 29,850; Silver returns = 2,580 - 180 orphans = 2,400. The subtraction must close.
- ☐ Gold SUM(gross_revenue) = SUM(quantity*unit_price) over Silver lines with non-NULL price, within a rupee; SUM(refund_value) = the same over the 2,400 accepted returns.
- ☐ P0000-P0029 hold about two thirds of all refund value: every one of the top 10 products by refund value comes from that block and the Q2 cut at 50 percent names about 20. A cut naming hundreds means your running total is not ordered by refund value descending.
- ☐ First-time buyers return about 10 percent of their lines against about 4 percent for repeat buyers. If the two rates come out equal, buyer_type was decided per row instead of per customer's earliest order_date.

IF YOU FINISH EARLY

Add a three-month trailing refund_share per product with ROWS BETWEEN 2 PRECEDING AND CURRENT ROW, and flag the first month each product crosses 15%.

THE SITUATION

Two products that always share a basket should not sit at opposite ends of the shop. Layout wants ranked pairs with a number saying how much stronger the pairing is than chance. Counting pairs is easy; counting them right is not - a one-item basket is still a basket, a cancelled basket is not a sale.

WHERE THE DATA COMES FROM

Online Retail II (UCI), real UK online-retail transactions, is the standard choice. Warnings: it is an Excel workbook of two sheets covering two periods, so convert to CSV; names differ across sheets and mirrors (Invoice/InvoiceNo, Price/UnitPrice, 'Customer ID' with a space), so confirm the header; cancellations are invoices starting 'C'.

GENERATE IT — THE EXACT SPECIFICATION

Seed 11; every count below is exact. products: 300, P000-P299, aisle = id div 20, so A00-A14 with 20 products each. baskets: 42,000, size 2 + id mod 5 (8,400 of each size 2-6, 168,000 lines); basket_ts = 2025-01-06 08:00 plus floor(id*2016/42000) hours, an even sweep of 12 weeks in which exactly 12,000 baskets fall on a Saturday or Sunday. A basket walks a route: its slots take consecutive aisles, one product from each, so no basket lists a product twice by accident. Plant into slots 1-2, adding no rows: the 14,000 baskets with id mod 3 = 0 take pair k = id mod 5 of (P011,P021), (P045,P075), (P100,P130), (P177,P198), (P250,P270) - 2,800 baskets each, 2,000 weekday and 800 weekend; the weekend baskets with id mod 3 = 1 take (P265,P285), exactly 4,000. Every fault below sits on id mod 3 = 2, so none touches a planted pair. Inject: 400 whole baskets re-sent (id mod 105 = 11, 3 lines each, disk 169,200); 700 baskets whose slot 3 repeats their own slot 1 product (id mod 60 = 2); 400 rows with product_id 'P999', in no master (id mod 105 = 8, slot 1); 1,400 quantity 'NA' (id mod 30 = 5, slot 2); 350 baskets marked 'CANCEL' with negative quantities (id mod 120 = 14, 2,100 lines).

GENERATOR CODE – RUN THIS FIRST, EXACTLY AS WRITTEN

This assumes MY_ID and VOL are already set, as on the second page of this document. Do not change the seed or any of the counts — your checks later depend on them.

```
import pyspark.sql.functions as F # generator seed 11; every count below is exact
def W(d, n): d.write.mode('overwrite').option('header', True).csv(f'{VOL}/raw/{n}')
TS = "timestamp'2025-01-06 08:00:00'"
PA = "array(11, 45, 100, 177, 250)" # planted pair, first member (aisle x)
PB = "array(21, 75, 130, 198, 270)" # planted pair, second member (aisle x + 1)
W(spark.range(300).selectExpr("format_string('P%03d', id) as product_id",
    "format_string('A%02d', cast(id / 20 as int)) as aisle",
    "format_string('SKU-%03d', id) as product_name"), 'products')
b = spark.range(42000).selectExpr("id", "format_string('B%06d', id) as basket_id",
    "cast(id % 5 + 2 as int) as sz", "cast(id % 5 as int) as k",
    f"timestampadd(hour, cast(id * 2016 / 42000 as int), {TS}) as basket_ts")
b = b.selectExpr("id", "dayofweek(basket_ts) in (1, 7) as wknd")
b = b.selectExpr("id", f"case when id % 3 = 0 then element_at({PA}, k + 1)"
    " when id % 3 = 1 and wknd then 265 else -1 end as pa")
b = b.selectExpr("id", f"case when id % 3 = 0 then element_at({PB}, k + 1)"
    " when pa = 265 then 285 else -1 end as pb",
    "if(pa >= 0, pmod(cast(pa / 20 as int) - 1, 15), pmod(hash(id, 99), 15)) as a0")
ln = b.withColumn('slot', F.expr('explode(sequence(1, sz))'))
ln = ln.selectExpr("id", "pmod(a0 + slot, 15) * 20 + pmod(hash(id, slot), 20) as p0")
ln = ln.selectExpr("id", "case when pa >= 0 and slot = 1 then pa"
    " when pa >= 0 and slot = 2 then pb else p0 end as p1")
ln = ln.selectExpr("id", "if(id % 60 = 2 and slot = 3," # 700 same-product slots
    " pmod(a0 + 1, 15) * 20 + pmod(hash(id, 1), 20), p1) as p2")
ln = ln.selectExpr("id", "basket_id", "slot",
    "if(id % 105 = 8 and slot = 1, 'P999', format_string('P%03d', p2)) as product_id",
    "if(id % 30 = 5 and slot = 2, 'NA', string(if(id % 120 = 14, -1, 1))"
    " * (pmod(hash(id, slot, 9), 3) + 1)) as quantity",
    "basket_ts", "if(id % 120 = 14, 'CANCEL', 'SALE') as txn_type")
W(ln.unionByName(ln.filter('id % 105 = 11')).drop('id'), 'basket_lines')
```

BRONZE — LAND IT, CHANGE NOTHING

Land both files with every column a string, quantity included - 1,400 values are the word 'NA'. Add `_source_file`, `_ingested_at` and `_row_hash`. Keep `txn_type` as written, 'CANCEL' included: do not filter cancellations here, the till really sent them.

SILVER — THE DECISIONS YOU MUST MAKE, AND RECORD

Each decision below has a number attached to it. If your number does not match, stop and find out why before you go on — that gap is the finding, not a nuisance.

1. CAST quantity to INT with `try_cast`: exactly 1,400 NULLs appear. Cast `basket_ts` to `TIMESTAMP` and derive `day_type` here, once - 'weekend' on Saturday and Sunday, else 'weekday' - so the weekday-versus-weekend question is later a `GROUP BY`, not a rewrite.
2. DEDUPE on (`basket_id`, `product_id`), not (`basket_id`, `slot`): 400 baskets were re-sent whole, and 700 more really do list one product in two slots, which would pair that product with itself. Lines fall from 169,200 to exactly 167,300. Keying on `slot` removes only the 1,200 re-sent rows and leaves 168,000. Print before and after.
3. REJECT: exactly 400 rows carry `product_id` 'P999', absent from the master, so no aisle and no name - `silver_rejects`, `reason='unknown_product'`, never in a pair. Exactly 350 baskets are `txn_type='CANCEL'`: drop all 2,100 of their lines before pairing and record the number.
4. BASKET DEFINITION: a basket is one distinct `basket_id` after cleaning, and 41,650 survive - 29,750 weekday, 11,900 weekend. A basket holding neither product of a pair adds nothing to that pair's numerator but stays in the denominator of support and lift: a real trip where the pair did not happen. Compute `total_baskets` once, one value per `day_type`.

GOLD — THE SHAPE THAT TRAVELS

Per unordered pair per `day_type`: `product_a`, `product_b`, `day_type`, `pair_baskets`, `baskets_a`, `baskets_b`, `total_baskets`, `support=pair/total`, `confidence_a_to_b=pair/baskets_a`, `lift=support/((baskets_a/total)*(baskets_b/total))`. `product_a<product_b`; keeping pairs with `pair_baskets>=5` leaves roughly 20,000 rows - print the number you get.

THE TRAP MOST PEOPLE FALL INTO

The self-join. Written `x.product_id <> y.product_id` it returns every pair twice, (A,B) and (B,A), doubling every count and every lift; with no predicate at all it joins rows to themselves and A-with-A tops the list. Use `x.product_id < y.product_id`. Second half: `total_baskets` must be every surviving basket, not only the baskets that hold one of the pair.

THE THREE QUESTIONS YOUR GOLD LAYER MUST ANSWER

1. Top 20 product pairs by co-occurrence count
2. Lift per pair - how much more often than chance the two appear together
3. Do the top pairs change between weekday and weekend?

IN SNOWFLAKE — THE HARD ONE, WORKED

Tables this query needs in Snowflake: **SILVER_BASKET_LINES**

```
-- Self-join on basket_id with x.product_id < y.product_id: each pair once.
WITH bl AS (SELECT DISTINCT basket_id, day_type, product_id, total_baskets
             FROM SILVER_BASKET_LINES)
SELECT x.day_type, x.product_id AS product_a, y.product_id AS product_b,
       COUNT(*) AS pair_baskets, MAX(x.total_baskets) AS total_baskets
FROM   bl x JOIN bl y ON x.basket_id = y.basket_id
       AND x.product_id < y.product_id
GROUP BY 1, 2, 3
QUALIFY ROW_NUMBER() OVER (PARTITION BY x.day_type
                           ORDER BY COUNT(*) DESC) <= 20
ORDER BY x.day_type, pair_baskets DESC;
```

YOU ARE DONE WHEN

- ☐ Bronze basket_lines = 169,200. Dedupe on (basket_id, product_id) leaves exactly 167,300; the 400 'P999' rejects leave 166,900; dropping 2,100 cancelled lines leaves 164,800. Print all four numbers as one line of arithmetic.
- ☐ Each of the five planted pairs, summed over both day_types, shows pair_baskets of at least 2,800 - about 2,000 weekday and 800 weekend - and a lift above 5. A missing pair means a wrong dedupe key or a wrong pair filter.
- ☐ (P265,P285) has 4,000 weekend baskets and almost no weekday ones: a large weekend lift and a weekday lift near 1. Print both; if its weekday row falls below the Gold cutoff, take that count from Silver.
- ☐ total_baskets is 29,750 for weekday and 11,900 for weekend, out of 41,650 surviving baskets, and is identical on every Gold row of its day_type. Smaller means the cancelled baskets came out twice, or a reject took its whole basket with it.
- ☐ Dedupe on (basket_id, slot) instead and you get 168,000 lines and 700 baskets pairing a product with itself, most of them at the top of the Q1 list. That is the tell.

IF YOU FINISH EARLY

Compute lift per calendar week instead of per day_type, then LAG over weeks to find the pair whose lift rose most from first week to last.

THE SITUATION

Someone changed about 900 prices last Tuesday. The product master holds one row per product, it was overwritten, and there is no backup. Finance wants to know how many prices moved, by how much, which categories were hit hardest, and wants the old list back. The project proves versioned storage replaces a backup - and shows two ways students destroy the history they need.

WHERE THE DATA COMES FROM

There is no public dataset for this and you should not look for one: you control both versions, so you know the right answer before you compute it. You may borrow StockCode, Description and Price from Online Retail II (UCI) for realistic names and prices, but its column names differ across sheets and mirrors and it has no before-and-after at all.

GENERATE IT – THE EXACT SPECIFICATION

Seed 23; every count below is exact. V1: 5,000 rows, P00001-P05000, columns product_id, category, brand, price, cost, mrp, updated_by, updated_at; category = id mod 10 (500 each), brand = id mod 40, price = round(exp(6.1 + randn*0.45), 2), cost = 0.62*price, mrp = 1.25*price, updated_at '2025-07-01 09:00:00', updated_by 'INIT'. V2 is built from V1, so a price nobody touched is byte-identical in both files: ids 1-600 multiplied by 1.03 to 1.119 (600 up); 601-850 by 0.80 to 0.949 (250 down); 851-900 by exactly 10 (50 fat-finger); 4,961-5,000 dropped (40 delisted); 60 new rows P05001-P05060. Every row the upload touched - the 900 repriced, the 110 broken below and the 60 new - carries updated_at '2025-07-08 14:30:00' and updated_by 'BULK_UPLOAD', which is 1,070 rows. V2 = 5,000 - 40 + 60 = 5,020. Inject into V2: 120 duplicate rows (ids 2,001-2,120, disk 5,140); an empty-string price on ids 1,001-1,080 (80); a negative price on ids 1,081-1,110 (30).

GENERATOR CODE – RUN THIS FIRST, EXACTLY AS WRITTEN

This assumes MY_ID and VOL are already set, as on the second page of this document. Do not change the seed or any of the counts — your checks later depend on them.

```
import pyspark.sql.functions as F # generator seed 23; every count below is exact
def W(d, n): d.write.mode('overwrite').option('header', True).csv(f'{VOL}/raw/{n}')
C = ['product_id', 'category', 'brand', 'price', 'cost', 'mrp', 'updated_by', 'updated_at']
v1 = spark.range(1, 5001).selectExpr("id", "format_string('P%05d', id) as product_id",
    "format_string('CAT%02d', id % 10) as category",
    "format_string('BR%02d', id % 40) as brand",
    "round(exp(6.1 + randn(23) * 0.45), 2) as p")
v1 = v1.selectExpr("string(p) as price", "round(p * 0.62, 2) as cost",
    "round(p * 1.25, 2) as mrp", "'INIT' as updated_by",
    "'2025-07-01 09:00:00' as updated_at")
W(v1.select(*C), 'pm_v1')
UP = "1.03 + pmod(hash(id, 24), 90) / 1000" # ids 1-600 go up 3% to 11.9%
DN = "0.80 + pmod(hash(id, 25), 150) / 1000" # ids 601-850 go down 20% to 5.1%
NP = (f"case when id <= 600 then round(p * ({UP}), 2)"
    f" when id <= 850 then round(p * ({DN}), 2)"
    f" when id <= 900 then round(p * 10, 2) else p end") # 851-900 fat-finger x10
T = "id <= 900 or id between 1001 and 1110" # rows the bulk upload touched
v2 = v1.filter('id <= 4960').selectExpr("id", "product_id", "category", "brand",
    f"{NP} as np", "cost", "mrp",
    f"if({T}, 'BULK_UPLOAD', 'INIT') as updated_by",
    f"if({T}, '2025-07-08 14:30:00', '2025-07-01 09:00:00') as updated_at")
v2 = v2.selectExpr("id", "product_id", "category", "brand",
    "case when id between 1001 and 1080 then '' # 80 empty-string prices
    " when id between 1081 and 1110 then string(-np) # 30 negative prices
    " else string(np) end as price", "cost", "mrp", "updated_by", "updated_at")
nw = spark.range(5001, 5061).selectExpr("id", "format_string('P%05d', id) as product_id",
    "format_string('CAT%02d', id % 10) as category",
    "format_string('BR%02d', id % 40) as brand",
    "round(exp(6.1 + randn(26) * 0.45), 2) as p")
nw = nw.selectExpr("id", "product_id", "category", "brand", "string(p) as price",
    "round(p * 0.62, 2) as cost", "round(p * 1.25, 2) as mrp",
    "'BULK_UPLOAD' as updated_by", "'2025-07-08 14:30:00' as updated_at")
out = v2.unionByName(nw) # 4,960 + 60 = 5,020 rows
W(out.unionByName(out.filter('id between 2001 and 2120')).select(*C), 'pm_v2')
```

BRONZE – LAND IT, CHANGE NOTHING

Load V1 into a Bronze table, all columns strings, with `_source_file`, `_ingested_at` and `_load_id`. Load V2 into the SAME table by overwriting contents - INSERT OVERWRITE, or TRUNCATE then COPY INTO - never drop and recreate. Time Travel is attached to the object, not the name. Note the exact timestamp before you overwrite.

SILVER – THE DECISIONS YOU MUST MAKE, AND RECORD

Each decision below has a number attached to it. If your number does not match, stop and find out why before you go on — that gap is the finding, not a nuisance.

1. CAST price, cost and mrp to DECIMAL(10,2) and updated_at to TIMESTAMP, all try_cast never cast: the 80 empty-string prices become exactly 80 NULLs, recorded in a data quality table before you touch the column.
2. DEDUPE on product_id: V2 falls from 5,140 rows on disk to exactly 5,020, keeping the latest _ingested_at with ROW_NUMBER over a window. Exactly 30 rows carry a negative price - broken, not a discount: all 30 to silver_rejects, reason='negative_price', never clamped to zero.
3. DEFINE A PRICE CHANGE once: $\text{abs}(\text{price_after} - \text{price_before}) > 0.005$. A product in V2 but not V1 is 'new'; in V1 but not V2 it is 'delisted'. Neither is a change: count them as changes and 900 becomes 1,000. A V2 price that is NULL or rejected is 'unpriced', not delisted.
4. FLAG FAT-FINGER: $\text{price_after} / \text{price_before} \geq 5$ is a probable misplaced decimal point, and you should get exactly 50. They go into the report on their own line: a 10x price is an incident, not a repricing.

GOLD – THE SHAPE THAT TRAVELS

Per product_id: product_id, category, brand, price_before, price_after, abs_change, pct_change, changed_at, change_type (increase, decrease, fat_finger, new, delisted, unpriced, unchanged). 5,060 rows: 600 up, 250 down, 50 fat_finger, 60 new, 40 delisted, 110 unpriced, 3,950 unchanged.

THE TRAP MOST PEOPLE FALL INTO

Students load V2 with CREATE OR REPLACE TABLE or DROP plus CREATE. That replaces the object, and Time Travel belongs to the object, so AT and BEFORE cannot reach the old rows; UNDROP helps only if you dropped. Overwrite contents instead. Second half: a Standard trial keeps one day - build Monday, recover Thursday, nothing there.

THE THREE QUESTIONS YOUR GOLD LAYER MUST ANSWER

1. How many prices changed, and by how much, between the two loads?
2. Which categories were repriced hardest?
3. Recover the pre-change price list without a backup

IN SNOWFLAKE – THE HARD ONE, WORKED

Tables this query needs in Snowflake: **PRODUCT_MASTER**, **PRODUCT_MASTER_BEFORE**

```
-- Q3: Time Travel recovers the pre-change list; a FULL OUTER JOIN diffs it.
-- A Standard trial keeps 1 day of history, so recover on the same day you overwrite.
CREATE OR REPLACE TABLE PRODUCT_MASTER_BEFORE AS SELECT * FROM PRODUCT_MASTER
  BEFORE (STATEMENT => '<query id of your V2 overwrite, from Query History>');
SELECT COALESCE(b.category, a.category) AS category,
       COUNT(*) AS products,
       COUNT_IF(ABS(a.price - b.price) > 0.005) AS repriced,
       ROUND(AVG(CASE WHEN ABS(a.price - b.price) > 0.005
                     THEN (a.price - b.price) / NULLIF(b.price, 0) * 100 END), 2) AS avg_pct_change
FROM   PRODUCT_MASTER_BEFORE b
FULL OUTER JOIN PRODUCT_MASTER a ON a.product_id = b.product_id
GROUP BY 1
ORDER BY repriced DESC;
```

YOU ARE DONE WHEN

- ☐ 600 + 250 + 50 = 900 changed prices: the Gold change_type counts must add to 900 changed and to exactly 5,060 rows overall.
- ☐ PRODUCT_MASTER_BEFORE has exactly 5,000 rows and its prices for products 851-900 are one tenth of the current ones. That check alone proves Time Travel worked.
- ☐ V2 Bronze 5,140, minus 120 duplicates = 5,020, minus the 30 negative-price rows sent to silver_rejects = 4,990 in Silver, with 80 NULL prices in the DQ table.
- ☐ The 40 delisted products appear only in PRODUCT_MASTER_BEFORE and the 60 new ones only in the current table. Count both and confirm 40 and 60.
- ☐ updated_by = 'BULK_UPLOAD' on exactly 1,070 V2 rows: 900 repriced + 110 broken + 60 new. If you count 5,020, the load stamped every row instead of the ones it changed.

IF YOU FINISH EARLY

Turn the two versions into an SCD type 2 - one row per product per price period with valid_from, valid_to, is_current - then answer by date range instead of Time Travel.

THE SITUATION

Footfall is climbing and sales are flat, and the store manager cannot say why: his footfall counter and his billing system have never sat in one table. Put them at one grain and compute conversion. There is no clever technique here - the value is honest medallion layers, because a dead sensor reporting zero visitors wrecks the answer.

WHERE THE DATA COMES FROM

Footfall paired with bill counts is almost never published, so do not spend a day hunting. The nearest real thing is Rossmann Store Sales (Kaggle), whose Customers column is a day-grain proxy per store with no hourly detail and whose column names have varied across versions - check the header. It cannot answer the hourly questions below.

GENERATE IT – THE EXACT SPECIFICATION

Seed 5; every count below is exact. stores: 12, ST01-ST12, city = id mod 6, format Mall on even ids and High Street on odd, plus a hidden conv_base = round(0.08 + rand*0.20, 3) that never reaches the file. footfall: 12 stores x 90 dates 2025-04-01 to 2025-06-29 x hours 10-21 = 12,960 rows of store_id, trade_date, hour, counter_status, footfall = round(30 * shape * wk * (0.7 + rand*0.6)), shape 1.8 at hours 18-20, 0.6 at 14-16, 1.0 elsewhere, wk 1.6 on Sat and Sun. bills per store-hour = round(footfall * conv_base), times 0.45 at hours 14-16 and times 0.85 at weekends because weekend crowds browse; explode each into its own row of bill_id, store_id, bill_ts, items 1-8, bill_amount = round(exp(6.0 + randn*0.7), 2). The total lands near 75,000 - print it and reuse that one number. Number the store-hours rn = 0 to 12,959 and inject: counter_status 'OFFLINE' with footfall written as 0 where rn mod 54 = 0 (240), footfall written as 'ERR' where rn mod 54 = 1 or rn mod 216 = 2 (300), both keeping their real bills. Then 130 exact duplicate bills, 75 bills for store ST13 inside trading hours, and 60 bills at 03:00-05:00 for real stores.

GENERATOR CODE – RUN THIS FIRST, EXACTLY AS WRITTEN

This assumes MY_ID and VOL are already set, as on the second page of this document. Do not change the seed or any of the counts — your checks later depend on them.

```
import pyspark.sql.functions as F # generator seed 5; every count below is exact
def W(d, n): d.write.mode('overwrite').option('header', True).csv(f'{VOL}/raw/{n}')
S = "if(hour between 18 and 20, 1.8, if(hour between 14 and 16, 0.6, 1.0))"
K = "if(dayofweek(trade_date) in (1, 7), 1.6, 1.0)" # 1 = Sunday, 7 = Saturday
C = "if(dayofweek(trade_date) in (1,7), .85, 1) * if(hour between 14 and 16, .45, 1)"
st = spark.range(12).selectExpr("int(id) as s", "format_string('ST%02d', id + 1) as store_id",
    "format_string('CITY%d', id%6) as city", "if(id % 2 = 0, 'Mall', 'High Street') as format",
    "round(0.08 + rand(5) * 0.20, 3) as conv_base") # never reaches the file
W(st.select('store_id', 'city', 'format'), 'stores')
gr = (st.crossJoin(spark.range(90).withColumnRenamed('id', 'd'))
    .crossJoin(spark.range(10, 22).withColumnRenamed('id', 'hour'))
    .selectExpr("*, date_add(date'2025-04-01', int(d)) as trade_date"))
gr = gr.selectExpr("*, ((s * 90 + int(d)) * 12 + int(hour) - 10) as rn",
    f"int(round(30 * ({S}) * ({K}) * (0.7 + rand(6) * 0.6))) as fc")
gr = gr.selectExpr("*, f'int(round(fc * conv_base * ({C}))) as nb") # bills per store-hour
OFF = "pmod(rn, 54) = 0" # 240 dead counters
ERR = "(pmod(rn, 54) = 1 or pmod(rn, 216) = 2)" # 240 + 60 = 300 unreadable readings
W(gr.selectExpr("store_id", "trade_date", "hour",
    f"if({OFF}, 'OFFLINE', 'OK') as counter_status",
    f"case when {ERR} then 'ERR' when {OFF} then '0' else string(fc) end as footfall"),
    'footfall')
bl = gr.filter('nb > 0').withColumn('m', F.explode(F.sequence(F.lit(1), F.col('nb'))))
bl = bl.selectExpr("format_string('B%02d%03d%02d%02d', s, int(d), int(hour), m) as bill_id",
    "store_id", "s * 90 + int(d) as k", "m", "hour",
    "format_string('%s %02d:%02d:00', string(trade_date), int(hour), m % 60) as bill_ts",
    "int(1 + rand(7) * 8) as items", "round(exp(6.0 + randn(8) * 0.7), 2) as bill_amount")
B = ['bill_id', 'store_id', 'bill_ts', 'items', 'bill_amount']
dup = bl.filter("hour = 12 and m = 1 and pmod(k, 8) = 3 and k < 1040") # 130 exact copies
ex = spark.range(135).selectExpr("format_string('B9%06d', id) as bill_id",
    "if(id < 75, 'ST13', format_string('ST%02d', id % 12 + 1)) as store_id",
    "format_string('2025-05-%02d %02d:15:00', int(id) % 28 + 1,
    " if(id < 75, 11 + int(id) % 9, 3 + int(id) % 3)) as bill_ts",
    "int(1 + pmod(id, 8)) as items", "round(400 + pmod(id, 900), 2) as bill_amount")
W(bl.select(*B).unionByName(dup.select(*B)).unionByName(ex.select(*B)), 'bills')
```

BRONZE – LAND IT, CHANGE NOTHING

Land all three files with every column a string - 300 footfall values are the word 'ERR'. Add `_source_file`, `_ingested_at` and `_row_hash`. Keep `counter_status` as written: do not replace 'OFFLINE' and do not drop the zero-footfall rows, because Bronze must show what the sensor reported. Bronze footfall = exactly 12,960 rows.

SILVER – THE DECISIONS YOU MUST MAKE, AND RECORD

Each decision below has a number attached to it. If your number does not match, stop and find out why before you go on — that gap is the finding, not a nuisance.

1. CAST footfall to INT and `bill_amount` to DOUBLE with `try_cast`, never cast: exactly 300 footfall values become NULL, recorded in a data quality table. Cast `trade_date` to DATE and `bill_ts` to TIMESTAMP.
2. DEAD SENSOR, the decision the project turns on: where `counter_status`='OFFLINE', set footfall to NULL, not 0 - exactly 240 rows. With the 300 'ERR' rows that is 540 unknown store-hours of 12,960, 4.2%; report it. Left as 0 you divide by zero, or guard it and rank those hours worst.
3. DEDUPE on `bill_id`: exactly 130 duplicate bills disappear, print before and after. Reject the 75 bills with `store_id` 'ST13' (no master row, so no city, no format) as 'unknown_store', and the 60 bills outside the 10:00-21:00 window as 'outside_trading_hours' - never bucket them.
4. DEFINE CONVERSION once: $\text{conversion_rate} = \text{bills} / \text{footfall}$, only where footfall is not NULL and above 0. Every store-hour keeps a `sensor_ok` boolean so later queries drop unknown hours from numerator and denominator without re-deriving the rule.

GOLD – THE SHAPE THAT TRAVELS

One row per store per `trade_date` per hour: `store_id`, `city`, `format`, `trade_date`, `hour`, `is_weekend`, `footfall`, `bills`, `revenue`, `conversion_rate`, `sensor_ok`. 12,960 rows, exactly 12,420 with `sensor_ok`=true and a usable `conversion_rate`. Keep all 12,960 - deleting a broken store-hour hides the outage.

THE TRAP MOST PEOPLE FALL INTO

The 240 OFFLINE rows carry footfall 0. Students divide bills by footfall, hit the divide-by-zero, wrap it in NULLIF and move on. Hours with bills but a dead counter return infinity and top the ranking, others return 0 and sink, so your three worst hours are three broken sensors. A zero from a dead sensor is not a measurement.

THE THREE QUESTIONS YOUR GOLD LAYER MUST ANSWER

1. Conversion rate - bills per visitor - per store per hour
2. The three worst hours of the day across the whole chain
3. Does conversion drop on the days with the highest footfall?

IN SNOWFLAKE – THE HARD ONE, WORKED

Tables this query needs in Snowflake: **GOLD_STORE_HOUR**

```
-- Q3: NTILE buckets each store's days by footfall; conversion is summed then
-- divided, never an average of averages.
WITH d AS (SELECT store_id, trade_date, SUM(footfall) f, SUM(bills) b
            FROM GOLD_STORE_HOUR WHERE sensor_ok GROUP BY 1, 2),
r AS (SELECT d.*, NTILE(10) OVER (PARTITION BY store_id ORDER BY f) AS decile
      FROM d)
SELECT decile, COUNT(*) AS store_days, SUM(f) AS visitors, SUM(b) AS bills,
       ROUND(SUM(b) / NULLIF(SUM(f), 0), 4) AS conversion_rate
FROM r
GROUP BY decile ORDER BY decile;
```

YOU ARE DONE WHEN

- ☐ Bronze footfall = 12,960 exactly = 12 stores x 90 dates x 12 hours. Compute that product in code and assert it.
- ☐ Silver bills = Bronze bills - 130 duplicates - 75 unknown-store - 60 out-of-hours. Print all four numbers and confirm the subtraction closes.
- ☐ Gold has 12,960 rows with exactly 540 at sensor_ok=false: 240 OFFLINE + 300 ERR = 540. Assert it.
- ☐ SUM(bills) in Gold equals the Silver bill count and SUM(revenue) equals SUM(bill_amount) within a rupee. Hours 14, 15 and 16 must come out the three worst by conversion.
- ☐ The top footfall decile converts about 15% below the bottom decile - that gap is the answer to Q3, and it survives only if the deciles are cut per store, never chain-wide.

IF YOU FINISH EARLY

Add a same-hour-last-week baseline with LAG partitioned by store_id and hour, ordered by trade_date, offset 7, and flag any store-hour 30% below its own baseline.

THE SITUATION

Fraud is rare, so every average hides it. Five thousand rupees is a routine transfer on one account and a red flag on another, so no fixed threshold works across accounts; the only useful comparison is an account against its own past. The table also holds account numbers analysts do not need to see, so make them unreadable.

WHERE THE DATA COMES FROM

There is no reliable public UPI dataset with account, merchant, device and timestamp columns, and distrust one claiming to be real - that data could not be published. Kaggle's Credit Card Fraud Detection is real but useless here: its features are PCA components V1 to V28, with no account, merchant or device, so it answers none of the three questions.

GENERATE IT – THE EXACT SPECIFICATION

Seed 99; every count below is exact. accounts: 5,000, ACC00000000-ACC00004999, bank = id mod 8, home_city = id mod 10, primary_device 'DEV' + the 7-digit id, secondary_device 'DEV9' + 6 digits. txns: 400,000 rows numbered 0-399,999. merchant_id = 'MER' + (row mod 1200), so 1,200 merchants; ix = row div 1200 gives hour = ix mod 24 and date = 2025-05-01 + (ix mod 45), 45 dates to 2025-06-14, filling all 1,200 x 24 merchant-hours. account = hash of the row over 5,000; device = that account's primary 92% of the time, its secondary 8%; amount = round(exp(6.2 + randn*0.5), 2). status FAILED for 4% of rows, but for MER0000-MER0039 6% before 23:00 and 34% in hours 23, 0 and 1. Plant: rows 0-599 as 200 groups of three, each group one account on one date with three new DEVX devices; rows 600-1199 one per account with an amount 8x to 20x normal, the first 40 on 2025-05-01, with no prior history. With j = row - 1200 inject amount 'NA' where j mod 398 = 0 and j < 398000 (1,000), account 'ACC99999999' where j mod 700 = 1 and j < 350000 (500), txn_ts 1970-01-01 where j mod 1200 = 5 and j < 360000 (300), and write a second copy of every row where j mod 100 = 9 and j < 160000 (1,600).

GENERATOR CODE – RUN THIS FIRST, EXACTLY AS WRITTEN

This assumes MY_ID and VOL are already set, as on the second page of this document. Do not change the seed or any of the counts — your checks later depend on them.

```
import pyspark.sql.functions as F # generator seed 99; every count below is exact
def W(d, n): d.write.mode('overwrite').option('header', True).csv(f'{VOL}/raw/{n}')
W(spark.range(5000).selectExpr("format_string('ACC%08d', id) as account_id",
    "format_string('BANK%d', id % 8) as bank", "format_string('CITY%d', id % 10) as home_city",
    "format_string('DEV%07d', id) as primary_device",
    "format_string('DEV9%06d', id) as secondary_device"), 'accounts')
t = spark.range(400000).selectExpr("id", "int(id % 1200) as mer", "int(id div 1200) as ix",
    "id - 1200 as j")
t = t.selectExpr("...", "pmod(ix, 24) as hh",
    "case when id < 600 then pmod(int(id div 3), 45) when id < 640 then 0"
    "    when id < 1200 then 10 + pmod(int(id), 35) else pmod(ix, 45) end as dd",
    "case when id < 600 then int(id div 3) when id < 1200 then int(id) - 400"
    "    else pmod(hash(id, 99), 5000) end as a")
t = t.selectExpr("...", "pmod(hash(id, 104), 1000) as fr",
    "case when id between 600 and 1199 then round(560 * (8 + pmod(int(id), 13)), 2)"
    "    else round(exp(6.2 + randn(100) * 0.5), 2) end as amt",
    "case when id < 600 then format_string('DEVX%06d', id)"
    "    when pmod(hash(id, 5), 100) < 8 then format_string('DEV9%06d', a)"
    "    else format_string('DEV%07d', a) end as device_id",
    "format_string('%s %02d:%02d:00', string(date_add(date'2025-05-01', dd)), "
    "    hh, int(id % 60)) as ts")
NA = "id >= 1200 and pmod(j, 398) = 0 and j < 398000" # 1,000 amounts read 'NA'
UNK = "id >= 1200 and pmod(j, 700) = 1 and j < 350000" # 500 unknown accounts
OLD = "id >= 1200 and pmod(j, 1200) = 5 and j < 360000" # 300 stamped 1970-01-01
DUP = "id >= 1200 and pmod(j, 100) = 9 and j < 160000" # 1,600 rows land twice
ST = "case when mer < 40 and hh in (23,0,1) then fr<340 when mer < 40 then fr<60 else fr<40 end"
x = t.selectExpr("format_string('T%08d', id) as txn_id",
    f"if({UNK}, 'ACC99999999', format_string('ACC%08d', a)) as account_id",
    "format_string('MER%04d', mer) as merchant_id",
    "format_string('MC%02d', pmod(mer, 12)) as merchant_category",
    f"if({OLD}, '1970-01-01 00:00:00', ts) as txn_ts", "device_id",
    f"if({ST}, 'FAILED', 'SUCCESS') as status",
    f"if({NA}, 'NA', string(amt)) as amount", f"({DUP}) as dp")
W(x.drop('dp').unionByName(x.filter('dp').drop('dp')), 'txns')
```

BRONZE – LAND IT, CHANGE NOTHING

Land both files with every column a string. Add `_source_file`, `_ingested_at` and `_row_hash`. Mask nothing here and keep the 300 1970 timestamps - a failed parse upstream is evidence for the source system. Bronze txns = exactly 401,600 rows, and it is the only place the raw account number sits in the clear.

SILVER – THE DECISIONS YOU MUST MAKE, AND RECORD

Each decision below has a number attached to it. If your number does not match, stop and find out why before you go on — that gap is the finding, not a nuisance.

1. CAST amount to DECIMAL(18,2) and `txn_ts` to TIMESTAMP with `try_cast`, never cast: exactly 1,000 amounts become NULL and go to `silver_rejects` as 'unparseable_amount' - unscorable, not zero. DEDUPE on `txn_id`: 401,600 -> 400,000, and record that count.
2. REJECT: exactly 500 rows carry `account_id` 'ACC999999999' with no master row, `reason='unknown_account'`. Exactly 300 sit at 1970-01-01, outside 2025-05-01 to 2025-06-14, `reason='timestamp_out_of_range'`. Range-check the window, do not look for the literal 1970.
3. THE ACCOUNT'S OWN AVERAGE, the definition the project rests on: the trailing 30-day average EXCLUDES the current transaction and day - partition by `account_id`, order by day, ROWS BETWEEN 30 PRECEDING AND 1 PRECEDING - and needs 5 prior transactions. Store it and `prior_txn_count` in Silver.
4. DEVICE COUNT: `distinct_devices` is per account per calendar day (timestamps are already IST), not per rolling 24 hours. Two definitions give two answers and only one can be the rule. Write down which you chose.

GOLD – THE SHAPE THAT TRAVELS

Two tables. `account_day`: `account_id`, `account_day`, `txns`, `max_amount`, `avg_amount_30d_trailing`, `prior_txn_count`, `distinct_devices`, `is_amount_spike`, `is_device_burst` - near 187,000 of the 5,000 x 45 cells, print the exact figure. `merchant_hour`: `merchant_id`, `hour_of_day`, `txns`, `failed_txns`, `failure_rate` - 1,200 x 24 = 28,800 rows with no hour missing, and Q3 needs it.

THE TRAP MOST PEOPLE FALL INTO

Almost everyone writes ROWS BETWEEN 30 PRECEDING AND CURRENT ROW, the usual moving-average frame. Fatal here: the huge transaction sits inside its own baseline and lifts it, so a 15x spike reads as 2x - the bigger the fraud, the better it hides. End the frame at 1 PRECEDING. An all-account average flags every business account.

THE THREE QUESTIONS YOUR GOLD LAYER MUST ANSWER

1. Accounts whose amount is 5x their own trailing 30-day average
2. Accounts using 3 or more distinct devices in a single day
3. Merchants whose failure rate spikes after 11 PM

IN SNOWFLAKE – THE HARD ONE, WORKED

Tables this query needs in Snowflake: **GOLD_ACCOUNT_DAY**

```
-- Masking policies need Snowflake Enterprise edition or above; on a Standard
-- trial skip the two DDL statements and run the SELECT unmasked.
CREATE MASKING POLICY mask_acct AS (v STRING) RETURNS STRING -> CASE WHEN
  CURRENT_ROLE()='FRAUD_OPS' THEN v ELSE 'XXXXXXX' || RIGHT(v, 4) END;
ALTER TABLE GOLD_ACCOUNT_DAY MODIFY COLUMN account_id SET MASKING POLICY mask_acct;
SELECT account_id, account_day, max_amount, avg_amount_30d_trailing AS base,
  COUNT(*) OVER (PARTITION BY account_id ORDER BY account_day
    ROWS BETWEEN 30 PRECEDING AND 1 PRECEDING) AS days_prior
FROM GOLD_ACCOUNT_DAY
QUALIFY prior_txn_count >= 5 AND max_amount >= 5 * avg_amount_30d_trailing;
```

YOU ARE DONE WHEN

- ☐ Bronze txns 401,600 - 1,600 duplicates - 1,000 unparseable amounts - 500 unknown accounts - 300 out-of-range timestamps = Silver 398,200. Print all five.
- ☐ 560 of the 600 planted spikes clear the rule. The other 40 sit on the opening date with no prior rows, so their baseline is NULL and they drop out - name that number. Roughly 140 more come from the lognormal tail; those are not bugs.
- ☐ is_device_burst is true for exactly 200 account-days. More means you counted per rolling 24 hours; fewer means dedupe removed real transactions.
- ☐ Pooled over MER0000-MER0039, hours 23, 0 and 1 fail near 34% against near 6% before 23:00, over 5x; every other merchant sits near 4% in both windows. Print both. A non-FRAUD_OPS role sees XXXXXXXX plus four digits.

IF YOU FINISH EARLY

Use gaps-and-islands on the flagged account-days - row number minus a date-based row number gives the group key - to find accounts flagged 3 days running.

THE SITUATION

On the day a loan defaults everyone sees it; the money was lost three months earlier, when the borrower missed two EMIs in a row and nobody called. Collections wants the accounts heading that way; risk wants to know if one month's disbursements behave worse than another's. Both need a loan measured by age, not by today's date.

WHERE THE DATA COMES FROM

Lending Club is the usual real credit-risk dataset, with two problems. The original Kaggle mirror was withdrawn and circulating copies differ in column sets and in how loan_status is spelled, so confirm the header. It also has one row per loan with a final status and no monthly ledger, so it cannot say if an EMI was missed in month four.

GENERATE IT – THE EXACT SPECIFICATION

Seed 77; every count below is exact. loan_master: 8,204 rows L000000-L008203 of loan_id, customer_id, grade, branch, city, tenure_months, principal, emi_amount, disbursement_date. The first 8,004 are real: cohort = id div 667 gives twelve cohorts of 667 disbursed on the 10th, 2024-01-10 to 2024-12-10; the last 200 are dated 2027, pure poison. grade from id mod 20 as 4 A, 5 B, 5 C, 4 D, 2 E, tenure 12, 24 or 36 by id mod 3; principal = round(exp(10.8 + randn*0.5)) clamped to 20,000-150,000; emi_amount = round(principal * 1.18 / tenure, 2). A loan sours when a hashed draw falls under its grade risk - A .015, B .035, C .06, D .10, E .16 - then misses months s to s+run-1, run 3 to 6, s = 1 + hash mod (tenure-2) pulled forward by the cohort number so later vintages sour sooner. Other months are LATE 12% of the time with days_past_due 1-45, else PAID at 0; MISSED carries no paid_amount and 90; after 2025-12-31 a due_date is FUTURE. ledger: one row per loan per due month = 2,668 x (12+24+36) real + 2,400 poison = 194,496. Inject: 900 exact duplicates, 350 orphans keyed L900000-L900349, days_past_due written 'NULL' on 1,200 rows, paid_amount of 3.4x EMI on 500. Ledger on disk = 195,746.

GENERATOR CODE – RUN THIS FIRST, EXACTLY AS WRITTEN

This assumes MY_ID and VOL are already set, as on the second page of this document. Do not change the seed or any of the counts — your checks later depend on them.

```
import pyspark.sql.functions as F # generator seed 77; every count below is exact
def W(d, n): d.write.mode('overwrite').option('header', True).csv(f'{VOL}/raw/{n}')
G = "element_at(split('A,A,A,A,B,B,B,B,C,C,C,C,D,D,D,E,E', ','), int(id % 20) + 1)"
Q = "element_at(array(0.015, 0.035, 0.06, 0.10, 0.16), instr('ABCDE', grade))"
lo = spark.range(8204).selectExpr("id", "format_string('L%06d', id) as loan_id",
    "format_string('C%06d', id % 6000) as customer_id", f"{G} as grade",
    "format_string('BR%02d', id % 30) as branch", "format_string('CITY%d', id % 6) as city",
    "int(id div 667) as coh", "if(id < 8004, 12 * int(id % 3) + 12, 12) as tenure_months",
    "int(least(greatest(round(exp(10.8 + randn(77) * 0.5)), 20000), 150000)) as principal")
lo = lo.selectExpr("round(principal * 1.18 / tenure_months, 2) as emi_amount",
    "add_months(date'2024-01-10', if(id < 8004, coh, 36 + int(id % 12))) as disbursal_date",
    f"pmod(hash(id, 71), 1000) / 1000.0 < {Q} as bad", "3 + pmod(hash(id, 73), 4) as run",
    "greatest(1, 1 + pmod(hash(id, 72), tenure_months - 2) - coh) as s0") # sours sooner
W(lo.drop('id', 'coh', 'bad', 's0', 'run'), 'loan_master')
le = lo.withColumn('m', F.explode(F.sequence(F.lit(1), F.col('tenure_months'))))
le = le.selectExpr("add_months(disbursal_date, m) as due_date",
    "pmod(hash(id, 74, m), 100) < 12 as lt", "bad and m between s0 and s0+run-1 as miss")
le = le.selectExpr("case when due_date > date'2025-12-31' then 'FUTURE'"
    " when miss then 'MISSED' when lt then 'LATE' else 'PAID' end as st0")
PRE = "(pmod(id, 16) = 3 and id < 8000 and m = 6)" # 500 prepayments at 3.4 x EMI
NUL = "(pmod(id, 20) = 11 and id < 8004 and m in (2, 4, 7))" # 1,200 dpd read 'NULL'
le = le.selectExpr("id", "m", "loan_id", "due_date", "emi_amount",
    f"if({PRE}, 'PAID', st0) as status",
    f"if({PRE}, round(emi_amount * 3.4, 2), if(st0 in ('PAID', 'LATE'), emi_amount, null))"
    " as paid_amount",
    f"if({PRE} or st0 = 'PAID', due_date, if(st0 = 'LATE',"
    " date_add(due_date, 1 + pmod(hash(id, 75, m), 45)), null)) as paid_date")
le = le.selectExpr("case when {NUL} then 'NULL' when status = 'MISSED' then '90'"
    " else string(datediff(paid_date, due_date)) end as days_past_due")
L = ['loan_id', 'due_date', 'emi_amount', 'status', 'paid_amount', 'paid_date', 'days_past_due']
orp = le.filter("pmod(id, 22) = 7 and m = 4 and id < 7700").selectExpr(" # 350 orphans
    "format_string('L9%05d', int(id div 22)) as loan_id", *L[1:])
dup = le.filter("pmod(id, 8) = 5 and m = 3 and id < 7200") # 900 rows land twice
W(le.select(*L).unionByName(dup.select(*L)).unionByName(orp), 'ledger')
```

BRONZE – LAND IT, CHANGE NOTHING

Land both files with every column a string, days_past_due included: 1,200 of its values are the word 'NULL' spelled out. Add _source_file, _ingested_at and _row_hash. Do not create disbursal_month here - Bronze holds disbursal_date as core banking sent it. Bronze ledger = 195,746 rows, loan_master = 8,204.

SILVER – THE DECISIONS YOU MUST MAKE, AND RECORD

Each decision below has a number attached to it. If your number does not match, stop and find out why before you go on — that gap is the finding, not a nuisance.

1. CAST emi_amount and paid_amount to DECIMAL(12,2), days_past_due to INT, due_date and paid_date to DATE, all try_cast never cast: exactly 1,200 days_past_due become NULL. DEDUPE on (loan_id, due_date): 195,746 -> 194,846. Keying on loan_id alone destroys the schedule.
2. REJECT: the 350 ledger rows whose loan_id has no master row as 'orphan_ledger_row'; the 200 loans dated 2027 as 'future_disbursal', and their ledger rows with them. That cascade is 2,400 rows, not 200, and it is what students forget. Silver ledger lands on 192,096.
3. THE COHORT COLUMN, made here and nowhere else: disbursal_month = date_trunc('month', disbursal_date) on the loan master; months_on_book on the ledger = whole months between disbursal_month and due_date. Bronze must not hold them, Gold must not recompute them.
4. MISSED, defined once: paid_amount IS NULL or paid_amount < 0.5*emi_amount. Paid in full with days_past_due above 0 is LATE. DEFAULT is 3 MISSED in a row and stays true after. A prepayment settles only its own month: the 500 must not cut a missed count. FUTURE rows are out of every rate.

GOLD – THE SHAPE THAT TRAVELS

One row per cohort per grade per months_on_book: disbursal_month, grade, months_on_book, loans_in_cohort, loans_active, loans_2_consecutive_missed, loans_defaulted, cumulative_default_rate, avg_days_past_due. Each cohort has lived a different number of months - 23 for January down to 12 for December - so 210 cohort-months x 5 grades = exactly 1,050 rows.

THE TRAP MOST PEOPLE FALL INTO

Students compare cohorts by calendar date: defaults booked so far, cohort against cohort. That mixes risk with exposure - the January 2024 cohort has had 23 months to go bad and December only 12 - and the two effects cancel, so all twelve look identical and the report says nothing. Read every cohort at the same months_on_book instead. Second half: the 500 prepayments make a customer look paid for three months if you compare paid_amount to emi_amount.

THE THREE QUESTIONS YOUR GOLD LAYER MUST ANSWER

1. Share of accounts that missed 2 consecutive EMIs, by loan grade
2. Average days-past-due trend by disbursal cohort
3. Which cohort's 6-month default rate is worst?

IN SNOWFLAKE – THE HARD ONE, WORKED

Tables this query needs in Snowflake: **SILVER_LEDGER, SILVER_LOAN**

```
-- Q1/Q3: LAG finds two misses in a row; months_on_book <= 6 ages cohorts alike.
WITH f AS (SELECT l.loan_id, m.grade, m.disbursal_month, l.status, LAG(l.status)
  OVER (PARTITION BY l.loan_id ORDER BY l.due_date) AS prev
  FROM SILVER_LEDGER l JOIN SILVER_LOAN m ON m.loan_id = l.loan_id
  WHERE l.status <> 'FUTURE' AND l.months_on_book <= 6)
SELECT disbursal_month, grade, COUNT(DISTINCT loan_id) AS loans,
  COUNT(DISTINCT CASE WHEN status='MISSED' AND prev='MISSED'
    THEN loan_id END) AS two_missed
FROM f GROUP BY 1, 2
ORDER BY two_missed / NULLIF(loans, 0) DESC;
```

YOU ARE DONE WHEN

- ☐ loan_master 8,204 on disk; the 8,004 real ones print as twelve disbursal months of exactly 667. Grades across those 8,004: 1,604 A, 2,000 B, 2,000 C, 1,600 D, 800 E.
- ☐ Bronze ledger 195,746 - 900 duplicates - 350 orphans - 2,400 rows belonging to the 200 loans dated 2027 = Silver 192,096. State all four numbers.
- ☐ The six-month two-consecutive-missed rate rises at every grade step A to E, near 1% at A and near 9% at E. If not, MISSED is catching LATE rows or dedupe ate rows.
- ☐ loans_in_cohort counts the loans in that cohort and grade, not 667; SUM it at months_on_book = 1 to get the Silver loan count 8,004.
- ☐ At six months on book the last three cohorts run about double the first three. Compared instead by calendar date all twelve look alike - that flatness is the artefact, not a finding.

IF YOU FINISH EARLY

Use gaps-and-islands on the ledger - row_number minus a month index gives a run key - for the longest run of MISSED months per loan.

THE SITUATION

A price is not a result. No two shares compare on closing price; meaning lives in the change from yesterday, and the risk desk wants how wide those changes have been over the last month. Both are derived columns, computed once in Silver, not three times in three queries with three answers.

WHERE THE DATA COMES FROM

NIFTY 50 daily OHLC sits in several Kaggle uploads and NSE reports; none is safe as-is. Names vary: Close, Close Price, one with a leading space; some carry Series (keep only 'EQ'); some are one file per symbol, some one long file with a Symbol column. None adjust for splits or bonuses, so a 1:5 split reads as an 80 percent crash. Print the header and ten rows before coding; flag any one-day move beyond 25 percent.

GENERATE IT – THE EXACT SPECIFICATION

Seed 8. symbols 10, three columns in the file plus two hidden: symbol, company_name (symbol + ' Ltd'), sector, and hidden sigma and beta. RELI TCSX HDFC INFY ICIC ITCX LTIM SBIN BHTI MARU, n=0-9; sector = n%5 over Energy/IT/Banking/Telecom/Auto, 2 each; sigma $0.010 + (n\%5) * 0.005 = 0.010 - 0.030$; beta 1.5 for n<5 else 0.4. prices 10 x 750 weekdays from 2023-01-02 = 7,500 rows (no holidays; the 750th is 2025-11-14): symbol, trade_date, open, high, low, close, volume. close starts at 500.00 and compounds logret = $\text{randn} * \text{sigma} + \text{beta} * (-0.045)$ on the ten weekdays 2025-03-03 to 2025-03-14; open = prev close * (1+randn*0.003), high = greatest(open,close)*1.004, low = least*0.996, volume 100,000-1,000,000. Inject on trading days 101-690 only, so every symbol keeps its first day: 180 close 'NA', 120 volume '-', 40 close 0, all disjoint; then 250 duplicate rows to 7,750 and 90 'ZZZTEST' rows on 90 distinct dates to 7,840.

GENERATOR CODE – RUN THIS FIRST, EXACTLY AS WRITTEN

This assumes MY_ID and VOL are already set, as on the second page of this document. Do not change the seed or any of the counts — your checks later depend on them.

```
import pyspark.sql.functions as F; from pyspark.sql import Window; SEED=8
sym=(spark.range(10).selectExpr(
    "element_at(array('RELI','TCSX','HDFC','INFY','ICIC','ITCX','LTIM','SBIN','
    'BHTI','MARU'),cast(id+1 as int)) symbol",
    "element_at(array('Energy','IT','Banking','Telecom','Auto'),cast(id%5+1 as int)) sector",
    "round(0.010+(id%5)*0.005,3) sigma","if(id<5,1.5,0.4) beta")
    .withColumn('company_name',F.concat('symbol',F.lit(' Ltd'))))
sym.select('symbol','company_name','sector').write.mode('overwrite') \
    .option('header',True).csv(f'{VOL}/raw/symbols')
cal=(spark.sql("SELECT date_add(date'2023-01-02',cast(id as int)) trade_date "
    "FROM range(1100)").filter('dayofweek(trade_date) between 2 and 6')
    .withColumn('d',F.row_number().over(Window.orderBy('trade_date'))).filter('d<=750'))
W=Window.partitionBy('symbol').orderBy('d')
SH="if(trade_date between date'2025-03-03' and date'2025-03-14',-0.045,0)"
px=(sym.crossJoin(cal)
    .withColumn('lr',F.randn(SEED)*F.col('sigma')+F.col('beta')*F.expr(SH))
    .withColumn('close',F.round(500*F.exp(F.sum('lr').over(W)),2))
    .withColumn('open',F.round(F.coalesce(F.lag('close').over(W),F.col('close'))
        *(1+F.randn(SEED+1)*0.003),2))
    .withColumn('vol',F.expr("cast(cast(rand(10)*900000+100000 as bigint) as string)"))
raw=px.selectExpr('d','symbol','trade_date','open',
    'round(greatest(open,close)*1.004,2) high','round(least(open,close)*0.996,2) low',
    "case when d between 101 and 118 then 'NA' when d between 301 and 304 then '0'"
    " else cast(close as string) end close",
    "if(d between 201 and 212,'-',vol) volume")
out=(raw.unionByName(raw.filter('d between 501 and 525'))
    .unionByName(raw.filter("symbol='RELI' and d between 601 and 690")
        .withColumn('symbol',F.lit('ZZZTEST'))))
out.drop('d').write.mode('overwrite').option('header',True).csv(f'{VOL}/raw/prices')
```

BRONZE – LAND IT, CHANGE NOTHING

Land both files all-string - 180 closes are 'NA', 120 volumes are '-', and a typed read either fails under ANSI or drops the rows you must count. Add `_source_file` (`input_file_name()`), `_ingested_at` (`current_timestamp()`) and `_row_hash`. No returns, no sorting, no repair here. Bronze prices = exactly 7,840 rows.

SILVER – THE DECISIONS YOU MUST MAKE, AND RECORD

Each decision below has a number attached to it. If your number does not match, stop and find out why before you go on — that gap is the finding, not a nuisance.

1. CAST open, high, low, close to DOUBLE and volume to BIGINT with `try_cast`, never cast - ANSI mode kills the job on a plain cast. Expect exactly 180 NULL closes and 120 NULL volumes; write both counts to a data quality table.
2. DEDUPE on (symbol, trade_date): 7,840 to 7,590, difference exactly 250. ORPHANS: exactly 90 rows carry symbol 'ZZZTEST', not in the symbols file, so no sector and no sigma - `silver_rejects`, reason='unknown_symbol'; 7,590 to 7,500. Print every count.
3. ZERO CLOSE: exactly 40 rows have close 0. Set NULL, copy to `silver_rejects`, reason='zero_close'. A zero makes today's return -100 percent and tomorrow's a division by zero, and both land in every 30-day vol window they touch, poisoning 60 more rows per symbol.
4. DAILY RETURN, only here: `prev_close` = the last available close before today, skipping the NULLs - in Databricks SQL that is `lag(close)` ignore nulls over (partition by symbol order by trade_date). `daily_return` = `close/prev_close - 1`. Exactly 10 rows, each symbol's first day, are NULL; that is not a zero.

GOLD – THE SHAPE THAT TRAVELS

One row per symbol per usable-close day: symbol, company_name, sector, trade_date, close, prev_close, daily_return, rolling_vol_30d (STDDEV of daily_return over ROWS BETWEEN 29 PRECEDING AND CURRENT ROW), rolling_mean_return_30d, obs_in_window (COUNT(*) over that same frame), market_return (the ten-symbol average daily_return for that date), is_worst10_day. 7,500 - 180 - 40 = 7,280 rows.

THE TRAP MOST PEOPLE FALL INTO

LAG written without PARTITION BY symbol. On a file sorted by symbol then date, each symbol's first row takes prev_close from the previous symbol's last row: nine huge fake returns that dominate the first month of nine symbols' 30-day vol. Second half: comparing symbols by summing daily returns. Returns compound, they do not add.

THE THREE QUESTIONS YOUR GOLD LAYER MUST ANSWER

1. Daily return per symbol, using LAG on the close price
2. 30-day rolling volatility per symbol
3. Which symbols moved together in the worst 10 days?

IN SNOWFLAKE – THE HARD ONE, WORKED

Tables this query needs in Snowflake: **GOLD_SYMBOL_DAY**

```
-- Q3: LAG builds the return, a rolling frame the vol, QUALIFY the 10 worst days.
WITH r AS (SELECT symbol, trade_date,
  close/NULLIF(LAG(close) OVER (PARTITION BY symbol ORDER BY trade_date),0)-1 AS ret
  FROM GOLD_SYMBOL_DAY),
v AS (SELECT symbol, trade_date, ret, STDDEV(ret) OVER (PARTITION BY symbol
  ORDER BY trade_date ROWS BETWEEN 29 PRECEDING AND CURRENT ROW) AS vol_30d FROM r),
w AS (SELECT trade_date, AVG(ret) AS mkt FROM r WHERE ret IS NOT NULL
  GROUP BY trade_date QUALIFY ROW_NUMBER() OVER (ORDER BY AVG(ret)) <= 10)
SELECT v.symbol, COUNT(*) AS worst_days, ROUND(AVG(v.ret)*100,3) AS avg_ret_pct,
  ROUND(AVG(v.vol_30d),4) AS avg_vol,
  ROUND(AVG(v.ret)/NULLIF(AVG(w.mkt),0),2) AS vs_market
FROM v JOIN w USING (trade_date) GROUP BY v.symbol ORDER BY vs_market DESC;
```

YOU ARE DONE WHEN

- ☐ Bronze prices = 7,840 = 7,500 + 250 duplicates + 90 orphans. Compute the sum in code and assert it against the count.
- ☐ Silver = 7,840 - 250 - 90 = 7,500, with 40 zero_close rejects, 180 NULL closes, 120 NULL volumes logged; Gold = 7,500 - 180 - 40 = 7,280. Print all seven.
- ☐ Exactly 10 Gold rows have NULL daily_return, one per symbol at its earliest trade_date, and exactly 290 rows (10 x 29) have obs_in_window below 30. Count both.
- ☐ All ten shock days, 2025-03-03 to 2025-03-14, appear in the worst ten; RELI, TCSX, HDFC, INFY and ICIC average three to four times more negative than the other five.

IF YOU FINISH EARLY

Annualise vol by sqrt(252), add a rolling 30-day beta against the equal-weighted index of all ten on the same frame, and flag the symbol whose beta moves most first window to last.

THE SITUATION

The bank's CRM says 'active' for 1.9 million people. Marketing sends the fuel offer and the travel offer to everyone, measuring nothing. Build the segment from what people spend. The catch: a segment is a decision - dominant category by visits and by spend differ for a quarter of the book.

WHERE THE DATA COMES FROM

No public dataset carries a customer id, a merchant category and a date together. Kaggle's 'Credit Card Customers' churn file is aggregate annual spend, no transactions and no categories, so it answers none of the three questions. The IBM synthetic card file has categories but its column names and category coding differ across mirrors and it runs to several GB. Generate.

GENERATE IT – THE EXACT SPECIFICATION

Seed 9. customers 2,000, CUST0000-CUST1999: city n%10 (200 each), card_type from n%4 (0 and 1 Silver, 2 Gold, 3 Platinum = 1,000/500/500), age_band n%5 (400 each), and a hidden home_cat = n%8 that never reaches the file. card_txns 2,000 x 6 months (Jan-Jun 2025) x 10 slots = 120,000 rows: txn_id, customer_id, txn_date, merchant_id, merchant_category, amount. Categories 0-7 Groceries/Fuel/Dining/Travel/Electronics/Apparel/Utilities/Health carry price multipliers 1.0/0.8/1.1/2.0/1.8/1.4/0.9/1.6, and amount = round(uniform(500,1000)*multiplier,2). Slots 1-7 take the home category h; slots 8, 9 and 10 take (h+1), (h+3) and (h+5) mod 8. Four categories per customer-month, and seven transactions make home the largest spend for everyone except customers 1500-1999, whose slot-10 purchase is 50x - big-ticket, so their top category by spend is not their top by visits. Customers 0-299 move to (h+3) from month 4. Inject: 800 amounts 'NA' (customers 1000-1199, months 1-4, slot 5, so no category empties); 600 slot-9 amounts turned negative as refunds; 250 spellings (100 'groceries ', 100 'DINING', 50 ' Fuel'); then 1,500 duplicate rows to 121,500 and 400 'CUST9999' rows to 121,900.

GENERATOR CODE – RUN THIS FIRST, EXACTLY AS WRITTEN

This assumes MY_ID and VOL are already set, as on the second page of this document. Do not change the seed or any of the counts — your checks later depend on them.

```
import pyspark.sql.functions as F; from pyspark.sql import Window; SEED=9
CAT="array('Groceries','Fuel','Dining','Travel','Electronics','Apparel','Utilities','Health')"
MUL="array(1.0,0.8,1.1,2.0,1.8,1.4,0.9,1.6)"
cust=spark.range(2000).selectExpr("format_string('CUST%04d',id) customer_id",
    "element_at(array('Delhi','Mumbai','Chennai','Kolkata','Pune','Jaipur','Kochi',"
    "'Surat','Indore','Patna'),cast(id%10+1 as int)) city",
    "case when id%4<2 then 'Silver' when id%4=2 then 'Gold' else 'Platinum' end card_type",
    "element_at(array('18-25','26-35','36-45','46-55','56plus'),cast(id%5+1 as int)) age_band")
cust.write.mode('overwrite').option('header',True).csv(f'{VOL}/raw/customers')
t=(spark.range(2000).withColumnRenamed('id','c')
    .withColumn('mth',F.explode(F.sequence(F.lit(1),F.lit(6))))
    .withColumn('slot',F.explode(F.sequence(F.lit(1),F.lit(10))))
    .withColumn('ci',F.expr('cast((if(c<300 and mth>=4,(c%8+3)%8,c%8)+case slot when 8 then 1'
    ' when 9 then 3 when 10 then 5 else 0 end)%8+1 as int)'))
    .selectExpr('c','mth','slot',f'element_at({CAT},ci) merchant_category',
    'if(c>=1500 and slot=10,50,1)*if(c between 400 and 999 and mth=2 and slot=9,-1,1)'
    f'*round((500+rand(9)*500)*element_at({MUL},ci),2) amt'))
w=Window.partitionBy('merchant_category').orderBy('c','mth','slot')
base=t.withColumn('rk',F.row_number().over(w)).selectExpr('c','mth','slot',
    "format_string('T%04d%d%02d',c,mth,slot) txn_id","format_string('CUST%04d',c) customer_id",
    "date_add(add_months(date'2025-01-01',mth-1),slot-1) txn_date",
    "format_string('M%03d',(c*7+slot)%200) merchant_id",
    "case when merchant_category='Groceries' and rk<=100 then 'groceries ' when"
    " merchant_category='Dining' and rk<=100 then 'DINING' when merchant_category='Fuel'"
    " and rk<=50 then ' Fuel' else merchant_category end merchant_category",
    "if(c between 1000 and 1199 and mth<=4 and slot=5,'NA',cast(amt as string)) amount")
orp=(base.filter('c between 1450 and 1499 and mth<=4 and slot in (2,3)')
    .withColumn('customer_id',F.lit('CUST9999'))
    .withColumn('txn_id',F.concat(F.lit('X'),F.col('txn_id'))))
out=base.unionByName(base.filter('c between 1200 and 1449 and slot=1')).unionByName(orp)
out.drop('c','mth','slot').write.mode('overwrite').option('header',True) \
    .option('ignoreLeadingWhiteSpace',False).option('ignoreTrailingWhiteSpace',False) \
    .csv(f'{VOL}/raw/card_txns')
```

BRONZE – LAND IT, CHANGE NOTHING

Land both files all-string - 800 amounts are 'NA'. Add `_source_file`, `_ingested_at`, `_row_hash`. Keep `merchant_category` byte for byte, including 'groceries ' and ' Fuel'; no trim, no upper-casing here. Those 250 rows evidence three spellings from the source - a finding for the card switch. Bronze `card_txns` = exactly 121,900 rows.

SILVER – THE DECISIONS YOU MUST MAKE, AND RECORD

Each decision below has a number attached to it. If your number does not match, stop and find out why before you go on — that gap is the finding, not a nuisance.

1. CAST amount to DECIMAL(12,2) with `try_cast`, never cast: exactly 800 go NULL. Move them to `silver_rejects` with `reason='unparseable_amount'`. Unknown spend is not zero spend, and only one of the two belongs in a denominator.
2. DEDUPE on `txn_id`: 121,900 to 120,400, difference exactly 1,500. ORPHANS: exactly 400 rows carry `customer_id` 'CUST9999' with no master row, so no city and no `card_type` - `silver_rejects`, `reason='unknown_customer'`; 120,400 to 120,000.
3. NORMALISE `merchant_category` here only: trim, then title-case to the eight canonical names. Exactly 250 rows change and SELECT DISTINCT then returns exactly 8 values, not the 11 you start with: 'groceries ', 'DINING' and ' Fuel' each earn a Gold row, a share and a rank.
4. DEFINE SPEND once: the net sum of amount including the 600 negative refunds - no `abs()`, no filter. Dominant category is by SPEND, not transaction count; put that line in a comment above the code, because both are defensible and only one can be the rule.

GOLD – THE SHAPE THAT TRAVELS

One row per customer per `merchant_category` per month: `customer_id`, city, `card_type`, `age_band`, month, `merchant_category`, `txns`, `spend`, `customer_month_spend`, `spend_share`, `is_dominant`. Exactly $2,000 \times 6 \times 4 = 48,000$ rows - the 800 rejected amounts thin one home category from seven transactions to six, they never empty a category, so no group disappears.

THE TRAP MOST PEOPLE FALL INTO

Dominant category as GROUP BY MAX(`spend`) joined back to detail. On a tie it returns two rows per customer-month: Gold grows past 48,000, shares stop summing to one, switchers exceed 300. Use QUALIFY ROW_NUMBER() with a tie-break sort key. Second half: averaging six monthly shares, not summing numerators and denominators.

THE THREE QUESTIONS YOUR GOLD LAYER MUST ANSWER

1. Spend share per merchant category per customer
2. Assign each customer to a dominant category using QUALIFY
3. How many customers changed dominant category over the six months?

IN SNOWFLAKE – THE HARD ONE, WORKED

Tables this query needs in Snowflake: **GOLD_CUSTOMER_CATEGORY_MONTH**

```
-- Q2/Q3: QUALIFY takes one dominant row per customer-month; the second sort key
-- breaks ties so one customer-month can never produce two dominant categories.
WITH dom AS (SELECT customer_id, month, merchant_category
  FROM GOLD_CUSTOMER_CATEGORY_MONTH QUALIFY ROW_NUMBER()
    OVER (PARTITION BY customer_id, month ORDER BY spend DESC, merchant_category)=1),
ends AS (SELECT customer_id,
  FIRST_VALUE(merchant_category) OVER (PARTITION BY customer_id ORDER BY month) f,
  LAST_VALUE(merchant_category) OVER (PARTITION BY customer_id ORDER BY month
    ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) l
  FROM dom QUALIFY ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY month)=1)
SELECT COUNT(*) AS switched FROM ends WHERE f <> l;
```

YOU ARE DONE WHEN

- ☐ Bronze card_txns = 121,900 = 120,000 + 1,500 dupes + 400 orphans; Silver = 121,900 - 1,500 - 400 - 800 = 119,200. Print all five and make it close.
- ☐ SELECT DISTINCT merchant_category on Silver returns exactly 8 rows. If it returns 11, normalisation did not run and every number below it is wrong.
- ☐ Per customer-month in Gold, COUNT(*) = 4 and SUM(spend_share) = 1.0000 to within 0.0001. A share that misses one means you divided by the six-month total.
- ☐ The SQL returns exactly 300, all customer numbers 0-299, all switching between month 3 and 4. A switch elsewhere means a tie or a count-based rule.
- ☐ Dominant by spend and dominant by transaction count disagree for exactly 500 customers, CUST1500-CUST1999, in every month. That is the segmentation argument, not a bug.

IF YOU FINISH EARLY

Add a stability score - how many of the six months the dominant category equals the customer's modal one - and flag anyone switching more than once; they have no dominant category at all.

THE SITUATION

The gateway reports 41,208 settled, the ledger 41,193, and everyone shrugs at 15. It is not 15: 208 the ledger never saw, 193 the gateway lacks, and 450 more that both hold and disagree on. A netted total hides its own size. Produce a break list a human can work through, with a reason on every line.

WHERE THE DATA COMES FROM

There is no public dataset and none should be trusted: a paired gateway extract and finance ledger for the same transactions is internal financial data, and anything claiming to be one is invented. That is the point - you own both sides, so you know every break count before you write the query.

GENERATE IT — THE EXACT SPECIFICATION

Seed 10. Base 41,000 transactions across 2025-06-01 to 2025-06-30, ids TXN0000000-TXN0040999: txn_id, txn_ts, merchant_id M000-M199, amount = $\text{round}(\exp(6.8 + \text{randn} * 0.7), 2)$, currency INR, payment_method UPI/CARD/NETBANKING/WALLET from $n\%4$ (10,250 each). Gateway = those 41,000 with status 'SUCCESS' plus 208 gateway-only rows TXNG000000-207 = 41,208. Ledger = the same 41,000 with 'POSTED' plus 193 ledger-only rows TXNL000000-192 = 41,193. Ledger amount edits by id: 0-399 x (1-0.0177), a fee; 400-589 +0.01; 590-639 x10, keying errors. Timestamps are forced, not sampled: ids 1000-1599 sit between 06:00 and 12:00 in the gateway and move +6h in the ledger, same day, not a break; ids 1600-1899 sit between 21:00 and 23:59 and move +6h into the next day - exactly 300 date breaks. Ids 3000-3209 get an amount above 1,500 so the ledger can print a thousands comma. Inject: 350 gateway amounts 'NA' (ids 2000-2349); 210 ledger amounts like '12,450.00'; 75 ledger txn_ids with two trailing spaces; then 260 gateway duplicates to 41,468 and 130 ledger duplicates to 41,323.

GENERATOR CODE – RUN THIS FIRST, EXACTLY AS WRITTEN

This assumes MY_ID and VOL are already set, as on the second page of this document. Do not change the seed or any of the counts — your checks later depend on them.

```
import pyspark.sql.functions as F; SEED=10
PM="array('UPI','CARD','NETBANKING','WALLET')"
b=spark.range(41000).selectExpr('id',"format_string('TXN%07d',id) txn_id",
    "format_string('M%03d',id%200) merchant_id","'INR' currency",
    f"element_at({PM},cast(id%4+1 as int)) payment_method",
    "if(id between 3000 and 3209,round(1500+rand(11)*8000,2),"
    "round(exp(6.8+randn(10)*0.7),2)) amount",
    "case when id between 1000 and 1599 then cast(rand(12)*30 as int)*1440+360"
    "+cast(rand(13)*360 as int) when id between 1600 and 1899 then"
    " cast(rand(14)*30 as int)*1440+1260+cast(rand(15)*179 as int)"
    " else cast(rand(16)*43200 as int) end mins")
b=b.selectExpr('id','txn_id','merchant_id','currency','payment_method','amount',
    "timestampadd(minute,mins,timestamp'2025-06-01 00:00:00') txn_ts")
g=b.selectExpr('id','txn_id','txn_ts','merchant_id',
    "if(id between 2000 and 2349,'NA',cast(amount as string)) amount",
    'currency','payment_method',"SUCCESS' status")
l=b.selectExpr('id',"if(id between 5000 and 5074,concat(txn_id,' '),txn_id) txn_id",
    "if(id between 1000 and 1899,timestampadd(hour,6,txn_ts),txn_ts) txn_ts",'merchant_id',
    "case when id<400 then cast(round(amount*(1-0.0177),2) as string)"
    " when id<590 then cast(round(amount+0.01,2) as string)"
    " when id<640 then cast(round(amount*10,2) as string)"
    " when id between 3000 and 3209 then format_number(amount,2)"
    " else cast(amount as string) end amount",'currency','payment_method',"POSTED' status")
gx=g.filter('id between 8000 and 8207').withColumn('txn_id',
    F.format_string('TXNG%06d',F.col('id')-8000))
lx=l.filter('id between 9000 and 9192').withColumn('txn_id',
    F.format_string('TXNL%06d',F.col('id')-9000))
go=g.unionByName(g.filter('id between 6000 and 6259')).unionByName(gx).drop('id')
lo=l.unionByName(l.filter('id between 7000 and 7129')).unionByName(lx).drop('id')
for d,n in [(go,'gateway'),(lo,'ledger')]:
    d.write.mode('overwrite').option('header',True) \
        .option('ignoreTrailingWhiteSpace',False).csv(f'{VOL}/raw/{n}')
```

BRONZE – LAND IT, CHANGE NOTHING

Two separate Bronze tables, all-string. Do not union them - two systems that disagree stay apart until you have defined agreement. Add `_source_file`, `_ingested_at`, `_row_hash`. Keep the 75 trailing-space `txn_ids` and the 210 comma amounts as they arrived. Bronze gateway = 41,468 rows, Bronze ledger = 41,323.

SILVER – THE DECISIONS YOU MUST MAKE, AND RECORD

Each decision below has a number attached to it. If your number does not match, stop and find out why before you go on — that gap is the finding, not a nuisance.

1. TRIM THE KEY first: exactly 75 ledger `txn_ids` carry trailing whitespace, so `trim()` once on both sides. Skip it and those 75 plus their 75 partners show as breaks - counts of 283 and 268 instead of 208 and 193, and an afternoon spent on healthy transactions.
2. CAST amount to `DECIMAL(14,2)` with `try_cast`, never cast. Gateway: exactly 350 stay NULL - log them and KEEP the rows, because an amount you cannot read is its own kind of break, not a mismatch. Ledger: strip the comma before casting - do it and 0 go NULL, skip it and 210 do, each one a fake amount mismatch.
3. DEDUPE on trimmed `txn_id`, each side separately: gateway 41,468 to 41,208, ledger 41,323 to 41,193. Print all four counts and report them, since a duplicate in a payments file is itself a finding. Map 'SUCCESS' and 'POSTED' to one settled flag here, not in the join.
4. DEFINE A MATCH once, tolerance written down: trimmed `txn_ids` equal and $\text{abs}(\text{gateway_amount} - \text{ledger_amount}) \leq 0.05$, so the 190 paise rows match and exactly 450 amount mismatches remain - 400 fee rows plus 50 keying errors. Add `settlement_date = date(txn_ts)` to both Silver tables.

GOLD – THE SHAPE THAT TRAVELS

One row per `txn_id` on either side: `txn_id`, `match_status`, `gateway_amount`, `ledger_amount`, `amount_diff`, `gateway_ts`, `ledger_ts`, `date_diff_days`, `settlement_date` (the gateway's date, else the ledger's), `merchant_id`, `payment_method`, `break_reason`. `match_status` is one of MATCHED, AMOUNT_MISMATCH, DATE_MISMATCH, GATEWAY_AMOUNT_MISSING, GATEWAY_ONLY, LEDGER_ONLY. 41,000 shared + 208 + 193 = 41,401 rows. The 600 same-day 6-hour gaps are not breaks.

THE TRAP MOST PEOPLE FALL INTO

An INNER JOIN cannot return a row that exists on one side only: it reports nothing missing from a file short 401 rows, and a LEFT JOIN from the gateway finds 208 and hides 193. Use FULL OUTER JOIN, COALESCE key, date and merchant. Never report net alone: 208 against 193, and 50 tenfold errors against 400 fee rows, nearly cancel.

THE THREE QUESTIONS YOUR GOLD LAYER MUST ANSWER

1. Rows in the gateway but not the ledger, and the reverse
2. Rows present in both but with different amounts
3. Total rupee value of the discrepancy, per day

IN SNOWFLAKE – THE HARD ONE, WORKED

Tables this query needs in Snowflake: **SILVER_GATEWAY**, **SILVER_LEDGER**

```
-- Only a FULL OUTER JOIN shows rows missing on either side; COALESCE the key
-- and the date, and keep the 350 unreadable amounts out of the money columns.
SELECT COALESCE(g.settlement_date, l.settlement_date) AS settle_day,
       COUNT_IF(l.txn_id IS NULL) AS gateway_only,
       COUNT_IF(g.txn_id IS NULL) AS ledger_only,
       COUNT_IF(g.txn_id IS NOT NULL AND g.amount IS NULL) AS amount_missing,
       COUNT_IF(ABS(g.amount - l.amount) > 0.05) AS amount_mismatch,
       ROUND(SUM(CASE WHEN g.txn_id IS NOT NULL AND g.amount IS NULL THEN 0
                     ELSE ABS(COALESCE(g.amount,0) - COALESCE(l.amount,0)) END),2) AS gross_break,
       ROUND(SUM(CASE WHEN g.txn_id IS NOT NULL AND g.amount IS NULL THEN 0
                     ELSE COALESCE(g.amount,0) - COALESCE(l.amount,0) END),2) AS net_break
FROM SILVER_GATEWAY g FULL OUTER JOIN SILVER_LEDGER l ON l.txn_id = g.txn_id
GROUP BY settle_day ORDER BY gross_break DESC;
```

YOU ARE DONE WHEN

- ☐ Bronze gateway 41,468 and ledger 41,323; after dedupe $41,468 - 260 = 41,208$ and $41,323 - 130 = 41,193$, the two numbers in the story. Print all six.
- ☐ Exactly 208 GATEWAY_ONLY and 193 LEDGER_ONLY. If you get 283 and 268 you did not trim the key; prove it by joining the two break lists on trim(txn_id).
- ☐ The six match_status counts, MATCHED 39,900 + AMOUNT_MISMATCH 450 + DATE_MISMATCH 300 + GATEWAY_AMOUNT_MISSING 350 + GATEWAY_ONLY 208 + LEDGER_ONLY 193, add to exactly 41,401. Assert it.
- ☐ Gross and net break value differ materially every day. Confirm the 400 fee rows' amount_diff equals 1.77 percent of their gateway amounts to within a rupee for the month.

IF YOU FINISH EARLY

Add a fee rule table - payment method, effective date, fee percent - and reclassify the 400 fee rows as EXPECTED_FEE. The human's daily list falls from 450 items to the 50 that are wrong.

THE SITUATION

Every course counts trips per hour and draws a bar chart. A driver has hours, not trips, and the only question is where an hour of their time is worth most: the unit becomes the minute. All the effort goes into Gold and one division the right way round.

WHERE THE DATA COMES FROM

Databricks ships an nyctaxi sample under /databricks-datasets, but on Free Edition serverless that path is often not readable at all. Try one ls; if it errors, or the folder looks nothing like the tutorial you are copying, stop and use the generator - do not spend the evening on it. Layout and names move between releases: tpep_pickup_datetime against pickup_datetime, PULocationID against pickup lat/long, and distance in miles, not km. printSchema() whatever you do load.

GENERATE IT — THE EXACT SPECIFICATION

Seed 11. zones 60: zone_id Z01-Z60, zone_name, borough n%5 (12 each). trips 500,000 over 28 days from 2025-03-01 (40,320 minutes): trip_id, pickup_ts, dropoff_ts, pickup_zone, dropoff_zone, passenger_count 1-4, payment_type, trip_distance_km = round(exp(0.9+randn*0.7),2), fare_amount, tip_amount, tolls_amount, total_amount. Minutes per km 6.0 in hours 8, 9, 17, 18, 19; 2.0 in hours 2-5; 3.2 otherwise; duration_min = greatest(round(km*rate),1) and dropoff_ts = pickup_ts + duration_min. fare_amount = 30+18*km, per km only, no waiting charge. payment_type CARD 60 / CASH 40; tip_amount = fare*uniform(0.05,0.25) on CARD else 0.00; tolls_amount 75.00 on 2 percent of trips; total_amount = fare + tip + tolls. Inject on disjoint id blocks: 1,800 fare_amount 'NA'; 950 trip_distance_km 0 with duration and fare left as generated; 700 dropoff_ts 5-40 minutes before pickup_ts; 450 pickup_zone 'Z99'; 260 passenger_count 0; then 3,000 duplicate rows to 503,000.

GENERATOR CODE – RUN THIS FIRST, EXACTLY AS WRITTEN

This assumes MY_ID and VOL are already set, as on the second page of this document. Do not change the seed or any of the counts — your checks later depend on them.

```
import pyspark.sql.functions as F; SEED=11
z=spark.range(60).selectExpr("format_string('Z%02d',id+1) zone_id",
    "format_string('Zone %02d',id+1) zone_name",
    "element_at(array('Manhattan','Brooklyn','Queens','Bronx','Staten Island'),'cast(id%5+1 as int)) borough")
z.write.mode('overwrite').option('header',True).csv(f'{VOL}/raw/zones')
t=spark.range(500000).selectExpr('id',"format_string('T%07d',id) trip_id",
    "timestampadd(minute,cast(rand(11)*40320 as int),timestamp'2025-03-01 00:00:00') pickup_ts",
    "format_string('Z%02d',cast(rand(12)*60 as int)+1) pickup_zone",
    "format_string('Z%02d',cast(rand(13)*60 as int)+1) dropoff_zone",
    "cast(rand(14)*4 as int)+1 passenger_count","if(rand(15)<0.6,'CARD','CASH') payment_type",
    "round(exp(0.9+randn(16)*0.7),2) km")
t=t.selectExpr('*', 'hour(pickup_ts) hr').selectExpr('*',
    "cast(greatest(round(km*if(hr in(8,9,17,18,19),6.0,"
    "if(hr between 2 and 5,2.0,3.2))),1) as int) dur",
    'round(30+18*km,2) fare')
t=t.selectExpr('*', "if(payment_type='CARD',round(fare*(0.05+rand(17)*0.20),2),0.00) tip",
    'if(rand(18)<0.02,75.00,0.00) tolls')
raw=t.selectExpr('id','trip_id','pickup_ts',
    "if(id between 30000 and 30699,timestampadd(minute,-cast(rand(19)*36+5 as int),pickup_ts),"
    "timestampadd(minute,dur,pickup_ts)) dropoff_ts",
    "if(id between 40000 and 40449,'Z99',pickup_zone) pickup_zone','dropoff_zone'",
    "if(id between 50000 and 50259,0,passenger_count) passenger_count','payment_type'",
    "if(id between 20000 and 20949,0.0,km) trip_distance_km",
    "if(id between 10000 and 11799,'NA',cast(fare as string)) fare_amount",
    'tip tip_amount','tolls tolls_amount','round(fare+tip+tolls,2) total_amount')
out=raw.unionByName(raw.filter('id between 60000 and 62999'))
out.drop('id').write.mode('overwrite').option('header',True).csv(f'{VOL}/raw/trips')
```

BRONZE – LAND IT, CHANGE NOTHING

Land it as it arrives, every column a string, plus `_source_file`, `_ingested_at`, `_row_hash`. No duration, no miles-to-km conversion, no dropping zero-distance trips here: Bronze lets you prove in three weeks whether the strangeness came in the door or you made it. With the generator, Bronze trips = exactly 503,000 rows.

SILVER – THE DECISIONS YOU MUST MAKE, AND RECORD

Each decision below has a number attached to it. If your number does not match, stop and find out why before you go on — that gap is the finding, not a nuisance.

1. CAST fare_amount, tip_amount, tolls_amount, total_amount to DECIMAL(10,2), trip_distance_km to DOUBLE, passenger_count to INT, pickup_ts and dropoff_ts to TIMESTAMP - try_cast, never cast. Exactly 1,800 fares go NULL: silver_rejects, reason='unparseable_fare'. Exactly 260 rows say passenger_count 0; keep them and log the count, they carry money all the same.
2. DEDUPE on trip_id: 503,000 to exactly 500,000. ORPHANS: exactly 450 rows carry pickup_zone 'Z99' with no zone master row, so no borough and no map position - silver_rejects, reason='unknown_zone'. Print every count.
3. DURATION once: duration_min = whole signed minutes from pickup_ts to dropoff_ts. Exactly 700 come out negative: silver_rejects, reason='negative_duration'. No abs(): a trip that ended before it started means the meter clock is wrong, not a 22-minute trip.
4. DEFINE UNPROFITABLE once, constants in a comment: cost is 20 rupees per trip plus 3.50 per minute, so is_unprofitable when total_amount < 20 + 3.50*duration_min. Keep the 950 zero-distance trips - the minutes are real - but bar them from per-km numbers with distance_ok = false.

GOLD – THE SHAPE THAT TRAVELS

One row per pickup zone per hour of day: pickup_zone, zone_name, borough, pickup_hour (0-23), trips, total_minutes, revenue (SUM of total_amount), revenue_per_hour (revenue / (total_minutes/60)), avg_distance_km (over distance_ok rows only), unprofitable_trips, unprofitable_share, avg_tip_rate. 60 x 24 = 1,440 rows.

THE TRAP MOST PEOPLE FALL INTO

Revenue per hour as AVG(fare per trip / trip hours), not SUM(revenue)/SUM(hours). A one-minute trip earning 48 rupees rates 2,880 an hour, so short-hop zones top the list, sending the driver to the slowest part of town. Second half: the 950 zero-distance trips divide by zero, and NULLIF drops them from the hourly numerator.

THE THREE QUESTIONS YOUR GOLD LAYER MUST ANSWER

1. Revenue per hour of driving, by pickup zone
2. Share of trips that are unprofitable after distance
3. The best and the worst hour of the day to be on the road

IN SNOWFLAKE – THE HARD ONE, WORKED

Tables this query needs in Snowflake: **GOLD_ZONE_HOUR**

```
-- Q1 and Q3: divide the sums, never average per-trip rates. RANK inside each
-- hour shows which zones are worth driving to at that hour.
SELECT pickup_hour, pickup_zone, borough, SUM(trips) AS trips,
       ROUND(SUM(revenue) / NULLIF(SUM(total_minutes)/60.0,0),2) AS rev_per_hr,
       ROUND(100.0*SUM(unprofitable_trips) / NULLIF(SUM(trips),0),2) AS unprof_pct,
       RANK() OVER (PARTITION BY pickup_hour
                   ORDER BY SUM(revenue)/NULLIF(SUM(total_minutes),0) DESC) AS zone_rank
FROM GOLD_ZONE_HOUR
GROUP BY pickup_hour, pickup_zone, borough
QUALIFY zone_rank <= 5 ORDER BY pickup_hour, zone_rank;
```

YOU ARE DONE WHEN

- ☐ Bronze trips = 503,000; Silver = 503,000 - 3,000 dupes - 1,800 unparseable fares - 450 unknown zones - 700 negative durations = 497,050. Print all five.
- ☐ Gold has exactly 1,440 rows - assert 60 x 24. SUM(trips) equals the Silver row count and SUM(revenue) equals SUM(total_amount) over Silver to within a rupee.
- ☐ Hours 8, 9, 17, 18, 19 are the five worst by revenue_per_hour, hours 2, 3, 4, 5 the four best. A rush hour near the top means you divided by trips, not hours.
- ☐ Unprofitable share is just under 3 percent - between 2.5 and 3.5 - and essentially every one of those pickups is in hours 8, 9, 17, 18, 19. A share above 20 percent means your duration is in seconds, not minutes.

IF YOU FINISH EARLY

Split revenue_per_hour into fare and tip, find the zone-hour where tips dominate, and compare CARD against CASH there - the advice may be about payment type, not geography.

THE SITUATION

The corporation reports 91 percent on time and passengers laugh: it counts a bus on time if it reaches the terminus within fifteen minutes, by which point the damage is done. Score the same fleet five minutes at every stop and it drops to about 61. The definition is the analysis. And a bus late at stop twelve did not lose the time there.

WHERE THE DATA COMES FROM

Matched scheduled and actual arrival times are rarely published. Indian city portals carry GTFS static feeds (stop_times.txt = the schedule) but the realtime feed is a live API with no history: a matched pair means weeks of your own collecting. If a file claims both, check the header for scheduled time, actual time, stop sequence and trip id; most have two of the four. The generator plants a known choke point per route.

GENERATE IT – THE EXACT SPECIFICATION

Seed 12. routes 40: route_id R01-R40, depot $n\%8$ (5 each), stops_count = $12+(n\%4)*2$, so ten routes each of 12, 14, 16 and 18 stops; a hidden choke_stop, 5 on R01-R20 and 9 on R21-R40, stays out of the file. stops $10*(12+14+16+18) = 600$ rows: route_id, stop_seq, stop_name, scheduled_offset_min = (stop_seq-1)*6. arrivals 40 routes x 60 service_dates from 2025-07-01 x 8 departures (06, 08, 10, 12, 14, 16, 18, 20) x that route's stops = 288,000 rows: trip_id, route_id, service_date, dep_hour, stop_seq, scheduled_arrival, actual_arrival. Every stop adds uniform(0,0.6) minutes and the choke stop 1.0-2.2 more, all multiplied by 1.0 for the 06-08 departures, 1.2 for 10-14 and 1.5 for 16-20; on top of that 9 percent of trips take one 10-45 minute incident at a random stop. Delay ACCUMULATES: running sum by trip_id over stop_seq, actual_arrival = scheduled_arrival + that sum. Inject on disjoint slices: 1,500 actual_arrival 'NA'; 900 arrivals 3-9 minutes early; 600 route_id overwritten to 'R99'; 400 pushed a whole day, 1,440 minutes; then 2,400 duplicate rows to 290,400.

GENERATOR CODE – RUN THIS FIRST, EXACTLY AS WRITTEN

This assumes MY_ID and VOL are already set, as on the second page of this document. Do not change the seed or any of the counts — your checks later depend on them.

```
import pyspark.sql.functions as F; from pyspark.sql import Window; SEED=12
r=spark.range(40).selectExpr("format_string('R%02d',id+1) route_id",
    "format_string('DEPOT%d',id%8+1) depot","cast(12+(id%4)*2 as int) stops_count",
    "cast(if(id<20,5,9) as int) choke")
r.drop('choke').write.mode('overwrite').option('header',True).csv(f'{VOL}/raw/routes')
s=r.withColumn('stop_seq',F.explode(F.sequence(F.lit(1),F.col('stops_count')))).selectExpr(
    'route_id','stop_seq',"format_string('%s Stop %02d',route_id,stop_seq) stop_name",
    '(stop_seq-1)*6 scheduled_offset_min')
s.write.mode('overwrite').option('header',True).csv(f'{VOL}/raw/stops')
a=(r.withColumn('service_date',F.explode(F.expr("sequence(date'2025-07-01',date'2025-08-29')"))))
    .withColumn('dep_hour',F.explode(F.array(*[F.lit(h) for h in [6,8,10,12,14,16,18,20]])))
    .withColumn('stop_seq',F.explode(F.sequence(F.lit(1),F.col('stops_count'))))
    .selectExpr('route_id','service_date','dep_hour','stop_seq','choke','stops_count',
        "concat_ws('_',route_id,string(service_date),string(dep_hour)) trip_id",
        'if(dep_hour<=8,1.0,if(dep_hour<=14,1.2,1.5)) tod')
a=a.selectExpr('*',"(rand(12)*0.6+if(stop_seq=choke,1.0+rand(13)*1.2,0))*tod add0",
    "if(pmod(hash(trip_id),100)<9 and stop_seq=pmod(hash(trip_id),7),stops_count)+1,"
    "10+rand(14)*35,0) inc")
W=Window.partitionBy('trip_id').orderBy('stop_seq')
O=Window.orderBy('route_id','service_date','dep_hour','stop_seq')
a=(a.withColumn('cum',F.round(F.sum(F.col('add0')+F.col('inc')).over(W)).cast('int'))
    .withColumn('rn',F.row_number().over(O))
    .withColumn('sched',F.expr("timestampadd(minute,(stop_seq-1)*6,"
        "timestampadd(hour,dep_hour,cast(service_date as timestamp))))))
raw=a.selectExpr('rn','trip_id',"if(rn%480=1,'R99',route_id) route_id",'service_date',
    'dep_hour','stop_seq','cast(sched as string) scheduled_arrival',
    "case when rn%192=0 then 'NA'"
    " when rn%720=2 then cast(timestampadd(minute,cum+1440,sched) as string)"
    " when rn%320=3 then cast(timestampadd(minute,-cast(rand(15)*7+3 as int),sched) as string)"
    " else cast(timestampadd(minute,cum,sched) as string) end actual_arrival")
out=raw.unionByName(raw.filter('rn%120=5'))
out.drop('rn').write.mode('overwrite').option('header',True).csv(f'{VOL}/raw/arrivals')
```

BRONZE – LAND IT, CHANGE NOTHING

Land all three files all-string - 1,500 actual arrivals are 'NA'. Add `_source_file`, `_ingested_at`, `_row_hash`. No delay here, no dropping the 400 rollover rows, no verdict on the 900 early arrivals: how many rows the tracker sent broken is a vendor figure. Bronze arrivals = exactly 290,400 rows.

SILVER – THE DECISIONS YOU MUST MAKE, AND RECORD

Each decision below has a number attached to it. If your number does not match, stop and find out why before you go on — that gap is the finding, not a nuisance.

1. CAST scheduled_arrival and actual_arrival to `TIMESTAMP`, stop_seq and dep_hour to `INT`, try_cast never cast. Exactly 1,500 actual arrivals go NULL - no ping is unknown, not late: silver_rejects, reason='no_tracker_ping', out of every numerator and denominator.
2. DEDUPE on (trip_id, stop_seq) - one trip passes one stop once: 290,400 to exactly 288,000. Keying on trip_id alone keeps one stop per trip and loses 95 percent of the data. ORPHANS: exactly 600 rows carry route_id 'R99' - silver_rejects, reason='unknown_route'.
3. IMPOSSIBLE DELAY: exactly 400 rows show about 1,440 minutes. Range-check delay to -60..+180 and reject outside with reason='delay_out_of_range'. Never hunt for 1440: the next tracker bug produces a different wrong value.
4. ON TIME once: delay_min = signed whole minutes actual minus scheduled; on_time when between -1 and +5. Early is not on time: 900 rows arrive 3-9 minutes early. Also incremental_delay = delay_min - LAG(delay_min) over trip_id by stop_seq, first stop 0.

GOLD – THE SHAPE THAT TRAVELS

One row per route-stop per departure hour: route_id, depot, stop_seq, stop_name, dep_hour, arrivals, on_time_arrivals, on_time_share, early_arrivals, avg_delay_min, avg_incremental_delay_min, p90_delay_min. $600 \times 8 = 4,800$ rows, not rolled up to the route, which hides where time is lost.

THE TRAP MOST PEOPLE FALL INTO

Averaging delay_min per stop and blaming the highest. Delay accumulates: the minute lost at stop 5 is still there at stop 18, so raw delay rises to every route's end. Difference it with LAG over stop_seq within a trip and the choke stop stands out about fivefold. Second half: abs() or clamping negatives calls the 900 early buses on time.

THE THREE QUESTIONS YOUR GOLD LAYER MUST ANSWER

1. On-time share per route, using a five-minute threshold
2. Which stop in a route first loses the time?
3. Does punctuality degrade through the day?

IN SNOWFLAKE – THE HARD ONE, WORKED

Tables this query needs in Snowflake: **GOLD_ROUTE_STOP_HOUR**

```
-- Q2: LAG turns the carried delay back into the minutes added at each stop,
-- and QUALIFY keeps the first stop on each route where a whole minute appears.
WITH s AS (SELECT route_id, stop_seq, AVG(avg_delay_min) AS carried
  FROM GOLD_ROUTE_STOP_HOUR GROUP BY route_id, stop_seq),
d AS (SELECT route_id, stop_seq, carried,
  carried - LAG(carried,1,0) OVER (PARTITION BY route_id ORDER BY stop_seq) AS added
  FROM s)
SELECT route_id, stop_seq AS first_loss_stop, ROUND(added,2) AS min_added_here,
  ROUND(carried,2) AS min_carried_from_here FROM d WHERE added > 1
QUALIFY ROW_NUMBER() OVER (PARTITION BY route_id ORDER BY stop_seq) = 1;
```

YOU ARE DONE WHEN

- ☐ Bronze arrivals = 290,400; Silver = 290,400 - 2,400 dupes - 1,500 missing pings - 600 unknown routes - 400 out-of-range = 285,500. Print all six.
- ☐ Gold has exactly 4,800 rows - assert 600 x 8 - SUM(arrivals) equals the Silver count of 285,500 and SUM(early_arrivals) equals 900.
- ☐ The SQL returns exactly 40 rows: R01-R20 all name stop 5, R21-R40 all name stop 9. Naming stop 1 or 2 means you read cumulative delay as per-stop delay.
- ☐ On-time share is near 61 percent overall and falls about 25 points across the bands: roughly 78 for the 06-08 departures, 63 for 10-14, 48 for 16-20. A route's last-stop avg_delay_min equals the sum of its avg_incremental_delay_min, to a minute.
- ☐ Score the terminus rule on Silver - each trip's last stop only, within fifteen minutes - and it returns about 91 percent. Two numbers, one fleet: the gap is the finding, not an error.

IF YOU FINISH EARLY

Count the stops after the choke point where incremental delay is negative - the driver made time back - and list routes with no recovery at all: a timetable problem, not a driver problem.

THE SITUATION

One booking in seven ends cancelled and the dashboard shows one line. It is two businesses: a rider who cancels rebooks in twenty minutes; a driver who cancels leaves someone on a road watching 'searching', and some uninstall. Split by who cancelled, not by the reason text: riders and drivers write the same words.

WHERE THE DATA COMES FROM

Kaggle's Ola- and Uber-style booking CSVs are reposted everywhere, mostly synthetic; names differ between uploads (Booking Status, booking_status, Status), reason wording differs, and several have no surge or fare-multiplier column, making the third question unanswerable. Confirm the header has a cancellation party column, not just a reason - that column is the project.

GENERATE IT – THE EXACT SPECIFICATION

Seed 13. areas 25, A01-A25: city_zone n%5 (5 each), area_type Airport (A01, A02), Tech Park (A06, A11, A16, A21), Residential for the other 19. bookings 300,000 over 2025-04-01 to 2025-05-15: booking_id, booked_ts, pickup_area, vehicle_type Auto/Mini/Sedan/SUV from n%4 (75,000 each), distance_km = round(exp(1.3+randn*0.6),2), surge_multiplier, fare_estimate = round((35+14*km)*surge,2), status, cancelled_by_raw, cancel_reason_text. Surge: in peak hours 8, 9, 18, 19, 20 seven bookings in ten draw 1.5, 1.8 or 2.2; the rest 1.2 with probability 0.15, else 1.0. Driver-cancel odds = (0.05+0.09*(surge-1)) x 2.4 at A01/A02 x 1.6 in hours 0-4; a booking that survives cancels rider-side with 0.07+0.06*(surge-1); the rest complete. status is only CANCELLED or COMPLETED - the party lives in cancelled_by_raw, one of six spellings of DRIVER, RIDER and CUSTOMER; cancel_reason_text is one of twelve strings mapping to five reasons, two differing only in case across the parties. Inject: 1,400 fare_estimate 'NA'; 900 surge written '1,5'; 700 pickup_area set to 'A99'; 500 cancelled rows with cancelled_by_raw empty; 300 COMPLETED rows keep stale reason text; then 2,500 duplicate rows to 302,500.

GENERATOR CODE – RUN THIS FIRST, EXACTLY AS WRITTEN

This assumes MY_ID and VOL are already set, as on the second page of this document. Do not change the seed or any of the counts — your checks later depend on them.

```
import pyspark.sql.functions as F; SEED=13
RS=("array('Driver not responding','driver unreachable','DRIVER ASKED TO CANCEL',"
  "'Driver asked to cancel','Rider not at pickup','rider absent','CUSTOMER NO SHOW',"
  "'Vehicle breakdown','vehicle issue','Waiting too long','ETA too high','Changed my mind')")
PS="array('DRIVER','Driver','driver ','RIDER','Customer','rider')"
ar=spark.range(25).selectExpr("format_string('A%02d',id+1) area_id",
  "element_at(array('North','South','East','West','Central'),cast(id%5+1 as int)) city_zone",
  "case when id<2 then 'Airport' when id%5=0 then 'Tech Park' else 'Residential' end area_type")
ar.write.mode('overwrite').option('header',True).csv(f'{VOL}/raw/areas')
b=spark.range(300000).selectExpr('id',"format_string('BK%07d',id) booking_id",
  "timestampadd(minute,cast(rand(13)*64800 as int),timestamp'2025-04-01 00:00:00') booked_ts",
  "format_string('A%02d',cast(rand(14)*25 as int)+1) pickup_area",
  "element_at(array('Auto','Mini','Sedan','SUV'),cast(id%4+1 as int)) vehicle_type",
  "round(exp(1.3+randn(15)*0.6),2) distance_km")
b=b.selectExpr('*', 'hour(booked_ts) hr').selectExpr('*',
  "if(hr in(8,9,18,19,20) and rand(16)<0.7,element_at(array(1.5,1.8,2.2),"
  "cast(rand(17)*3 as int)+1),if(rand(18)<0.15,1.2,1.0)) surge")
b=b.selectExpr('*', "if(rand(19)<(0.05+0.09*(surge-1))*if(pickup_area in('A01','A02'),2.4,1)"
  "**if(hr<=4,1.6,1), 'D',if(rand(20)<0.07+0.06*(surge-1), 'R', 'C')) w")
b=b.selectExpr('*', "if(w='C', 'COMPLETED', 'CANCELLED') status",
  "if(pmod(hash(booking_id,3),12)+1 in (3,4),if(w='D',3,4),pmod(hash(booking_id,3),12)+1) ri")
b=b.selectExpr('*', "row_number() over (partition by surge order by booking_id) k2",
  "row_number() over (partition by (status='COMPLETED') order by booking_id) k")
raw=(b.withColumn('pickup_area',F.expr("if(id between 100000 and 100699,'A99',pickup_area)"))
  .withColumn('surge_multiplier',F.expr("if(surge=1.5 and k2<=900,'1.5',string(surge))"))
  .withColumn('fare_estimate',F.expr("if(id between 200000 and 201399,'NA',"
  "string(round((35+14*distance_km)*surge,2)))"))
  .withColumn('cancelled_by_raw',F.expr("case when status='COMPLETED' or k<=500 then '"
  f" else element_at({PS},pmod(hash(booking_id),3)+if(w='D',1,4)) end"))
  .withColumn('cancel_reason_text',F.expr(f"case when status='CANCELLED' or k<=300"
  f" then element_at({RS},ri) else '' end")).drop('hr','surge','w','ri','k','k2'))
out=raw.unionByName(raw.filter('id between 250000 and 252499'))
out.drop('id').write.mode('overwrite').option('header',True) \
  .option('ignoreTrailingWhiteSpace',False).csv(f'{VOL}/raw/bookings')
```

BRONZE – LAND IT, CHANGE NOTHING

Land both files all-string with `_source_file`, `_ingested_at`, `_row_hash`; keep `cancel_reason_text` and `cancelled_by_raw` as they arrived: all twelve reason spellings, all six party spellings. No trim, no lower-casing, no mapping. When a thirteenth string arrives, only raw Bronze against your Silver mapping reveals it. Bronze bookings = 302,500.

SILVER – THE DECISIONS YOU MUST MAKE, AND RECORD

Each decision below has a number attached to it. If your number does not match, stop and find out why before you go on — that gap is the finding, not a nuisance.

1. CAST fare_estimate to DECIMAL(10,2), surge_multiplier to DOUBLE, try_cast never cast. Exactly 1,400 fares go NULL - count them, keep the rows; a cancelled booking with no fare is still a cancellation. Swap comma for dot before casting surge, or 900 high-surge rows go NULL.
2. DEDUPE on booking_id: 302,500 to exactly 300,000. ORPHANS: exactly 700 rows carry pickup_area 'A99', no area master row - silver_rejects, reason='unknown_area'; 300,000 to 299,300.
3. MAP THE REASON in Silver only: upper-case, trim, map the twelve raw strings to five categories - DRIVER_UNRESPONSIVE, RIDER_ABSENT, VEHICLE_ISSUE, WAIT_TOO_LONG, CHANGED_MIND. Keep the raw column beside the clean one and an UNMAPPED bucket counted every run.
4. WHO CANCELLED, from cancelled_by_raw only - status never says: trim, upper-case, DRIVER stays 'DRIVER', RIDER and CUSTOMER become 'RIDER'. Exactly 500 arrive empty and read back as NULL, so 'UNKNOWN': in the cancel-rate denominator, out of the split, reported as data quality. On the 300 stale COMPLETED rows status wins and the reason goes NULL.

GOLD – THE SHAPE THAT TRAVELS

One row per pickup area per hour per surge band, bands '1.0', '1.2', '1.5-1.8' and '2.2': pickup_area, area_type, city_zone, booking_hour, surge_band, bookings, cancelled_total, cancelled_by_rider, cancelled_by_driver, cancelled_unknown, cancel_rate, rider_cancel_rate, driver_cancel_rate. The two high bands exist only in the peak hours, so Gold is $25 \times 19 \times 2 + 25 \times 5 \times 4 =$ exactly 1,450 rows, not $25 \times 24 \times 4$.

THE TRAP MOST PEOPLE FALL INTO

'Driver asked to cancel' (a RIDER cancelling after the driver phoned) and 'DRIVER ASKED TO CANCEL' (driver app) differ only in case: lower(trim()) merges them and gives rider cancellations to drivers. The party column is the authority. Second half: dropping the 500 unattributed, or dividing by cancellations not bookings.

THE THREE QUESTIONS YOUR GOLD LAYER MUST ANSWER

1. Cancellation rate by hour, split rider against driver
2. Pickup areas with the worst driver-cancel rate
3. Does surge pricing raise or lower cancellations?

IN SNOWFLAKE – THE HARD ONE, WORKED

Tables this query needs in Snowflake: **GOLD_AREA_HOUR_SURGE**

```
-- Q3: rates as sums over sums, one row per surge band, and LAG for what each
-- step up in surge adds. Unattributed cancellations stay in the denominator.
SELECT surge_band, SUM(bookings) AS bookings,
       ROUND(100.0*SUM(cancelled_total)/NULLIF(SUM(bookings),0),2) AS cancel_pct,
       ROUND(100.0*SUM(cancelled_by_driver)/NULLIF(SUM(bookings),0),2) AS driver_pct,
       ROUND(100.0*SUM(cancelled_by_rider)/NULLIF(SUM(bookings),0),2) AS rider_pct,
       ROUND(100.0*SUM(cancelled_by_driver)/NULLIF(SUM(bookings),0)
         - LAG(100.0*SUM(cancelled_by_driver)/NULLIF(SUM(bookings),0))
         OVER (ORDER BY surge_band),2) AS driver_pts_vs_prev_band
FROM GOLD_AREA_HOUR_SURGE GROUP BY surge_band ORDER BY surge_band;
```

YOU ARE DONE WHEN

- ☐ Bronze bookings = 302,500; Silver = 302,500 - 2,500 dupes - 700 unknown areas = 299,300, with 1,400 NULL fares and 0 NULL surges logged. Print all five.
- ☐ Over cancelled rows, DISTINCT cancel_reason_clean returns exactly 5 values with 0 UNMAPPED and cancelled_by returns exactly 3: DRIVER, RIDER, UNKNOWN. Twelve or six means no mapping ran.
- ☐ Gold has exactly 1,450 rows; on every row cancelled_by_rider + cancelled_by_driver + cancelled_unknown = cancelled_total; across the table SUM(bookings) = 299,300 and SUM(cancelled_unknown) = 500.
- ☐ Driver cancel rate in the 2.2 band is at least 2.5x the 1.0 band, the rider rate rises far less; A01 and A02 lead all 25 areas by a double; hours 0-4 beat hours 5-7.
- ☐ Overall cancel rate lands near 15 percent, the driver and rider halves within two points of each other. A driver share near zero means the 500 UNKNOWN rows were quietly counted as rider cancellations.

IF YOU FINISH EARLY

Add a seven-day trailing driver-cancel rate per pickup area over a day window; flag the first day each area crosses 15 percent - when the supply problem started.

THE SITUATION

Sixty public charging points. Drivers say they wait forty minutes for a bay at the mall stations in the evening; finance says half the network sits idle and should be sold. Both are right, about different stations, and the only report anyone runs is units sold per month. Units sold cannot show a wait: a wait is three cars wanting two bays at the same moment, and you see it only by comparing time intervals against each other.

WHERE THE DATA COMES FROM

No free dataset has EV session start, end and station in one file. Caltech ACN-Data has real connect and disconnect times but needs a registered API token and renames fields between releases. Boulder and Palo Alto extracts come and go, differ by year, and several give only energy per session with no end time, which cannot answer the wait question at all. Confirm column names and timestamp format from the header before you write any code.

GENERATE IT – THE EXACT SPECIFICATION

Seed 14. stations 60: station_id ST001-ST060, city 'CITY0'-'CITY5' by id%6 (10 each). ST001-ST030 are mall points: bays 2, connector_type 'Type2', rated_kw 22.0. ST031-ST060 are highway points: bays 4, 'CCS2', 60.0. sessions 60x90x18 = 97,200 over 2025-03-01..2025-05-29: session_id, station_id, connector_type, start_ts, end_ts, kwh. start_hour drawn from a 48-slot array (19 seven times, 18 and 20 five times each, 8/9/10/17/21 three times each, the other sixteen hours once), start minute 0-59. duration_min = 20+int(rand*100) at a 2-bay station, 10+int(rand*30) at a 4-bay one. kwh = round(duration_min/60*rated_kw*(0.45+rand*0.35),2). PLANT: ST001-ST010, every one of the 90 days, 3 extra sessions all starting at exactly 19:00 and lasting 40 minutes = 2,700 rows, so 99,900. Inject on disjoint slices: 800 kwh 'NA', 400 end_ts 5-30 min before start_ts, 250 end_ts empty, 200 end_ts = start_ts + 3 days; then 1,100 duplicate rows and 300 copies re-stamped station 'ST999' with a new session_id, giving 101,300.

GENERATOR CODE – RUN THIS FIRST, EXACTLY AS WRITTEN

This assumes MY_ID and VOL are already set, as on the second page of this document. Do not change the seed or any of the counts — your checks later depend on them.

```
import pyspark.sql.functions as F
H=[19]*7+[18]*5+[20]*5+[8,9,10,17,21]*3+list(range(8))+[11,12,13,14,15,16,22,23]
A="array("+",".join(str(x) for x in H)+")"
st=spark.range(1,61).selectExpr("id sid","concat('ST',lpad(id,3,'0')) station_id",
    "concat('CITY',id%6) city","if(id<=30,2,4) bays",
    "if(id<=30,'Type2','CCS2') connector_type","if(id<=30,22.0,60.0) rated_kw")
st.drop('sid').write.mode('overwrite').option('header',True).csv(f'{VOL}/raw/stations')
b=(st.crossJoin(spark.range(90).withColumnRenamed('id','d'))
    .crossJoin(spark.range(18).withColumnRenamed('id','k'))
    .selectExpr("","((sid-1)*90+d)*18+k n","cast(rand(1)*60 as int) mi",
        f"element_at({A},cast(rand(14)*48+1 as int)) hr",
        "if(bays=2,20+cast(rand(2)*100 as int),10+cast(rand(3)*30 as int)) dur")
    .selectExpr("station_id","connector_type","dur","cast(n%1944 as int) c",
        "concat_ws('-',station_id,d,k) session_id",
        "timestampadd(minute,cast(d*1440+hr*60+mi as int),timestamp'2025-03-01') s_ts",
        "round(dur/60.0*rated_kw*(0.45+rand(5)*0.35),2) kwh"))
p=(st.filter('sid<=10').crossJoin(spark.range(90).withColumnRenamed('id','d'))
    .crossJoin(spark.range(3).withColumnRenamed('id','k'))
    .selectExpr("station_id","connector_type","40 dur","999 c",
        "concat_ws('-',station_id,d,'q',k) session_id",
        "timestampadd(minute,cast(d*1440+1140 as int),timestamp'2025-03-01') s_ts",
        "round(40/60.0*rated_kw*0.62,2) kwh"))
a=b.unionByName(p).selectExpr("","
    "case when c between 16 and 23 then timestampadd(minute,-5-cast(rand(6)*26 as int),s_ts)"
    " when c between 29 and 32 then timestampadd(minute,cast(dur+4320 as int),s_ts)"
    " else timestampadd(minute,cast(dur as int),s_ts) end e_ts")
o=(a.filter('c between 70 and 75').withColumn('station_id',F.lit('ST999'))
    .withColumn('session_id',F.concat(F.lit('ST999-'),F.col('session_id'))))
out=(a.unionByName(a.filter('c between 40 and 61')).unionByName(o).selectExpr(
    "session_id","station_id","connector_type","cast(s_ts as string) start_ts",
    "case when c between 24 and 28 then '' else cast(e_ts as string) end end_ts",
    "if(c<16,'NA',cast(kwh as string)) kwh"))
out.write.mode('overwrite').option('header',True).csv(f'{VOL}/raw/sessions')
```

BRONZE – LAND IT, CHANGE NOTHING

Land both files with every column a string: 800 kwh values are 'NA', 250 end_ts are empty and 400 sit before their own start, so a typed read either fails under ANSI mode or silently drops the rows this project is about. Add _source_file from input_file_name(), _ingested_at from current_timestamp() and _row_hash. Repair nothing here. Bronze sessions = exactly 101,300 rows, Bronze stations = 60.

SILVER – THE DECISIONS YOU MUST MAKE, AND RECORD

Each decision below has a number attached to it. If your number does not match, stop and find out why before you go on — that gap is the finding, not a nuisance.

1. CAST kwh to DOUBLE and start_ts, end_ts to TIMESTAMP with try_cast, never cast - a plain cast kills the job under ANSI mode. Exactly 800 kwh and exactly 250 end_ts turn NULL, and both counts are findings you write to a data quality table. DEDUPE on session_id: 101,300 falls to exactly 100,200. session_id is unique by construction, so any other number means you keyed on the wrong column.
2. ORPHANS AND BAD INTERVALS: 300 rows carry station 'ST999', which has no row in the stations file - reject to silver_rejects with reason 'unknown_station'. 400 rows end before they start - 'end_before_start'; a negative duration subtracts occupancy from a busy bay. 250 have a NULL end_ts - 'session_never_closed'. Silver = 100,200 - 300 - 400 - 250 = 99,250. Print every count.
3. MARATHONS: exactly 200 sessions run for 3 days. Do not reject them - the bay really was blocked. Clip end_ts to start_ts plus 12 hours, set is_clipped=true and report those 200 and their clipped minutes separately. Then clip every end_ts again at the window edge, 2025-05-30 00:00, so no session spends minutes outside the reporting grid.
4. OCCUPANCY, once and per hour: a session gives each clock hour only the minutes it actually spends inside that hour, so 23:40 to 01:10 is 20 minutes to hour 23, 60 to hour 0 and 10 to hour 1 - not 90 minutes to hour 23. Carry that split, plus max concurrent sessions per station-hour from a sweep line, into Gold.

GOLD – THE SHAPE THAT TRAVELS

One row per station, date and hour, on a full grid so empty hours survive: station_id, city, connector_type, bays, session_date, hour, sessions_started, occupied_minutes, bay_minutes_available (bays*60), utilisation_pct, kwh_delivered, max_concurrent_sessions, queue_minutes (minutes with concurrency above bays). $60 \times 90 \times 24 = 129,600$ rows. utilisation_pct above 100 is legal: it means more cars than bays, which is the wait you were asked to find.

THE TRAP MOST PEOPLE FALL INTO

Everyone buckets a session into its start hour. A session starting 23:40 and running 90 minutes then puts 90 minutes into hour 23, which only has 60 bay-minutes per bay, and hour 0 looks empty. Split each session across the hours it actually spans. Second trap: the overlap self-join matches a session with itself and counts every real pair twice - add a.session_id < b.session_id, or use the sweep line above and skip the join entirely.

THE THREE QUESTIONS YOUR GOLD LAYER MUST ANSWER

1. Utilisation percentage per station per day
2. Peak hour per station, and how far it sits from the network mean
3. Which stations make cars wait, that is, overlap beyond the bay count

IN SNOWFLAKE – THE HARD ONE, WORKED

Tables this query needs in Snowflake: **SILVER_SESSIONS**, **DIM_STATION**

```
-- Q3 sweep line: +1 at each start, -1 at each end; order by ts then delta so an
-- ending session frees its bay before an arriving one takes it. Running sum = bays in use.
WITH ev AS (
  SELECT station_id, start_ts AS ts, 1 AS delta FROM SILVER_SESSIONS
  UNION ALL
  SELECT station_id, end_ts, -1 FROM SILVER_SESSIONS),
run AS (
  SELECT station_id, ts, SUM(delta) OVER (PARTITION BY station_id ORDER BY ts, delta
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS concurrent
  FROM ev)
SELECT r.station_id, s.bays, MAX(r.concurrent) AS peak_concurrent,
  COUNT_IF(r.concurrent > s.bays) AS queue_events
FROM run r JOIN DIM_STATION s ON s.station_id = r.station_id
GROUP BY r.station_id, s.bays
HAVING MAX(r.concurrent) > s.bays
ORDER BY peak_concurrent DESC, queue_events DESC;
```

YOU ARE DONE WHEN

- ☐ Bronze sessions = 101,300 = 97,200 base + 2,700 planted 19:00 rows + 1,100 duplicates + 300 orphans. Compute that sum in code and assert it against the count.
- ☐ Silver = 101,300 - 1,100 - 300 - 400 - 250 = 99,250, which is also 99,900 - 400 - 250. Print both routes and make them agree.
- ☐ Gold = 60 x 90 x 24 = 129,600 rows and SUM(occupied_minutes) equals the total clipped Silver duration to the minute. On every Gold row occupied_minutes ≤ 60 * max_concurrent_sessions; if any row breaks that, you put a whole session into its start hour.
- ☐ All 900 station-days of ST001-ST010 reach max_concurrent_sessions of at least 3 against 2 bays, and hour 19 leads the network on both sessions_started and occupied_minutes.
- ☐ The thirty 4-bay stations average under 12 percent daily utilisation, the thirty 2-bay stations over 40 percent. That gap is the half of the network finance wants to sell.

IF YOU FINISH EARLY

For each moment concurrency passes the bay count, measure the wait as the gap to the next end_ts at that station, then rank stations by total waiting minutes rather than by how often a wait happened.

THE SITUATION

A district hospital books about 500 outpatient appointments a day and roughly one slot in six goes empty. Each empty slot is somebody who waited three weeks for it. Administration already sends SMS reminders and wants two answers: do the reminders work, and whom should the ward clerk phone instead. The same table also carries patient identifiers that an analyst counting empty slots has no clinical reason to see.

WHERE THE DATA COMES FROM

Medical Appointment No Shows (Kaggle, Vitoria, Brazil), about 110,000 rows. Three warnings. The original header is misspelled - several condition columns and the no-show column itself - and mirrors corrected different subsets, so copy the names from the header character by character. The two timestamps use different formats inside the same file. And it is 2016 Brazilian data, not current Indian figures, so quote it as a shape, never as a rate for your own hospital.

GENERATE IT – THE EXACT SPECIFICATION

Seed 15. patients 30,000: patient_id PAT000000-PAT029999, gender 'F' when id%3<2 else 'M', neighbourhood 'N00'-'N19' by id%20, scholarship 1 when id%10=0 (3,000), age = 5+int(rand*80). appointments 120,000: appointment_id A0000000-A0119999, patient_id, department 'DEPT0'-'DEPT7' by id%8, status 'SHOWED' or 'NOSHOW', scheduled_ts spread over 180 days from 2025-01-01 08:00, lead_days drawn from [0 x5, 1 x3, 2 x2, 3, 4, 5, 7, 10, 14, 21, 30, 45, 60], appointment_date = scheduled date + lead_days, sms_received = 0 when lead_days=0 else 1 with probability 0.65. No-show probability = 0.06, +0.24 when lead_days>=7, +0.06 when lead_days is 1-6, +0.05 when age<25, +0.06 on a Saturday, -0.04 when sms_received=1, clipped to 0.02-0.60; that averages about 17 percent. lead_days is NOT written to the file. Inject on disjoint slices: 1,500 sms_received 'NA', 450 appointment_date set 1-5 days BEFORE the scheduled date, 700 ages -1 and 300 ages 215 in the patient file; then 2,400 duplicate appointments and 600 copies re-stamped 'PAT999999' with a new appointment_id, giving 123,000, and 120 duplicate patient rows, giving 30,120.

GENERATOR CODE – RUN THIS FIRST, EXACTLY AS WRITTEN

This assumes MY_ID and VOL are already set, as on the second page of this document. Do not change the seed or any of the counts — your checks later depend on them.

```
import pyspark.sql.functions as F
D=[0]*5+[1]*3+[2]*2+[3,4,5,7,10,14,21,30,45,60]
L="array("+",".join(str(x) for x in D)+")"
p0=spark.range(30000).selectExpr("concat('PAT',lpad(id,6,'0')) patient_id",
    "if(id%3<2,'F','M') gender","concat('N',lpad(id%20,2,'0')) neighbourhood",
    "if(id%10=0,1,0) scholarship","5+cast(rand(15)*80 as int) age",
    "cast(id%1500 as int) pr")
pw=p0.selectExpr("patient_id","gender","neighbourhood","scholarship","pr",
    "case when pr<35 then -1 when pr<50 then 215 else age end age")
pw.unionByName(pw.filter('pr between 100 and 105')).drop('pr').write.mode(
    'overwrite').option('header',True).csv(f'{VOL}/raw/patients')
ap=(spark.range(120000).selectExpr("concat('A',lpad(id,7,'0')) appointment_id",
    "concat('PAT',lpad(cast(rand(1)*30000 as int),6,'0')) patient_id",
    "concat('DEPT',id%8) department","cast(id%4000 as int) ar",
    "timestampadd(minute,cast(rand(2)*259200 as int),timestamp'2025-01-01 08:00:00') sch")
    .selectExpr("","f"element_at({L},cast(rand(3)*20+1 as int)) lead_days")
    .selectExpr("","if(lead_days=0,0,if(rand(4)<0.65,1,0)) sms",
    "case when ar between 300 and 314"
    " then date_add(to_date(sch),-1-cast(rand(5)*5 as int))"
    " else date_add(to_date(sch),cast(lead_days as int)) end appointment_date"))
j=(ap.join(p0.select('patient_id','age'),'patient_id')
    .selectExpr("","0.06+if(lead_days>=7,0.24,0)+if(lead_days between 1 and 6,0.06,0)"
    "+if(age<25,0.05,0)+if(dayofweek(appointment_date)=7,0.06,0)"
    "-if(sms=1,0.04,0) pz")
    .selectExpr("appointment_id","patient_id","department","ar","sch","appointment_date",
    "cast(sms as string) sms_received",
    "if(rand(6)<least(0.60,greatest(0.02,pz)), 'NOSHOW', 'SHOWED') status"))
o=(j.filter('ar between 100 and 119').withColumn('patient_id',F.lit('PAT999999'))
    .withColumn('appointment_id',F.concat(F.lit('X'),F.col('appointment_id'))))
out=(j.unionByName(j.filter('ar<80')).unionByName(o).selectExpr("appointment_id",
    "patient_id","department","cast(sch as string) scheduled_ts",
    "cast(appointment_date as string) appointment_date","status",
    "if(ar between 200 and 249,'NA',sms_received) sms_received"))
out.write.mode('overwrite').option('header',True).csv(f'{VOL}/raw/appointments')
```

BRONZE – LAND IT, CHANGE NOTHING

Land both files with every column a string: 1,500 sms_received values are 'NA' and 1,000 ages are impossible. Add _source_file, _ingested_at and _row_hash. Bronze appointments = 123,000 rows, Bronze patients = 30,120. Mask nothing here - hashing before the dedupe would break it. Bronze is the one layer that holds readable patient identifiers, so it is the one layer you restrict.

SILVER – THE DECISIONS YOU MUST MAKE, AND RECORD

Each decision below has a number attached to it. If your number does not match, stop and find out why before you go on — that gap is the finding, not a nuisance.

1. CAST age and sms_received to INT, scheduled_ts to TIMESTAMP and appointment_date to DATE with try_cast, never cast. Exactly 1,500 sms_received turn NULL: flag them sms_unknown and drop them from the SMS comparison only - they still count in every other total.
2. DEDUPE: appointment_id takes 123,000 to exactly 120,600; patient_id takes 30,120 to exactly 30,000. Print all four counts. ORPHANS AND BAD VALUES: 600 rows carry 'PAT999999', which has no patient row - reject as 'unknown_patient'. 450 have an appointment_date before the scheduled date - reject as 'negative_lead_time'. Silver = 120,600 - 600 - 450 = 119,550.
3. 1,000 patients carry age -1 or 215. Set age NULL and age_band 'Unknown'. Never let an impossible age reach a mean: -1 and 215 together pull the average almost back to where it started, so the damage hides.
4. DEFINE THE BANDS ONCE, in Silver, and never again: lead_days = DATEDIFF(appointment_date, CAST(scheduled_ts AS DATE)) - end date first in Databricks SQL, or every lead comes out negative. lead_band '0','1-2','3-6','7-30','31+'. age_band '0-17','18-29','30-44','45-59','60+','Unknown'. day_of_week from appointment_date, never from the scheduled date.
5. Build patient_key = SHA2(patient_id,256) here and join through it from here on. is_no_show = 1 when status='NOSHOW'. Nothing downstream needs the readable identifier, so nothing downstream gets it.

GOLD – THE SHAPE THAT TRAVELS

A conformed fact table, one row per surviving appointment and no readable identifier: appointment_id, patient_key, department, appointment_date, day_of_week, age_band, gender, lead_days, lead_band, sms_received (0, 1 or NULL), status, is_no_show. Exactly 119,550 rows. Every question below is a GROUP BY over this one table, which is the point: the bands were argued about once, in Silver, not three times in three dashboards.

THE TRAP MOST PEOPLE FALL INTO

Answered as a two-row GROUP BY on sms_received, question two says reminders make things worse by 3 points. No SMS is ever sent for a same-day appointment, so the no-SMS group is half same-day walk-ins who almost always turn up, while the SMS group is full of long waits. You are comparing waiting times, not reminders. Compare inside each lead band and SMS wins by about 4 points everywhere. Do not then average the five band rates either - weight them by appointments.

THE THREE QUESTIONS YOUR GOLD LAYER MUST ANSWER

1. No-show rate by age band, gender and lead time
2. Does an SMS reminder actually change the rate?
3. Which day of the week is worst?

IN SNOWFLAKE – THE HARD ONE, WORKED

Tables this query needs in Snowflake: **SILVER_APPOINTMENT**

```
-- Masking policies need Snowflake Enterprise Edition or above. On a Standard
-- trial this CREATE fails: fall back to the patient_key column built in Silver,
-- drop patient_id from the shared view, and say so in your write-up.
CREATE OR REPLACE MASKING POLICY mask_patient AS (v STRING) RETURNS STRING ->
  CASE WHEN CURRENT_ROLE() = 'CLINICAL_STAFF' THEN v ELSE SHA2(v, 256) END;
ALTER TABLE SILVER_APPOINTMENT MODIFY COLUMN patient_id
  SET MASKING POLICY mask_patient;
-- Q2: the SMS effect, compared inside each lead band so the two groups are alike.
-- The '0' band has no SMS group at all - that is the whole confounder.
SELECT lead_band,
       sms_received,
       COUNT(*) AS appts,
       COUNT_IF(status = 'NOSHOW') AS no_shows,
       ROUND(100.0 * COUNT_IF(status = 'NOSHOW') / NULLIF(COUNT(*), 0), 2) AS pct
FROM SILVER_APPOINTMENT
WHERE sms_received IS NOT NULL
GROUP BY lead_band, sms_received
ORDER BY lead_band, sms_received;
```

YOU ARE DONE WHEN

- ☐ Bronze appointments = 123,000 = 120,000 + 2,400 duplicates + 600 orphans; Bronze patients = 30,120 = 30,000 + 120. Assert both sums in code.
- ☐ Silver appointments = 123,000 - 2,400 - 600 - 450 = 119,550; Silver patients = 30,000, of which exactly 1,000 have a NULL age, being the 700 at -1 plus the 300 at 215.
- ☐ Gold = 119,550 rows and the overall no-show share lands between 16 and 18 percent, near the one-in-six the hospital quotes.
- ☐ Saturday is the worst weekday by four to six points. If it is not, day_of_week came from the scheduled date instead of the appointment date.
- ☐ Naively, SMS looks about 3 points WORSE. Inside every lead band it is about 4 points BETTER. Print both tables side by side - that reversal is the finding.

IF YOU FINISH EARLY

Add each patient's own prior no-show rate over their earlier appointments, framed to 1 PRECEDING so today cannot leak into its own predictor, and test it against age, gender and lead time.

THE SITUATION

Twenty pharmacy branches run out of medicines and nobody notices until a patient is turned away at the counter. Each branch sends one file a day and management believes the feed is complete. It is not: some days no file arrives at all, and a day with no row looks exactly like a day with nothing to report. A measured zero and an assumed zero are different facts, and only one of them is yours to report.

WHERE THE DATA COMES FROM

No reliable public dataset of branch-level daily pharmacy stock exists - stock positions are commercially sensitive. Indian health portals list drug availability irregularly, usually as district aggregates rather than one row per branch per day, with column sets that change between releases. Kaggle pharmacy files hold sales, not stock, and no sale looks identical to no stock. Generate it, and say in your write-up that you generated it.

GENERATE IT – THE EXACT SPECIFICATION

Seed 16. skus 300: sku_id SKU000-SKU299, therapeutic_class 'TC00'-'TC09' by id%10, is_essential 1 when id%4=0 (75), base_demand = 3+id%5 (3 to 7), unit_cost. branches 20: branch_id BR01-BR20, city 'CITY0'-'CITY4' by id%5, plus a hidden delivery day cyc = id%14. stock_daily 300 x 20 x 180 = 1,080,000 rows over 2025-01-01..2025-06-29: branch_id, sku_id, snapshot_date, dispensed_qty, receipts_qty, closing_qty. dispensed_qty = round(base_demand*mx*(0.5+rand)); receipts_qty = 120 on days where day_index%14 = cyc, else 0; closing_qty = 250 + running sum of (receipts - dispensed), negatives kept. mx is 1 everywhere except the PLANT: mx=6 for SKU014, SKU049, SKU094, SKU139, SKU214 and SKU279 on 2025-03-02..2025-03-15, a fever outbreak that empties those six. With mx=1 no SKU ever reaches zero. DELETE: BR07 for 2025-03-10..03-18 (300 x 9 = 2,700 rows) and SKU123 in every branch for 2025-05-01..05-14 (20 x 14 = 280): 2,980 rows never sent, leaving 1,077,020. Inject on disjoint single days: 2,000 closing_qty 'NA', 800 dispensed_qty between -1 and -40, 400 snapshot_date '0000-00-00'; then 3,500 duplicate rows and 600 copies re-stamped 'SKU999', giving 1,081,120.

GENERATOR CODE – RUN THIS FIRST, EXACTLY AS WRITTEN

This assumes MY_ID and VOL are already set, as on the second page of this document. Do not change the seed or any of the counts — your checks later depend on them.

```
import pyspark.sql.functions as F
from pyspark.sql import Window
sku=spark.range(300).selectExpr("cast(id as int) sid",
    "concat('SKU',lpad(id,3,'0')) sku_id",
    "concat('TC',lpad(id%10,2,'0')) therapeutic_class","if(id%4=0,1,0) is_essential",
    "cast(3+id%5 as int) base_demand","round(exp(3.0+randn(16)*0.6),2) unit_cost")
sku.drop('sid').write.mode('overwrite').option('header',True).csv(f'{VOL}/raw/skus')
br=spark.range(1,21).selectExpr("concat('BR',lpad(id,2,'0')) branch_id",
    "concat('CITY',id%5) city","cast(id%14 as int) cyc")
br.drop('cyc').write.mode('overwrite').option('header',True).csv(f'{VOL}/raw/branches')
g=(sku.crossJoin(br).crossJoin(spark.range(180).withColumnRenamed('id','d'))
    .selectExpr("sid","sku_id","branch_id","cast(d as int) d","cyc","base_demand",
        "date_add(date'2025-01-01',cast(d as int)) snapshot_date",
        "if(sid in (14,49,94,139,214,279) and d between 60 and 73,6,1) mx",
        "if(d%14=cyc,120,0) receipts_qty")
    .selectExpr("","case when d=16 and sid<40 then -1-cast(rand(3)*40 as int)"
        " else cast(round(base_demand*mx*(0.5+rand(1))) as int) end dispensed_qty"))
w=Window.partitionBy('branch_id','sku_id').orderBy('d').rowsBetween(
    Window.unboundedPreceding,0)
s=(g.withColumn('closing_qty',(250+F.sum(F.col('receipts_qty')
    -F.col('dispensed_qty')).over(w)).cast('int'))
    .filter("not(branch_id='BR07' and d between 68 and 76)")
    .filter("not(sku_id='SKU123' and d between 120 and 133)"))
o=s.filter('d=12 and sid<30').withColumn('sku_id',F.lit('SKU999'))
out=(s.unionByName(s.filter('d=10 and sid<175')).unionByName(o).selectExpr(
    "branch_id","sku_id","dispensed_qty","receipts_qty",
    "if(d=18 and sid<20,'0000-00-00',cast(snapshot_date as string)) snapshot_date",
    "if(d=14 and sid<100,'NA',cast(closing_qty as string)) closing_qty"))
out.write.mode('overwrite').option('header',True).csv(f'{VOL}/raw/stock_daily')
```

BRONZE – LAND IT, CHANGE NOTHING

Land all three files with every column a string: 2,000 closing quantities are 'NA' and 400 snapshot dates are '0000-00-00', which no date parser accepts. Add `_source_file`, `_ingested_at` and `_row_hash`. Bronze `stock_daily` = exactly 1,081,120 rows, `skus` = 300, `branches` = 20. Build no calendar and fill no gap here: the 2,980 rows nobody ever sent must stay absent, because that absence is the finding.

SILVER – THE DECISIONS YOU MUST MAKE, AND RECORD

Each decision below has a number attached to it. If your number does not match, stop and find out why before you go on — that gap is the finding, not a nuisance.

1. CAST `dispensed_qty`, `receipts_qty` and `closing_qty` to INT and `snapshot_date` to DATE with `try_cast`, never cast: exactly 2,000 `closing_qty` and exactly 400 `snapshot_date` turn NULL. DEDUPE on all three of (`branch_id`, `sku_id`, `snapshot_date`): 1,081,120 falls to exactly 1,077,620. Key on only two of them and 180 days collapse into 1.
2. ORPHANS AND BAD DATES: 600 rows carry 'SKU999', which has no row in the SKU file - reject as 'unknown_sku'. 400 rows have a `snapshot_date` nothing can parse, so they cannot be placed on any calendar - reject as 'bad_snapshot_date'. Silver = 1,077,620 - 600 - 400 = 1,076,620.
3. NEGATIVE DISPENSE: exactly 800 rows have `dispensed_qty` below zero - customer returns posted into the dispense column, which makes the running balance invent stock that was never there. Set those 800 to 0 and `qty_corrected=true`, but keep the row: that day's closing balance was still genuinely reported.
4. THE EXPECTED GRID: build all $300 \times 20 \times 180 = 1,080,000$ (`branch`, `sku`, `date`) keys from a date spine crossed with both masters, then LEFT JOIN the survivors onto it. 2,980 keys have no row at all and 400 more lost their date, so 3,380 cells hold no usable measurement: `feed_missing=true`, every quantity NULL, coverage reported as $1,076,620 / 1,080,000 = 99.69$ percent.

GOLD – THE SHAPE THAT TRAVELS

One row per SKU, branch and date across the whole expected grid: `snapshot_date`, `branch_id`, `city`, `sku_id`, `therapeutic_class`, `is_essential`, `dispensed_qty`, `receipts_qty`, `closing_qty`, `feed_missing`, `is_stock_out` (`closing_qty <= 0 AND NOT feed_missing`), `avg_daily_dispensed_28d`, `days_of_cover`. Exactly 1,080,000 rows, exactly 3,380 with `feed_missing = true`. `is_stock_out` is never true on a `feed_missing` row and never NULL.

THE TRAP MOST PEOPLE FALL INTO

The 2,980 missing rows, wrong two ways. COALESCE `closing_qty` to 0 and BR07 reports 300 SKUs out for 9 days: 2,700 stock-out days invented, and they land in the busiest week of the year. Carry the last balance forward instead and BR07 looks fully stocked right through an outbreak. Neither is a measurement. Mark those cells unknown, quantities NULL, out of both the numerator and the denominator, and print the coverage percentage beside every rate you quote.

THE THREE QUESTIONS YOUR GOLD LAYER MUST ANSWER

1. Stock-out days per SKU per branch
2. SKUs stocked out in 5 or more branches on the same day
3. Days of cover remaining at the current rate of consumption

IN SNOWFLAKE – THE HARD ONE, WORKED

Tables this query needs in Snowflake: **DIM_BRANCH**, **DIM_SKU**, **SILVER_STOCK_DAILY**

```
-- Cross the expected calendar with every branch and SKU, then find the keys that
-- came back empty. ROW_NUMBER, not SEQ4 alone - SEQ4 is not promised to be gapless.
WITH cal AS (
  SELECT DATEADD(day, ROW_NUMBER() OVER (ORDER BY SEQ4()) - 1, DATE '2025-01-01')
  AS snapshot_date
  FROM TABLE(GENERATOR(ROWCOUNT => 180)),
expected AS (
  SELECT c.snapshot_date, b.branch_id, s.sku_id
  FROM cal c CROSS JOIN DIM_BRANCH b CROSS JOIN DIM_SKU s)
SELECT e.snapshot_date, e.branch_id, COUNT(*) AS missing_cells
FROM expected e
LEFT JOIN SILVER_STOCK_DAILY d
  ON d.snapshot_date = e.snapshot_date
  AND d.branch_id = e.branch_id
  AND d.sku_id = e.sku_id
WHERE d.sku_id IS NULL
GROUP BY e.snapshot_date, e.branch_id
ORDER BY missing_cells DESC, e.snapshot_date, e.branch_id;
```

YOU ARE DONE WHEN

- ☐ The expected grid is $300 \times 20 \times 180 = 1,080,000$. Compute that product in code and assert it before anything else.
- ☐ Bronze = $1,081,120 = 1,080,000 - 2,980$ never sent + 3,500 duplicates + 600 orphans; Silver = $1,081,120 - 3,500 - 600 - 400 = 1,076,620$. Print all six numbers.
- ☐ The gap query returns 3,380 empty cells: 2,700 for BR07 across 9 days at 300 SKUs, 280 for SKU123 across 14 days at 20 branches, and 400 rows that arrived but lost their date. 2,980 were never sent; 400 you rejected. Report the two separately.
- ☐ SKU014, SKU049, SKU094, SKU139, SKU214 and SKU279 are each out in at least 10 of the 20 branches on every date from 2025-03-15 to 2025-04-15, and no other SKU stocks out at all in the 180 days. A seventh means your running balance is wrong.
- ☐ No BR07 date between 2025-03-10 and 2025-03-18 is a stock-out anywhere, because BR07 sent nothing at all in that window - the exact days the outbreak was emptying the shelves.

IF YOU FINISH EARLY

days_of_cover = closing_qty over a 28-day trailing average of dispensed_qty, framed to 1 PRECEDING so today's demand cannot cover today. Alert where cover is under 7 days and the branch's next delivery day is more than 7 days away.

THE SITUATION

A municipal complaint portal stamps every action with a time and nobody has ever computed how long anything takes. The commissioner asks two things: which wards are slow, and is the pile growing. There is no answer to either. Counting what is open today is easy. Counting what was open at the end of every past week means rebuilding history from two timestamps, because the status column only ever tells you about today.

WHERE THE DATA COMES FROM

Indian portals publish grievance data unusably. The central system publishes aggregate counts as PDF, and no median and no backlog can be recovered from a total. data.gov.in municipal listings change schema, vanish, and often carry a count with no closure timestamp at all. The closest real file is NYC 311 Service Requests, which does have created and closed dates, agency, complaint type and borough - but its columns have been renamed over the years and it is not an Indian municipality.

GENERATE IT – THE EXACT SPECIFICATION

Seed 17. wards 40: ward_id W01-W40, zone 'Z0'-'Z4' by id%5, 1,500 complaints each. complaints 60,000: complaint_id C000001-C060000, ward_id, category, raised_ts, closed_ts, status ('OPEN' or 'CLOSED', the only two values). 8 categories, 7,500 complaints each, with planted median resolution hours: Stray Animals 12, Streetlight 18, Garbage 30, Water Supply 48, Property Tax 72, Drainage 96, Road Repair 240, Encroachment 360. resolution_hours = median*exp(randn*0.6), so the median of each category is the number above. raised_ts is uniform over the 151 days 2025-01-01..2025-05-31; the observation window closes 2025-06-29 23:59:59, a Sunday, which gives exactly 26 whole Monday-start weeks from 2024-12-30. A complaint is OPEN, with an empty closed_ts, when id%20 is 3, 11 or 17 (9,000 rows, 15 percent) or when its closure would fall after the window. Inject on disjoint slices: 1,200 rows keep status 'CLOSED' but have closed_ts 'NA' (the closure happened, the timestamp was lost), 900 have closed_ts 1-72 hours BEFORE raised_ts, 700 have raised_ts in 1970; then 1,800 duplicate rows and 500 copies re-stamped ward 'W99' with a new complaint_id, giving 62,300.

GENERATOR CODE – RUN THIS FIRST, EXACTLY AS WRITTEN

This assumes MY_ID and VOL are already set, as on the second page of this document. Do not change the seed or any of the counts — your checks later depend on them.

```
import pyspark.sql.functions as F
A="array('Streetlight','Garbage','Water Supply','Drainage','Road Repair',"
A+=" 'Stray Animals','Property Tax','Encroachment')"
M="array(18.0,30.0,48.0,96.0,240.0,12.0,72.0,360.0)"
w=spark.range(1,41).selectExpr("concat('W',lpad(id,2,'0')) ward_id",
    "concat('Z',cast(id%5 as int)) zone")
w.write.mode('overwrite').option('header',True).csv(f'{VOL}/raw/wards')
c=(spark.range(1,60001).selectExpr("cast((id-1) div 7500 as int) ix",
    "cast(id%600 as int) r","concat('C',lpad(id,6,'0')) complaint_id",
    "concat('W',lpad(cast((id-1)%40+1 as int),2,'0')) ward_id",
    "timestampadd(minute,cast(rand(17)*217440 as int),timestamp'2025-01-01') raised_ts")
    .selectExpr("*,f"element_at({A},ix+1) category",
        f"element_at({M},ix+1)*exp(randn(2)*0.6) res_h")
    .selectExpr("*,timestampadd(minute,cast(res_h*60 as int),raised_ts) c_ts")
    .selectExpr("*,case when r%20 in (3,11,17) then 'OPEN'"
        " when c_ts>timestamp'2025-06-29 23:59:59' then 'OPEN' else 'CLOSED' end st0")
    .selectExpr("complaint_id","ward_id","category","r",
        "case when r between 40 and 48 then 'CLOSED'"
        " when r<=13 and r not in (3,11) then 'CLOSED' else st0 end status",
        "cast(if(r between 20 and 26,timestamp'1970-01-05 00:00:00',raised_ts)"
        " as string) raised_ts",
        "case when r between 40 and 48 then cast(timestampadd(minute,"
        "-60-cast(rand(3)*4260 as int),raised_ts) as string)"
        " when r<=13 and r not in (3,11) then 'NA'"
        " when st0='OPEN' then '' else cast(c_ts as string) end closed_ts"))
o=(c.filter('r between 60 and 64').withColumn('ward_id',F.lit('W99'))
    .withColumn('complaint_id',F.concat(F.lit('C9'),F.expr("substr(complaint_id,3)"))))
out=c.unionByName(c.filter('r between 80 and 97')).unionByName(o).drop('r')
out.write.mode('overwrite').option('header',True).csv(f'{VOL}/raw/complaints')
```

BRONZE – LAND IT, CHANGE NOTHING

Land both files with every column a string, both timestamps included: 1,200 closure times are the text 'NA' and 700 raise times sit in 1970. Add `_source_file`, `_ingested_at` and `_row_hash`. Bronze complaints = exactly 62,300 rows, Bronze wards = 40. Compute no resolution time here and do not touch status - Bronze is what arrived, not what you wish had arrived.

SILVER – THE DECISIONS YOU MUST MAKE, AND RECORD

Each decision below has a number attached to it. If your number does not match, stop and find out why before you go on — that gap is the finding, not a nuisance.

1. CAST `raised_ts` and `closed_ts` to `TIMESTAMP` with `try_cast`, never cast: exactly 1,200 `closed_ts` turn NULL. Range-check `raised_ts` against 2025-01-01..2025-05-31 rather than testing for the literal 1970 - next year's bad file will use a different wrong value. DEDUPE on `complaint_id`: 62,300 falls to exactly 60,500.
2. KEEP THE TWO KINDS OF NULL APART. An OPEN complaint has no closure time because it has not been closed. A CLOSED one with an unreadable `closed_ts` had a closure and lost the timestamp. Add `closure_known`: exactly 1,200 rows are status 'CLOSED' with `closure_known=false`. Merge them into the open set and every week's backlog is 1,200 too high.
3. ORPHANS AND BAD TIMES: 500 rows carry ward 'W99', which has no master row - reject as 'unknown_ward'. 700 raise outside the window - 'raised_ts_out_of_range'. 900 close before they were raised - 'closed_before_raised'; a negative duration drags a ward's median below zero and nobody notices. Silver = 60,500 - 500 - 700 - 900 = 58,400.
4. DEFINE RESOLUTION AND BREACH ONCE: `resolution_hours` = `closed_ts` minus `raised_ts`, and only where `status='CLOSED'` AND `closure_known`. An open complaint has no resolution time; it does not have a resolution time of zero. `sla_breach` is true for a closed complaint over 168 hours and for any complaint still open 168 hours before the window closes.

GOLD – THE SHAPE THAT TRAVELS

One row per ward, category and Monday-start week, on a full spine so quiet weeks still appear: `week_start`, `ward_id`, `zone`, `category`, `raised`, `closed`, `open_at_week_end`, `median_resolution_hours`, `p90_resolution_hours`, `breached`, `breach_rate`. $40 \times 8 \times 26$ = exactly 8,320 rows. The medians and p90s are NULL where no complaint closed that week, which is not the same as zero hours.

THE TRAP MOST PEOPLE FALL INTO

Everyone counts rows WHERE `status='OPEN'` and groups by week. `status` is current state and carries no history, so the same complaints reappear in every week; a complaint closed in week 12 really was open in weeks 1 to 11 and your chart will never show it. Backlog is arrivals minus departures, accumulated, built from the two timestamps. Second: the 1,200 rows whose closure time was lost look open. Read them as open and the backlog never comes down.

THE THREE QUESTIONS YOUR GOLD LAYER MUST ANSWER

1. Median resolution time by category and ward
2. Share of complaints breaching a 7-day SLA
3. Backlog - how many complaints were still open at the end of each week

IN SNOWFLAKE – THE HARD ONE, WORKED

Tables this query needs in Snowflake: **SILVER_COMPLAINT**

```
ALTER SESSION SET WEEK_START = 1; -- Monday, so DATE_TRUNC matches the spine
WITH sp AS (SELECT DATEADD(week, ROW_NUMBER() OVER (ORDER BY SEQ4()) - 1,
    DATE '2024-12-30') AS w FROM TABLE(GENERATOR(ROWCOUNT => 26))),
    k AS (SELECT DISTINCT ward_id, category FROM SILVER_COMPLAINT),
    ev AS (
        SELECT DATE_TRUNC('week', raised_ts)::DATE w, ward_id, category, COUNT(*) r, 0 c
        FROM SILVER_COMPLAINT GROUP BY 1, 2, 3
        UNION ALL
        SELECT DATE_TRUNC('week', closed_ts)::DATE, ward_id, category, 0, COUNT(*)
        FROM SILVER_COMPLAINT WHERE closure_known GROUP BY 1, 2, 3)
SELECT sp.w, k.ward_id, k.category,
    COALESCE(SUM(ev.r), 0) AS raised, COALESCE(SUM(ev.c), 0) AS closed,
    SUM(COALESCE(SUM(ev.r), 0) - COALESCE(SUM(ev.c), 0))
    OVER (PARTITION BY k.ward_id, k.category ORDER BY sp.w
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS open_at_week_end
FROM sp CROSS JOIN k
LEFT JOIN ev ON ev.w = sp.w AND ev.ward_id = k.ward_id AND ev.category = k.category
GROUP BY sp.w, k.ward_id, k.category
ORDER BY k.ward_id, k.category, sp.w;
```

YOU ARE DONE WHEN

- ☐ Bronze = 62,300 = 60,000 + 1,800 duplicates + 500 orphans; Silver = 62,300 - 1,800 - 500 - 700 - 900 = 58,400. Print all six numbers.
- ☐ Exactly 1,200 Silver complaints are status 'CLOSED' with closure_known=false, and the count of rows in both that set and the open set is 0.
- ☐ Gold = 40 x 8 x 26 = exactly 8,320 rows. Summed over all wards and categories, open_at_week_end in the last week equals Silver rows minus rows with closure_known=true; if it does not, your two UNION branches disagree.
- ☐ Category medians come out in this order, low to high: Stray Animals, Streetlight, Garbage, Water Supply, Property Tax, Drainage, Road Repair, Encroachment. That is planted, so one out of place means negative durations survived.

IF YOU FINISH EARLY

Split each week's backlog by age at week end - 0-7 days, 8-30, 31 or more - off the same week spine. A backlog of 4,000 that is mostly under a week is a busy department; 4,000 sitting at 31 or more days is a news story.

THE SITUATION

A state health department reports vaccination coverage by district and the numbers do not add up. Dose counts come from the field, one row per district per month. The child population, the denominator, comes from a different office in a different file with a district code the field file does not carry. Join the two on the name and only a third of the districts match. The inner join drops the rest silently: no error, no NULL, just 84 districts that quietly stop existing.

WHERE THE DATA COMES FROM

District-level Indian vaccination files circulated in 2021-2022, but those portals have changed and the surviving mirrors differ in columns and naming. Census figures are from 2011 and many districts have been split or renamed since, so the two sides genuinely do not line up - that is the real version of this problem, not an artificial one. data.gov.in immunisation listings vary release to release and several are state totals; WHO and UNICEF estimates are country-level only.

GENERATE IT – THE EXACT SPECIFICATION

Seed 18. population 120: district_code D001-D120, district_name 'District 001'..'District 120', state by id%8, target_population = 40,000+rand*160,000 but 0 for D110, D113, D116, D119; plus a hidden final_cov = 0.40+n*0.01 for D001-D018 (0.41 to 0.58) and 0.62+rand*0.33 for the rest. vaccination 120 x 12 = 1,440 rows for 2025: district_name, month, doses_dose1, doses_dose2 - and no district code at all. Monthly weights [.04,.05,.06,.08,.10,.11,.12,.11,.10,.09,.08,.06] sum to 1.00, so December cumulative dose-1 coverage equals final_cov exactly. doses_dose1 = round(target_population*final_cov*weight); doses_dose2 = round(doses_dose1*0.78). DIRTY KEY: of D001-D108, id%3=0 writes the name exactly (36), id%3=1 UPPERCASE + ' Dist.' + a trailing space (36), id%3=2 a leading space + lowercase (36). D109-D120 get 12 hand-made spellings ('Distrikt 109', 'Dist. 110 Rural' and so on) no regex repairs. Inject: 60 doses_dose1 'NA' (D091-D105, first 4 months), 36 multiplied by 100 (D061-D072, first 3 months), 6 duplicate population rows (126); then 96 duplicate vaccination rows (D001-D008) and 24 rows from two districts on no list, 'Unknown District' and 'Not Specified', 12 months each: 1,560.

GENERATOR CODE – RUN THIS FIRST, EXACTLY AS WRITTEN

This assumes MY_ID and VOL are already set, as on the second page of this document. Do not change the seed or any of the counts — your checks later depend on them.

```
AL=['Distrikt 109','Dist. 110 Rural','DISTRICT-111','distric 112',
'District 113 Nagar','Dist 114 Taluk','D-115 District','DISTRIKT 116',
'District 117 (New)','Dist-118 South','District 119 East','D 120']
AA="array("+",".join("'" + a + "'" for a in AL)+")"
W=[0.04,0.05,0.06,0.08,0.10,0.11,0.12,0.11,0.10,0.09,0.08,0.06]
WW="array("+",".join(str(x) for x in W)+")"
pop=spark.range(1,121).selectExpr("cast(id as int) id",
"concat('D',lpad(id,3,'0')) district_code","concat('STATE',id%8) state",
"concat('District ',lpad(id,3,'0')) district_name",
"cast(40000+rand(18)*160000 as int) bp",
"if(id<=18,0.40+id*0.01,0.62+rand(1)*0.33) final_cov")
pw=pop.selectExpr("district_code","district_name","state","id",
"if(id in (110,113,116,119),0,bp) target_population")
pw.unionByName(pw.filter('id in (3,21,47,68,84,99)').drop('id').write.mode(
'overwrite').option('header',True).csv(f'{VOL}/raw/population'))
v=(pop.crossJoin(spark.range(12).withColumnRenamed('id','m'))
.selectExpr("id","district_name","cast(m as int) m",
"add_months(date'2025-01-01',cast(m as int)) month",
f"cast(round(bp*final_cov*element_at({WW},cast(m as int)+1) as int) d1")
.selectExpr("id","m","cast(month as string) month","d1",
f"case when id>=109 then element_at({AA},cast(id-108 as int))"
" when id%3=1 then concat(upper(district_name),' Dist. ')"
" when id%3=2 then concat(' ',lower(district_name))"
" else district_name end district_name"))
u=(v.filter('id<=2').selectExpr("id","m","month","d1",
"if(id=1,'Unknown District','Not Specified') district_name"))
out=(v.unionByName(v.filter('id<=8')).unionByName(u).selectExpr("district_name",
"month","case when id between 91 and 105 and m<4 then 'NA'"
" when id between 61 and 72 and m<3 then cast(d1*100 as string)"
" else cast(d1 as string) end doses_dose1",
"cast(round(d1*0.78) as int) doses_dose2"))
out.write.mode('overwrite').option('header',True).option(
'ignoreLeadingWhiteSpace',False).option(
'ignoreTrailingWhiteSpace',False).csv(f'{VOL}/raw/vaccination')
```

BRONZE – LAND IT, CHANGE NOTHING

Land both files with every column a string: 60 doses_dose1 values are 'NA'. Add _source_file, _ingested_at and _row_hash. Bronze vaccination = 1,560 rows, Bronze population = 126. Do not normalise the name here - Bronze keeps it exactly as the field office typed it, trailing space and all. Read the CSV with ignoreLeadingWhiteSpace and ignoreTrailingWhiteSpace both false, or Spark eats the very defect you came for.

SILVER – THE DECISIONS YOU MUST MAKE, AND RECORD

Each decision below has a number attached to it. If your number does not match, stop and find out why before you go on — that gap is the finding, not a nuisance.

1. CAST doses_dose1, doses_dose2 and target_population to INT and month to DATE with try_cast, never cast: exactly 60 dose-1 values turn NULL. D110, D113, D116 and D119 carry target_population 0. A zero denominator has no coverage, so set it NULL and report 'denominator missing' - never 0, which divides to infinity or, worse, to a plausible-looking number.
2. NORMALISE THE KEY ONCE, the same way on both sides: trim, collapse runs of whitespace to one space, lowercase, then strip a trailing ' dist.' or ' district'. Call the result district_key. That repairs the 36 uppercase and the 36 lowercase variants. Matches go from 36 to 108, not to 120, and stopping here is the trap.
3. THE 12 STRING FUNCTIONS CANNOT FIX: D109-D120 are spelled differently, not formatted differently. Build a 12-row alias table, alias_key to canonical_key, by hand, keep it in version control beside the code, and apply it after normalisation. Only now do all 120 match. A human wrote those 12 rows and a human signs off on changes to them.
4. DEDUPE only after the key is clean: (district_key, month) takes vaccination 1,560 to exactly 1,464; district_code takes population 126 to exactly 120. Reject the 24 rows keyed 'unknown district' and 'not specified' as 'unmapped_district': $1,464 - 24 = 1,440$. NULL the 36 dose-1 values larger than the district's whole target population, reason 'doses_exceed_target'; with the 60 'NA', 96 district-months have no dose-1.

GOLD – THE SHAPE THAT TRAVELS

One row per district per month, $120 \times 12 =$ exactly 1,440: district_code, district_name, state, month, target_population, doses_dose1, doses_dose2, cum_doses_dose1, coverage_pct_dose1, coverage_pct_dose2, mom_change_pp, denominator_missing, series_complete. Exactly 48 rows have denominator_missing and exactly 96 have a NULL doses_dose1. series_complete is false for any district with a gap, so nobody reads its December coverage as final.

THE TRAP MOST PEOPLE FALL INTO

Join on the raw name and nothing appears to go wrong: an INNER JOIN silently omits the 84 districts that do not match. No error, no NULL row, just healthy-looking coverage over a third of the state while 84 districts go unmeasured. Second trap: stopping when trim and lowercase lift matches from 36 to 108 and calling it done. The last 12 are different spellings that no UPPER, no TRIM and no regex will join. LEFT JOIN, then count the misses, every single time.

THE THREE QUESTIONS YOUR GOLD LAYER MUST ANSWER

1. Coverage percentage per district per month
2. Districts still below 60 percent at the end of the period
3. Month-over-month change in coverage, ranked

IN SNOWFLAKE – THE HARD ONE, WORKED

Tables this query needs in Snowflake: **SILVER_VACCINATION**, **DISTRICT_ALIAS_MAP**, **SILVER_POPULATION**

```
-- Join on the normalised key plus the 12-row alias map, never on the raw name.
WITH v AS (
  SELECT COALESCE(m.canonical_key, s.district_key) AS k, s.month,
         SUM(s.doses_dose1) AS doses
  FROM SILVER_VACCINATION s
  LEFT JOIN DISTRICT_ALIAS_MAP m ON m.alias_key = s.district_key
  GROUP BY 1, 2),
cov AS (
  SELECT p.district_code, p.district_name, v.month,
         ROUND(100.0 * SUM(v.doses) OVER (PARTITION BY p.district_code
                                           ORDER BY v.month ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
              / NULLIF(p.target_population, 0), 2) AS coverage_pct
  FROM v JOIN SILVER_POPULATION p ON p.district_key = v.k)
SELECT district_name, month, coverage_pct,
       coverage_pct - LAG(coverage_pct) OVER (PARTITION BY district_code
                                              ORDER BY month) AS mom_change_pp
FROM cov
ORDER BY mom_change_pp DESC NULLS LAST
LIMIT 20;
```

YOU ARE DONE WHEN

- ☐ Bronze vaccination = 1,560 = 1,440 + 96 duplicates + 24 unmapped; Bronze population = 126 = 120 + 6. Silver = 1,440 and 120. Print all eight numbers.
- ☐ Count how many of the 120 districts join at three stages - raw name, normalised key, key plus alias map. You must get 36, then 108, then 120. Those three numbers are the entire argument of this project.
- ☐ Gold = 120 x 12 = 1,440 rows, exactly 48 with denominator_missing and exactly 96 with a NULL doses_dose1, being 60 'NA' plus 36 impossible values.
- ☐ 27 districts (D061-D072 and D091-D105) have an incomplete dose-1 series and 4 (D110, D113, D116, D119) have no denominator, leaving 89 districts with a computable December figure. Exactly 18 of them, D001-D018, finish under 60 percent.

IF YOU FINISH EARLY

For names still unmatched, propose the three closest canonical names by EDITDISTANCE or JAROWINKLER_SIMILARITY into a review table with a confidence score. A human approves each one. Fuzzy matching suggests; it never decides.

THE SITUATION

A utility bills households on monthly units, and the monthly unit is the only number anyone ever looks at. But the grid is built for the worst half-hour of the year, somewhere around 7 PM on a hot evening. Three hundred units spread evenly through a month is cheap to serve; the same three hundred all arriving at 7 PM is expensive. Build the load curve, find the peak half-hour, and find the households whose own peak lands somewhere else entirely.

WHERE THE DATA COMES FROM

UCI's Individual Household Electric Power Consumption is real and about two million minute-level rows. Three checks before you trust it. It is one household, so the 500-home question and the off-peak-home question cannot be answered from it at all. Circulating copies are semicolon-delimited with missing values written as '?', so a naive read gives you one wide column and zero NULLs. And date and time sit in two separate columns that you must combine yourself.

GENERATE IT – THE EXACT SPECIFICATION

Seed 19. homes 500: home_id H0001-H0500, city 'CITY0'-'CITY4' by id%5, has_ac 1 when id%5<2 (200), tariff_slab id%4, sanctioned_load_kw = 2+id%7, plus a hidden base = 0.25+(id%7)*0.05. readings 500 x 365 x 96 = 17,520,000 fifteen-minute rows of 2025, written as Parquet: home_id, reading_ts, kw, voltage, meter_status. kw = round(base*shape*season*ac*(0.75+rand*0.5),3). shape is a 24-hour curve [.5,.4,.4,.4,.4,.5,.8,1.1,1.2,1,.9,.9,1,1,.9,.9,1,1.4,2.2,2.8,2.4,1.7,1.1,.7] with +0.25 added on the single half-hour starting 19:00, so 19:00 is the strict peak and never ties with 19:30. season 1.45 Apr-Jun, 1.15 Mar and Jul-Sep, 0.90 Feb and Oct-Nov, 0.80 Dec-Jan. ac 1.6 for has_ac homes in Apr-Jun during hours 13-16 and 22-02 only, which never touches the 19:00 peak. PLANT: H0476-H0500, 25 homes, rotate the shape 7 hours forward so their own peak lands at 02:00. DELETE H0123 for 2025-08-01..08-04 = 4 x 96 = 384 rows, leaving 17,519,616. Inject on disjoint 15-minute slices, every home: 25,000 kw 'NA', 8,000 meter_status 'FAULT' with kw 0, 12,000 kw multiplied by 1,000; then 40,000 duplicate rows and 5,000 copies of H0001 re-stamped 'H9999', giving 17,564,616.

GENERATOR CODE – RUN THIS FIRST, EXACTLY AS WRITTEN

This assumes MY_ID and VOL are already set, as on the second page of this document. Do not change the seed or any of the counts — your checks later depend on them.

```
import pyspark.sql.functions as F
S=[.5,.4,.4,.4,.4,.5,.8,1.1,1.2,1.,.9,.9,1.,1.,.9,.9,1.,1.4,2.2,2.8,2.4,1.7,1.1,.7]
SS="array("+",".join(str(x) for x in S)+")"
h=spark.range(1,501).selectExpr("concat('H',lpad(id,4,'0')) home_id",
    "concat('CITY',id%5) city","if(id%5<2,1,0) has_ac","cast(id%4 as int) tariff_slab",
    "cast(2+id%7 as int) sanctioned_load_kw","0.25+(id%7)*0.05 base","id>=476 rot")
h.drop('base','rot').write.mode('overwrite').option('header',True) \
    .csv(f'{VOL}/raw/homes')
r=(h.crossJoin(spark.range(35040).withColumnRenamed('id','slot'))
    .selectExpr("home_id","base","has_ac","cast(slot as int) sl",
        "timestampadd(minute,cast(slot*15 as int),timestamp'2025-01-01') ts",
        "if(rot,(cast(slot%96 as int) div 2+34)%48,cast(slot%96 as int) div 2) hh")
    .selectExpr("","f"element_at({SS},hh div 2+1)+if(hh=38,0.25,0) shape",
        "case when month(ts) in (4,5,6) then 1.45 when month(ts) in (3,7,8,9) then 1.15"
        " when month(ts) in (2,10,11) then 0.90 else 0.80 end season",
        "if(has_ac=1 and month(ts) in (4,5,6)"
        " and hour(ts) in (13,14,15,16,22,23,0,1,2),1.6,1.0) ac")
    .selectExpr("home_id","sl","cast(ts as string) reading_ts",
        "round(230+randn(1)*8,1) voltage",
        "round(base*shape*season*ac*(0.75+rand(19)*0.5),3) kw")
    .filter("not(home_id='H0123' and sl between 20352 and 20735)"))
b=r.selectExpr("home_id","reading_ts","sl","cast(voltage as string) voltage",
    "if(sl between 2000 and 2015,'FAULT','OK') meter_status",
    "case when sl between 1000 and 1049 then 'NA'"
    " when sl between 2000 and 2015 then '0'"
    " when sl between 3000 and 3023 then cast(round(kw*1000,3) as string)"
    " else cast(kw as string) end kw")
o=(b.filter("home_id='H0001' and sl between 5000 and 9999")
    .withColumn('home_id',F.lit('H9999')))
out=b.unionByName(b.filter('sl between 4000 and 4079')).unionByName(o).drop('sl')
out.write.mode('overwrite').parquet(f'{VOL}/raw/readings')
```

BRONZE – LAND IT, CHANGE NOTHING

Land both files with every column a string: 25,000 kw values are 'NA'. Add `_source_file`, `_ingested_at` and `_row_hash`. Bronze readings = exactly 17,564,616 rows, Bronze homes = 500. Do not repair the unit errors and do not drop the FAULT rows here - the count of faulty meter-intervals is itself a number the metering team has been asking for. No cache and no persist anywhere: Free Edition is serverless.

SILVER – THE DECISIONS YOU MUST MAKE, AND RECORD

Each decision below has a number attached to it. If your number does not match, stop and find out why before you go on — that gap is the finding, not a nuisance.

1. CAST kw and voltage to DOUBLE and reading_ts to TIMESTAMP with `try_cast`, never cast: exactly 25,000 kw values turn NULL. DEDUPE on (home_id, reading_ts): 17,564,616 falls to exactly 17,524,616. This is 17 million rows on a serverless warehouse, so write it once and read it back; do not try to cache or persist it.
2. ORPHANS AND DEAD METERS: 5,000 rows carry home 'H9999', which has no master row - reject as 'unknown_home', leaving exactly 17,519,616, which is also $500 \times 365 \times 96 - 384$. Separately, 8,000 rows have meter_status 'FAULT' with kw 0: set kw NULL and keep the row. A dead meter's zero is not a quiet house, and averaging it in drags the whole curve down.
3. UNIT ERROR: exactly 12,000 rows have kw above 50, which is impossible against a sanctioned load of 2 to 8 kW - somebody wrote watts into a kilowatt column. Divide those by 1,000, set `unit_corrected=true` and keep them; deleting them would remove 12,000 real intervals. Then assert that no surviving kw exceeds $3 \times \text{sanctioned_load_kw}$.
4. DEFINE THE HALF-HOUR AND THE SEASON ONCE: `half_hour_start` truncates `reading_ts` to 30 minutes, so two readings make one half-hour and its value is their AVERAGE, never their sum. kwh for one reading is $\text{kw} \times 0.25 \text{ hours}$. season: Winter Dec-Feb, Spring Mar, Summer Apr-Jun, Monsoon Jul-Sep, PostMonsoon Oct-Nov.

GOLD – THE SHAPE THAT TRAVELS

One row per home per half-hour on a full $500 \times 365 \times 48$ spine = exactly 8,760,000 rows: home_id, city, tariff_slab, has_ac, sanctioned_load_kw, half_hour_start, hour, half_hour_index (0-47), season, is_weekend, avg_kw, kwh, readings_present (0, 1 or 2) and `unit_corrected_readings`. Exactly 192 rows have `readings_present = 0`, all of them H0123 on 2025-08-01..08-04, and their avg_kw is NULL rather than 0.

THE TRAP MOST PEOPLE FALL INTO

kW is a rate, not an amount. Four fifteen-minute readings of 1.2 kW are not 4.8 kW, and SUM over a half-hour makes every load curve twice too high and every daily curve four times. Average the kW; multiply by 0.25 hours for kWh. Second trap: clustering on home_id prunes nothing at all when every query filters on time, and Automatic Clustering bills serverless credits even on a trial. Take the before-and-after numbers, then drop the clustering key.

THE THREE QUESTIONS YOUR GOLD LAYER MUST ANSWER

1. Average load curve by hour of day, by season
2. The 20 highest network half-hours of the year
3. Homes whose own peak half-hour is far from the network peak

IN SNOWFLAKE – THE HARD ONE, WORKED

Tables this query needs in Snowflake: **GOLD_HOME_HALF_HOUR**

```
-- Load Gold into Snowflake ordered by home_id, so that before clustering every
-- micro-partition spans the whole year and a date filter can prune nothing.
-- Run Q2 first, record the numbers, then cluster and run it again.
ALTER TABLE GOLD_HOME_HALF_HOUR CLUSTER BY (TO_DATE(half_hour_start));
SELECT half_hour_start, SUM(avg_kw) AS network_kw
FROM GOLD_HOME_HALF_HOUR
WHERE half_hour_start >= '2025-04-01':TIMESTAMP_NTZ
  AND half_hour_start < '2025-07-01':TIMESTAMP_NTZ
GROUP BY half_hour_start
ORDER BY network_kw DESC
LIMIT 20;
-- Proof, and the only place these two columns exist. Read it after each run.
SELECT query_id, partitions_scanned, partitions_total,
       ROUND(100.0 * partitions_scanned / NULLIF(partitions_total, 0), 1) AS pct_scanned
FROM TABLE(INFORMATION_SCHEMA.QUERY_HISTORY())
ORDER BY start_time DESC
LIMIT 10;
ALTER TABLE GOLD_HOME_HALF_HOUR DROP CLUSTERING KEY; -- stop the credit burn
```

YOU ARE DONE WHEN

- ☐ Bronze = 17,564,616 = 17,520,000 - 384 deleted + 40,000 duplicates + 5,000 orphans. Silver = 17,564,616 - 40,000 - 5,000 = 17,519,616 = 500 x 365 x 96 - 384.
- ☐ Silver holds exactly 25,000 kw nulled by try_cast, 8,000 more nulled because the meter read FAULT, and 12,000 rows with unit_corrected=true. Gold = 500 x 365 x 48 = 8,760,000 rows with exactly 192 at readings_present = 0.
- ☐ The network curve peaks at the half-hour starting 19:00 in every one of the five seasons, and the Summer figure at 19:00 is at least 1.4 times the Winter figure at 19:00. Both are planted.
- ☐ Exactly 25 homes, H0476-H0500, have their own peak half-hour at 02:00, and no other home does. Rank homes by the gap between their peak half-hour and 19:00 and those 25 sit alone at the top.
- ☐ Record partitions_scanned and partitions_total for the same Q2 before and after clustering. Scanned should fall sharply; total may also move, because reclustering rewrites micro-partitions. Print all four numbers, not a ratio.

IF YOU FINISH EARLY

Compute each home's coincidence factor - its own peak load divided by its draw at the network peak half-hour - and rank homes by it against their total units, to find who is expensive to serve rather than who simply uses a lot.

THE SITUATION

An inverter that fails rings an alarm. An inverter running 8 percent down rings nothing: it makes power all day and quietly loses money for twenty-five years. Output rises and falls with the sun, so a bad sky and a bad inverter look identical. The only way to tell them apart is to compare each inverter against the others standing in the same field, under the same sky, on the same day. Four of these forty-four are 8 percent down. Nobody has noticed.

WHERE THE DATA COMES FROM

The usual public reference is Kaggle's 'Solar Power Generation Data': two Indian plants, about 34 days, 15-minute readings, four files - check that yourself, mirrors differ. DATE_TIME is formatted differently in the Plant 1 and Plant 2 files, so one parse pattern silently NULLs a whole plant. SOURCE_KEY is an opaque code, not a name. DAILY_YIELD is a cumulative counter that resets at midnight, not interval yield, so summing it gives roughly a hundred times too much. IRRADIATION lives in the weather file, one row per plant per timestamp, so join on plant and timestamp. Read all four headers first.

GENERATE IT – THE EXACT SPECIFICATION

Seed 20. inverters 44: ids 0-21 PLANT_1, 22-43 PLANT_2, 22 each; inverter_id = 'INV_' + plant_id + '_' + lpad(id%22+1,2); rated_w 9000. derate = 0.92 for ids 6, 14, 25 and 41, = INV_PLANT_1_07/15 and INV_PLANT_2_04/20, else round(0.98+rand*0.04,3). derate is never written to any file. irradiation = greatest(0, sin((slot/4-6)*pi/12.75)) times a plant-day factor of 0.72 to 1.00; above 0.04 on exactly slots 25-74 (06:15-18:30), 0 on the other 46. weather 2x34x96 = 6,528 rows over 2025-05-15..06-17: plant_id, ts, ambient_temp, module_temp, irradiation. generation 44x34x96 = 143,616 rows: ts, plant_id, inverter_id, dc_power = round(irradiation*9000*derate*(0.97+rand*0.06),2), ac_power = round(dc_power*0.978,2). Injected on disjoint daylight slices: 300 rows get ac_power = dc_power*1.4, 900 get dc_power = 'NA', 1,400 rows are duplicated, and 500 fresh rows arrive on inverter_id 'INV_X999', each with its own timestamp, giving 145,516. Weather gets 240 irradiation = -0.02 and 120 duplicated rows, giving 6,648.

GENERATOR CODE – RUN THIS FIRST, EXACTLY AS WRITTEN

This assumes MY_ID and VOL are already set, as on the second page of this document. Do not change the seed or any of the counts — your checks later depend on them.

```
def wr(df, p): df.write.mode('overwrite').option('header', True).csv(f'{VOL}/raw/{p}')
P = "case when id<22 then 'PLANT_1' else 'PLANT_2' end"
D = "case when id in (6,14,25,41) then 0.92 else round(0.98+rand(20)*0.04,3) end"
inv = spark.range(44).selectExpr("id", f"{P} plant_id", f"{D} derate",
    f"concat('INV_',{P},'_',format_string('%02d',id%22+1)) inverter_id")
wr(inv.selectExpr("inverter_id", "plant_id", "9000 rated_w"), 'inverters')
CS = "(0.72+pmod(d*37+if(plant_id='PLANT_2',13,0),29)/100.0)"
IR = f"round(greatest(0.0,sin((slot/4.0-6.0)*3.14159/12.75))*{CS},4)"
TS = "timestampadd(minute,cast(d*1440+slot*15 as int),timestamp'2025-05-15')"
g = (inv.crossJoin(spark.range(34).withColumnRenamed('id', 'd'))
    .crossJoin(spark.range(96).withColumnRenamed('id', 'slot'))
    .selectExpr(f"cast({TS} as string) ts", "plant_id", "inverter_id",
        "if(slot between 25 and 74,pmod(((id*34+d)*50+slot-25)*30011,74800),-1) md",
        f"round({IR}*9000*derate*(0.97+rand(22)*0.06),2) dcp"))
b = g.selectExpr("ts", "plant_id", "inverter_id", "md",
    "if(md between 300 and 1199,'NA',cast(dcp as string)) dc_power",
    "cast(round(dcp*if(md between 0 and 299,1.4,0.978),2) as string) ac_power")
orp = spark.range(500).selectExpr(
    "cast(timestampadd(minute,cast(id*15 as int),timestamp'2025-05-15') as string) ts",
    "'PLANT_1' plant_id", "'INV_X999' inverter_id", "cast(-1 as bigint) md",
    "cast(round(200+id*3.1,2) as string) dc_power",
    "cast(round((200+id*3.1)*0.978,2) as string) ac_power")
wr(b.unionByName(b.filter('md between 1200 and 2599')).unionByName(orp).drop('md'),
    'generation')
w = (spark.createDataFrame([('PLANT_1',), ('PLANT_2',)], 'plant_id string')
    .crossJoin(spark.range(34).withColumnRenamed('id', 'd'))
    .crossJoin(spark.range(96).withColumnRenamed('id', 'slot'))
    .selectExpr("plant_id", f"cast({TS} as string) ts", f"{IR} irr",
        "pmod((if(plant_id='PLANT_2',3264,0)+d*96+slot)*1237,6528) wm")
    .selectExpr("ts", "plant_id", "wm",
        "cast(round(22+irr*8+rand(24)*3,1) as string) ambient_temp",
        "cast(round(22+irr*30+rand(25)*2,1) as string) module_temp",
        "cast(if(wm<240,-0.02,irr) as string) irradiation"))
wr(w.unionByName(w.filter('wm between 240 and 359')).drop('wm'), 'weather')
```

BRONZE – LAND IT, CHANGE NOTHING

Land all three files with every column read as a string, dc_power included: 900 of its values are the text 'NA', and a typed read throws under ANSI mode. Add _source_file from input_file_name(), _ingested_at from current_timestamp() and _row_hash. Repair nothing here, and keep both the 46 night slots per day and the negative irradiation, because O&M needs both counts. Bronze generation = 145,516 rows, Bronze weather = 6,648, Bronze inverters = 44.

SILVER – THE DECISIONS YOU MUST MAKE, AND RECORD

Each decision below has a number attached to it. If your number does not match, stop and find out why before you go on — that gap is the finding, not a nuisance.

1. CAST with try_cast, never cast: dc_power, ac_power, irradiation, ambient_temp and module_temp to DOUBLE, ts to TIMESTAMP. A plain cast kills the job under ANSI mode. Exactly 900 dc_power values land as NULL; write that count to a data quality table before you go any further, after the next step you cannot tell a value that failed to parse from a row that was never sent.
2. DEDUPE: key (inverter_id, ts) for generation and (plant_id, ts) for weather, keeping one row per key with ROW_NUMBER ordered by _ingested_at. Generation 145,516 -> 144,116, so exactly 1,400 go; weather 6,648 -> 6,528, exactly 120. Print all four numbers. The 500 'INV_X999' rows carry timestamps of their own, so dedupe must not touch them: land below 144,116 and you keyed on the wrong column.
3. REJECT to silver_rejects with a reason, in this order: 500 rows whose inverter_id is 'INV_X999', which appears nowhere in the inverters file, reason 'unknown_inverter', leaving 143,616; then 300 rows where ac_power is greater than dc_power, reason 'ac_exceeds_dc', leaving 143,316. Do not clamp the second group. You do not know which of the two numbers is wrong, and a clamped row hides a failed sensor forever.
4. DAYLIGHT and ENERGY, defined here once. Join weather on (plant_id, ts). Set the 240 negative irradiation readings to NULL, not 0: a broken sensor is not darkness. is_daylight = irradiation > 0.02, true for exactly 50 of the 96 slots on every plant-day. Drop a slot for every inverter of a plant when that plant's irradiation is NULL: you cannot normalise against an unknown sky. ac_kwh = ac_power/1000/4.

GOLD – THE SHAPE THAT TRAVELS

One row per inverter per day: inverter_id, plant_id, gen_date, daylight_slots, dc_kwh, ac_kwh, inverter_efficiency (ac_kwh/dc_kwh over the slots where dc_power is not NULL), plant_irradiation (that plant-day's summed irradiation), specific_yield (ac_kwh/plant_irradiation), peer_median_specific_yield (median across that plant's 22 inverters that day) and perf_ratio (specific_yield divided by peer_median). Exactly $44 \times 34 = 1,496$ rows.

THE TRAP MOST PEOPLE FALL INTO

Dividing by irradiation with no daylight guard. At 03:00 both numbers are zero, 0/0 is NULL, a COALESCE turns it into 0, and every inverter looks broken all night; at first light irradiation 0.004 gives a yield in the thousands. Filter is_daylight in Silver and never divide below 0.02. Second trap: normalising on AVG. The four inverters you are hunting sit inside that average, so it sags toward them and hides part of the gap you were asked to measure.

THE THREE QUESTIONS YOUR GOLD LAYER MUST ANSWER

1. Yield per inverter per day, normalised by that plant's irradiation
2. Inverters consistently below their own plant's median, not the fleet average
3. Total kWh lost to underperformance across the 34 days

IN SNOWFLAKE – THE HARD ONE, WORKED

Tables this query needs in Snowflake: **GOLD_INVERTER_DAY**

```
-- Q2/Q3: the peer group is the other inverters in the same plant on the same day, so
-- normalise on MEDIAN. AVG is dragged down by the very inverters you are hunting.
WITH d AS (
  SELECT gen_date, plant_id, inverter_id, specific_yield, plant_irradiation,
         MEDIAN(specific_yield) OVER (PARTITION BY plant_id, gen_date) AS peer_median
  FROM GOLD_INVERTER_DAY
  WHERE daylight_slots >= 40)
SELECT plant_id, inverter_id, COUNT(*) AS days,
       ROUND(AVG(specific_yield / NULLIF(peer_median, 0)), 4) AS perf_ratio,
       ROUND(SUM(GREATEST(peer_median - specific_yield, 0) * plant_irradiation), 1)
       AS kwh_lost
FROM d
GROUP BY plant_id, inverter_id
HAVING AVG(specific_yield / NULLIF(peer_median, 0)) < 0.95
ORDER BY kwh_lost DESC;
```

YOU ARE DONE WHEN

- ☐ Bronze generation = 145,516 = 143,616 base + 1,400 duplicates + 500 'INV_X999' rows, and 143,616 = 44 x 34 x 96. Compute that product in code and assert it against the count.
- ☐ Silver = 145,516 - 1,400 duplicates - 500 unknown inverters - 300 ac-above-dc = 143,316, with the 900 NULL dc_power values logged separately, never subtracted.
- ☐ Gold = 1,496 rows (44 x 34). daylight_slots is 40 or more on every row - it is 50 minus the slots that plant lost to NULL irradiation - and SUM(ac_kwh) matches SUM(ac_power)/1000/4 over the daylight Silver rows to within 1 kWh.
- ☐ INV_PLANT_1_07, INV_PLANT_1_15, INV_PLANT_2_04 and INV_PLANT_2_20 come out at perf_ratio 0.91 to 0.93 over 34 days each; every other inverter is above 0.97. Exactly four rows pass the HAVING.
- ☐ Swap MEDIAN for AVG and those four ratios all rise by roughly half a point, because the baseline now contains the culprits. Print both: the gap is what the wrong statistic costs you.

IF YOU FINISH EARLY

Add a seven-day trailing perf_ratio with ROWS BETWEEN 6 PRECEDING AND CURRENT ROW and flag the first day it drops below 0.95 and stays there. That date is when a technician should have gone, and the kWh from there to the end of the file is what the delay cost.

THE SITUATION

Every agriculture meeting opens with 'the rains failed, so the yield fell'. For an irrigated crop that sentence is close to meaningless, yet relief policy leans on it everywhere. You have rainfall by month and production by year - two different grains - for districts whose names arrive spelled three ways in the same file. Put a number on the link crop by crop, then name the crops where the number is near zero. Those are the crops where a rainfall-based relief formula pays the wrong people.

WHERE THE DATA COMES FROM

District crop data on data.gov.in, usually mirrored as apy.csv (State_Name, District_Name, Crop_Year, Season, Crop, Area, Production), differs between mirrors: read the header before trusting a column name. Production holds blanks and the text 'NA'; Area holds zeros. District names shift as districts split and are renamed, and the same name occurs in more than one state, so a name is not a key. IMD's rainfall series covers 36 meteorological SUB-DIVISIONS, not districts, so it will not join to a district table without a mapping you build and defend.

GENERATE IT – THE EXACT SPECIFICATION

Seed 61. districts 120 (D001-D120): irrigation_idx 0.150-0.850 by hash of id; state from 8 names by id%8; district_name from 15 names by id/8, so each name appears in every state and is not a key. rainfall_monthly 120x10x12 = 14,400 rows, 2015-2024: rainfall_mm by hash of district, year and month: 120-380 in months 6-9, else 5-50, 1dp. crop_yearly 120x10x5 = 6,000 over Rice, Wheat, Cotton, Sugarcane, Groundnut: area_ha = round(exp(8.2+randn*0.6)); monsoon_mm is Jun-Sep of the same calendar year, read back from the rainfall table; yield = base*(1+gamma*(monsoon_mm-1000)/1000+(1-gamma)*0.4*(irrigation_idx-0.5))*(1+randn*0.08), base/gamma Rice 3.2/0.45, Wheat 3.0/0.05, Cotton 0.55/0.20, Sugarcane 70.0/0.03, Groundnut 1.4/0.50; production_tonnes = round(area_ha*yield,1). district_name in the crop file takes three forms by id%3: as is, upper with a trailing space, lower with a leading space. Injected in 2015-2023 only, on disjoint slices: 200 rainfall_mm = -999, 150 = 'NA', 300 duplicated rainfall rows giving 14,700; 180 production_tonnes empty, 90 area_ha = 0, 120 duplicated crop rows, and 45 rows on district_id 'D999', one per crop and year so all 45 keys differ, giving 6,165.

GENERATOR CODE – RUN THIS FIRST, EXACTLY AS WRITTEN

This assumes MY_ID and VOL are already set, as on the second page of this document. Do not change the seed or any of the counts — your checks later depend on them.

```
OPT = {'header': True, 'ignoreLeadingWhiteSpace': False, 'ignoreTrailingWhiteSpace': False}
def wr(d, p): d.write.mode('overwrite').options(**OPT).csv(f'{VOL}/raw/{p}')
ST = "array('Punjab','Kerala','Odisha','Bihar','Assam','Gujarat','Haryana','Karnataka')"
NM = ("array('Anantapur','Bellary','Nashik','Betul','Salem','Warangal','Etawah','Guntur','"
      "'Hassan','Latur','Rewa','Moga','Erode','Khammam','Bhilwara')")
dt = spark.range(120).selectExpr("id", "round(0.15+pmod(hash(id,11),701)/1000.0,3) ii",
  "format_string('D%03d',id+1) district_id", f"element_at({ST},cast(id%8+1 as int)) state",
  f"element_at({NM},cast(id/8+1 as int)) dn")
wr(dt.selectExpr("district_id", "dn district_name", "state", "ii irrigation_idx"), 'districts')
r = (dt.crossJoin(spark.range(2015, 2025).withColumnRenamed('id', 'yr'))
  .crossJoin(spark.range(1, 13).withColumnRenamed('id', 'mo')).selectExpr("district_id",
  "id", "yr", "mo", "round(if(mo between 6 and 9,120+pmod(hash(district_id,yr,mo),26001)"
  "/100.0,5+pmod(hash(district_id,yr,mo),4501)/100.0),1) mm",
  "if(yr<2024,pmod(((id*9+yr-2015)*12+mo-1)*1237,12960),-1) rm")
RN = "if(rm between 0 and 199,-999',if(rm between 200 and 349,'NA',cast(mm as string)))"
ru = r.selectExpr("district_id", "yr year", "mo month", "rm", f"{RN} rainfall_mm")
wr(ru.unionByName(ru.filter('rm between 350 and 649')).drop('rm'), 'rainfall_monthly')
mon = r.filter('mo between 6 and 9').groupBy('district_id', 'yr').sum('mm')
CR = "array('Rice','Wheat','Cotton','Sugarcane','Groundnut')"
B = "element_at(array(3.2,3.0,0.55,70.0,1.4),cast(ci+1 as int))"
G2 = "element_at(array(0.45,0.05,0.20,0.03,0.50),cast(ci+1 as int))"
y = (mon.join(dt, 'district_id').crossJoin(spark.range(5).withColumnRenamed('id', 'ci'))
  .selectExpr("district_id", "dn", "state", "yr crop_year", "ci", "id",
  f"{B}*(1+{G2}*(sum(mm)-1000)/1000+(1-{G2})*0.4*(ii-0.5))*(1+randn(65)*0.08) yv",
  f"element_at({CR},cast(ci+1 as int)) crop", "round(exp(8.2+randn(64)*0.6)) ah")
  .selectExpr("if(crop_year<2024,pmod(((id*9+crop_year-2015)*5+ci)*1237,5400),-1) cm"))
c = y.selectExpr("district_id", "case when id%3=1 then concat(upper(dn),' ')"
  " when id%3=2 then concat(' ',lower(dn)) else dn end district_name", "state",
  "crop_year", "crop", "cm", "cast(if(cm between 180 and 269,0,ah) as string) area_ha",
  "if(cm between 0 and 179,'',cast(round(ah*yv,1) as string)) production_tonnes")
o = spark.range(45).selectExpr("'D999' district_id", "'Unknown' district_name",
  "'Unknown' state", "cast(2015+id/5 as bigint) crop_year", "cast(-1 as bigint) cm",
  f"element_at({CR},cast(id%5+1 as int)) crop", "'1200' area_ha", "'3120.5' production_tonnes")
wr(c.unionByName(c.filter('cm between 270 and 389')).unionByName(o).drop('cm'), 'crop_yearly')
```

BRONZE – LAND IT, CHANGE NOTHING

Land all three files with every column read as a string, rainfall_mm and production_tonnes included: 150 rainfall values are the text 'NA' and 180 production values are empty. Add _source_file, _ingested_at and _row_hash. Keep -999 exactly as written and do not roll months up into years here: Bronze is what arrived. Bronze rainfall = 14,700, Bronze crop = 6,165, Bronze districts = 120.

SILVER – THE DECISIONS YOU MUST MAKE, AND RECORD

Each decision below has a number attached to it. If your number does not match, stop and find out why before you go on — that gap is the finding, not a nuisance.

1. CAST and SENTINEL: try_cast rainfall_mm, area_ha and production_tonnes to DOUBLE, never cast. The 150 'NA' rainfall values and the 180 empty production values become NULL, and so must the 200 rows holding -999: a gauge that failed, not a month without rain, and -999 passes try_cast happily. 350 of 14,400 district-months end up unknown, 2.4 percent. Carry rainfall_months_known.
2. DEDUPE: rainfall on (district_id, year, month), 14,700 -> 14,400; crop on (district_id, crop_year, crop), 6,165 -> 6,045. Then reject the 45 'D999' rows, reason 'unknown_district', leaving 6,000, and the 90 rows with area_ha = 0, reason 'zero_area', leaving 5,910; you cannot divide by zero hectares. The 45 orphans carry 45 different keys, so dedupe leaves every one for the join to catch.
3. GRAIN RECONCILIATION. Rainfall is monthly, crops yearly, and crop year Y runs June Y to May Y+1. monsoon_rainfall_mm for Y is June to September of calendar Y; annual_rainfall_mm is all twelve months of that window; rainfall_months_known counts the non-NULL ones. Rainfall stops in December 2024, so crop year 2024 has seven of twelve months: reject those 600 rows, reason 'incomplete_rainfall_year', leaving 5,310.
4. YIELD, defined once: yield_t_per_ha = production_tonnes / area_ha, computed only where area_ha > 0 and production_tonnes is not NULL. A NULL production is unknown, not zero. COALESCE those 180 to zero and you have invented 180 failed harvests in real districts, and every correlation below moves. Join on district_id, never on district_name.

GOLD – THE SHAPE THAT TRAVELS

One row per district per crop per crop year: district_id, district_name (canonical, from the districts file), state, crop, crop_year, area_ha, production_tonnes, yield_t_per_ha, monsoon_rainfall_mm, annual_rainfall_mm, rainfall_months_known, irrigation_idx. 5,310 rows = 120 districts x 5 crops x 9 crop years, less the 90 zero-area rows.

THE TRAP MOST PEOPLE FALL INTO

Joining rainfall to crops on the calendar year. Crop year Y runs June Y to May Y+1: kharif is sown in June and rabi in November of Y and cut in March Y+1, so a January-to-December sum hands every rabi row the wrong monsoon and flattens every correlation, Rice included. Second: -999 passes try_cast without a word; two sentinel months leave a district looking 2,000 mm drier. Third: LAG over a series with holes compares 2015 with 2017 in silence - check that crop_year minus its lag is 1.

THE THREE QUESTIONS YOUR GOLD LAYER MUST ANSWER

1. Yield per hectare by district and crop, for each crop year
2. Correlation between monsoon rainfall and yield, per crop
3. Districts where yield rose in a year when the monsoon fell

IN SNOWFLAKE – THE HARD ONE, WORKED

Tables this query needs in Snowflake: **GOLD_DISTRICT_CROP_YEAR**

```
-- Q3: LAG on the reconciled crop-year grain puts last year's yield and last year's
-- monsoon in the same row, so divergence becomes a subtraction. Complete years only.
WITH y AS (
  SELECT district_id AS d, state AS s, crop AS c, crop_year AS cy,
         yield_t_per_ha AS yv, monsoon_rainfall_mm AS mv
  FROM GOLD_DISTRICT_CROP_YEAR
  WHERE rainfall_months_known = 12 AND yield_t_per_ha IS NOT NULL),
l AS (
  SELECT d, s, c, cy, yv, mv,
         LAG(yv) OVER (PARTITION BY d, c ORDER BY cy) AS yp,
         LAG(cy) OVER (PARTITION BY d, c ORDER BY cy) AS cyp,
         LAG(mv) OVER (PARTITION BY d, c ORDER BY cy) AS rp
  FROM y)
SELECT c AS crop, d AS district_id, s AS state, cy AS crop_year,
       ROUND((yv - yp) / NULLIF(yp, 0) * 100, 1) AS yield_pct,
       ROUND((mv - rp) / NULLIF(rp, 0) * 100, 1) AS rain_pct
FROM l
WHERE cy - cyp = 1 AND yp > 0 AND rp > 0 AND yv > yp * 1.05 AND mv < rp * 0.90
ORDER BY yield_pct DESC;
```

YOU ARE DONE WHEN

- ☐ Bronze rainfall = 14,700 and Bronze crop = 6,165. Silver rainfall = 14,700 - 300 duplicates = 14,400 = 120 x 10 x 12; assert the product, not just the count.
- ☐ Silver crop = 6,165 - 120 duplicates - 45 unknown districts - 90 zero-area = 5,910, less the 600 crop-year-2024 rows = 5,310 rows in Gold. Print all six.
- ☐ Exactly 350 rainfall values are NULL in Silver, 200 from the sentinel and 150 unparseable, and no rainfall value anywhere is negative. Assert that 200 + 150 = 350.
- ☐ CORR(monsoon_rainfall_mm, yield_t_per_ha) per crop, over rows with rainfall_months_known = 12: Rice and Groundnut above +0.45, Cotton +0.15 to +0.40, Wheat and Sugarcane between -0.10 and +0.20. A high Wheat number means you joined on the wrong year.
- ☐ COUNT(DISTINCT district_name) in the crop file is 45, against 120 district_id values: fifteen names, each written three ways. Group on the name and 120 districts become 45.

IF YOU FINISH EARLY

Split monsoon_rainfall_mm into its four months and correlate yield against each one, per crop; for Rice, July and August normally beat June and September. Then run the same correlation on the top and bottom third of irrigation_idx separately, and see how much of the rainfall link irrigation buys back.

THE SITUATION

A city station posts a six-month mean AQI of 186 and the report files it as 'Moderate'. Inside that mean sit thirty-nine days above 200, days when schools should have stopped outdoor games - and a fortieth day, in the middle of the worst spell of the winter, on which the station sent only twelve hours, so it counts as nothing at all. An average hides runs, and it is the run that puts a child in a clinic. Count the days above the line first, then work out which of them stood next to each other.

WHERE THE DATA COMES FROM

CPCB's CCR portal serves station data through a manual export form rather than a bulk download, station ids are not consistent from year to year, and whole days go missing with nothing to mark them. OpenAQ v3 needs a free API key and returns pollutant measurements, never a computed AQI. Treat any column already named 'AQI' with suspicion: AQI is derived, from CPCB sub-index breakpoints applied to 24-hour means, worst sub-index winning. The breakpoint table has been revised before, so quote the one you used in your README.

GENERATE IT – THE EXACT SPECIFICATION

Seed 22. stations 10, STN01-STN10 for Delhi, Kanpur, Patna, Lucknow, Jaipur, Kolkata, Mumbai, Pune, Hyderabad, Chennai, with hourly base PM2.5 of 68, 62, 60, 58, 40, 38, 30, 26, 24, 20 and PM10 ratios of 1.8, 2.6, 2.0, 2.1, 3.4, 1.9, 2.0, 1.8, 2.2, 1.9. aq_hourly $10 \times 180 \times 24 = 43,200$ rows from 2024-10-01: $pm25 = base * episode * diurnal * (0.8 + rand * 0.4)$, where diurnal is 1.30 for hours 22, 23 and 0-8 and 0.85 otherwise, and episode is 2.6 inside a planted window and 1.0 outside; $pm10 = pm25 * ratio * (0.95 + rand * 0.1)$. Planted windows: Delhi six of them, 40 days in all, the longest 2024-12-10 to 12-20; Patna one of 18 days, 2024-12-08 to 12-25; Kanpur 6 and 7 days; Lucknow 7; Jaipur 5; Kolkata 4; the other four cities none. Deleted: hours 0-11 of 24 city-days, one of which is Delhi 2024-12-14, plus all 24 hours of Pune 2025-01-12 and of Chennai 2024-12-03, 336 rows in all, leaving 42,864. Injected on surviving rows only, at most one per city-day and never on a deleted hour: 900 $pm25 = 'NA'$, 700 $pm25 = '-999'$, 1,300 duplicated rows, and 400 fresh rows on station_id 'STN_X99' with timestamps of their own, giving 44,564.

GENERATOR CODE – RUN THIS FIRST, EXACTLY AS WRITTEN

This assumes MY_ID and VOL are already set, as on the second page of this document. Do not change the seed or any of the counts — your checks later depend on them.

```
def wr(d, p): d.write.mode('overwrite').option('header', True).csv(f'{VOL}/raw/{p}')
CB = [(0,'STN01','Delhi',68,1.8), (1,'STN02','Kanpur',62,2.6), (2,'STN03','Patna',60,2.0),
      (3,'STN04','Lucknow',58,2.1), (4,'STN05','Jaipur',40,3.4), (5,'STN06','Kolkata',38,1.9),
      (6,'STN07','Mumbai',30,2.0), (7,'STN08','Pune',26,1.8), (8,'STN09','Hyderabad',24,2.2),
      (9,'STN10','Chennai',20,1.9)]
st = spark.createDataFrame(CB, 'i int, station_id string, city string, b int, r10 double')
wr(st.selectExpr("station_id", "city"), 'stations')
EP = [(0,35,41), (0,52,57), (0,70,80), (0,86,89), (0,97,105), (0,116,118), (1,45,50),
      (1,95,101), (2,68,85), (3,72,78), (4,60,64), (5,100,103)]
HD = [74,140,8,20,130,90,10,150,120,30,160,50,5,170,15,145,25,155,35,165,45,175,55,115]
HC = [0,0,0,1,1,1,2,2,2,3,3,3,4,4,5,5,6,6,7,7,8,8,9,9]
EQ = " ".join(f"when i={c} and d between {a} and {b} then 2.6" for c, a, b in EP)
HQ = " or ".join(f"(i={c} and d={x})" for c, x in zip(HC, HD))
FQ = "(i=7 and d=103) or (i=9 and d=63)"
J = "((i*180+d)-if(i*180+d>1363,1,0)-if(i*180+d>1683,1,0))"
aq = (st.crossJoin(spark.range(180).withColumnRenamed('id', 'd'))
     .crossJoin(spark.range(24).withColumnRenamed('id', 'h'))
     .selectExpr("station_id", "b", "r10", "h",
                  "timestampadd(hour,cast(d*24+h as int),timestamp'2024-10-01') ts",
                  f"case {EQ} else 1.0 end mu", f"if({HQ},1,0) hf", f"if({FQ},1,0) fl",
                  f"pmod({J}*7,1798) v", f"pmod({J}*11,6)+12 nh",
                  f"if(pmod({J}*13,1798)<1300,pmod({J}*5,6)+18,-1) dh")
     .filter("not((hf=1 or fl=1) and h<12) and not(fl=1 and h>=12)")
     .selectExpr("cast(ts as string) ts", "station_id", "v", "h", "nh", "dh", "r10",
                  "round(b*mu*if(h>=22 or h<=8,1.30,0.85)*(0.8+rand(22)*0.4),1) p"))
q = aq.selectExpr("ts", "station_id", "if(h=dh,1,0) dupf",
                  "case when h<>nh then cast(p as string) when v<900 then 'NA' when v<1600 then '-999'"
                  " else cast(p as string) end pm25",
                  "cast(round(p*r10*(0.95+rand(23)*0.1),1) as string) pm10")
o = spark.range(400).selectExpr(
    "cast(timestampadd(hour,cast(id as int),timestamp'2024-10-01') as string) ts",
    "'STN_X99' station_id", "0 dupf", "cast(round(55+id*0.3,1) as string) pm25",
    "cast(round(110+id*0.6,1) as string) pm10")
wr(q.unionByName(q.filter('dupf=1')).unionByName(o).drop('dupf'), 'aq_hourly')
```

BRONZE – LAND IT, CHANGE NOTHING

Land both files with every column read as a string: 900 pm25 values are the text 'NA' and 700 are -999. Add `_source_file`, `_ingested_at` and `_row_hash`. Do not fill the deleted hours and do not invent rows for them - the count of missing hours is itself one of the findings you owe the city. Bronze `aq_hourly` = 44,564 rows, Bronze stations = 10.

SILVER – THE DECISIONS YOU MUST MAKE, AND RECORD

Each decision below has a number attached to it. If your number does not match, stop and find out why before you go on — that gap is the finding, not a nuisance.

1. CAST with `try_cast`, never cast: pm25 and pm10 to DOUBLE, ts to TIMESTAMP. Exactly 900 pm25 values fail to parse and become NULL; then turn the 700 rows holding -999 into NULL as well. That is 1,600 unusable hours out of 42,864, 3.7 percent. Record the two counts apart: one is a parse failure and one is a sentinel a sensor wrote on purpose, and different people fix them.
2. DEDUPE and ORPHANS: key on (station_id, ts). Keying on (city, ts) gives the same answer today and halves your data the morning a city gets its second station. 44,564 -> 43,264, exactly 1,300 gone. The 400 rows carrying station_id 'STN_X99' match nothing in the stations file and so have no city: reject them, reason 'unknown_station', leaving 42,864. Then 42,864 + 336 unreported hours = 43,200 = 10 x 180 x 24.
3. DAY COMPLETENESS decides every run length below. Roll the hours up to a daily mean per station. A day is COMPLETE only with 18 or more non-NULL pm25 hours; otherwise it is INCOMPLETE and its AQI stays NULL, never zero. Exactly 26 city-days come out INCOMPLETE: two at 0 valid hours and twenty-four at 11 or 12. Every other day has 23 or 24. Print the distribution of hours_valid.
4. AQI, defined once. PM2.5 daily mean 0-30 -> 0-50, 31-60 -> 51-100, 61-90 -> 101-200, 91-120 -> 201-300, 121-250 -> 301-400, above 250 -> 401-500. PM10: 0-50 -> 0-50, 51-100 -> 51-100, 101-250 -> 101-200, 251-350 -> 201-300, 351-430 -> 301-400, above 430 -> 401-500. Interpolate inside the band. AQI is the higher sub-index, dominant_pollutant the winner, is_breach = AQI > 200.

GOLD – THE SHAPE THAT TRAVELS

One row per station per day: station_id, city, obs_date, hours_valid, avg_pm25, avg_pm10, aqi_pm25, aqi_pm10, aqi, dominant_pollutant, is_breach, day_status. 10 x 180 = 1,800 rows, built against a full calendar so that a day with no rows at all still appears. Keep the INCOMPLETE days in the table: dropping them is exactly how a two-week outage becomes invisible.

THE TRAP MOST PEOPLE FALL INTO

An INCOMPLETE day is not a clean day. Delhi 2024-12-14 kept only hours 12 to 23, the calmer half, so a twelve-hour mean reads about a tenth low; here it still breaches, but on a marginal day those same twelve hours turn a breach into a clean day and the mean alone will never tell you which. Second: build the island key from the date, not from the row number alone, and never from MAX(date) - MIN(date) + 1, which spans the first breach to the last and calls the whole winter one run.

THE THREE QUESTIONS YOUR GOLD LAYER MUST ANSWER

1. Days above the 'poor' threshold, per city per month
2. Longest run of consecutive breach days, per city
3. Which pollutant drives the breach most often, and where

IN SNOWFLAKE – THE HARD ONE, WORKED

Tables this query needs in Snowflake: **GOLD_CITY_DAY**

```
-- Q2: gaps and islands. Number the breach days per city in date order; inside a run
-- the date minus the row number is constant, and at any gap it changes. INCOMPLETE
-- days are excluded, so an unmeasured day breaks a run instead of being assumed clean.
WITH b AS (
  SELECT city, obs_date,
         ROW_NUMBER() OVER (PARTITION BY city ORDER BY obs_date) AS rn
  FROM GOLD_CITY_DAY
  WHERE is_breach AND day_status = 'COMPLETE'),
i AS (
  SELECT city, obs_date, DATEADD(day, -rn, obs_date) AS island_key FROM b),
r AS (
  SELECT city, island_key, COUNT(*) AS run_days,
         MIN(obs_date) AS run_start, MAX(obs_date) AS run_end
  FROM i GROUP BY city, island_key)
SELECT city, run_days, run_start, run_end
FROM r
QUALIFY ROW_NUMBER() OVER (PARTITION BY city ORDER BY run_days DESC, run_start) = 1
ORDER BY run_days DESC;
```

YOU ARE DONE WHEN

- ☐ Bronze aq_hourly = 44,564 = (43,200 - 336 deleted) + 1,300 duplicates + 400 orphans. Compute 10 x 180 x 24 in code rather than typing 43,200 in by hand.
- ☐ Silver = 44,564 - 1,300 - 400 = 42,864, and 42,864 + 336 = 43,200. Exactly 1,600 pm25 values are NULL: 900 unparseable plus 700 sentinel. Assert all five of those numbers.
- ☐ Gold has exactly 1,800 rows and exactly 26 with day_status INCOMPLETE - two at hours_valid = 0 and twenty-four at 11 or 12. No COMPLETE day has fewer than 23 valid hours.
- ☐ 86 breach days in all: Delhi 39, Patna 18, Kanpur 13, Lucknow 7, Jaipur 5, Kolkata 4, and none for Mumbai, Pune, Hyderabad or Chennai. PM10 is dominant_pollutant on Kanpur's and Jaipur's 18, PM2.5 on the other 68.
- ☐ Longest strict runs: Patna 18, Delhi 9, Kanpur 7, Lucknow 7, Jaipur 5, Kolkata 4. Delhi's December window is 11 days wide but 2024-12-14 is INCOMPLETE, splitting it into 4 and 6. Report 9 and 11 side by side.

IF YOU FINISH EARLY

Add a second island definition that tolerates one missing or clean day inside a run, and report strict and tolerant lengths side by side per city. Delhi is the case that moves: 9 strict, 11 tolerant. Then say which of the two you would put in front of a school principal, and why.

THE SITUATION

The dashboard says nine lakh plays in nine weeks and the product team calls it growth. A play is logged the moment somebody presses play, so forty seconds of a ninety-minute film counts the same as the whole film. What decides a second season is the minute at which half the audience has gone. Compute that minute per title, then check whether it moves when the same title is watched on a phone instead of a television.

WHERE THE DATA COMES FROM

No public dataset of per-view watch time exists. The platforms that hold it do not publish it, because a session row identifies both the title and the viewer. MovieLens gives ratings with timestamps and no duration; the Netflix Prize files are ratings only; anything advertised online as real session logs is synthetic in a way nobody documented, so you cannot check an answer against it. You generate this one instead, which is the only way to know the true median before you compute it.

GENERATE IT – THE EXACT SPECIFICATION

Seed 73. titles 400, T0001-T0400: genre from eight names by id%8, 50 each; even id 'Film' at 5,400 seconds, odd id 'Episode' at 2,400, 200 each; hook = round(0.70+rand*0.60,3). users 30,000, U00000-U29999, with signup_date, plan (Basic, Standard, Premium by id%3, 10,000 each) and city. view_events 900,000 across nine whole weeks, 2025-03-03 to 2025-05-04, 90,720 minutes. One draw sets device: Mobile below 0.55, TV below 0.85, else Laptop. A second draw bounces the viewer with probability 0.30 Mobile, 0.08 TV, 0.18 Laptop, and a bounce watches 0.002 to 0.012 of the runtime, 5 to 65 seconds. Everyone else watches least(0.99, med*hook*(0.4+rand*1.3)), med 0.50 Mobile, 0.85 TV, 0.65 Laptop; the 0.99 ceiling makes the capped rows countable. seconds_watched = round(runtime_seconds * that fraction). Defects sit on disjoint classes of pmod(id*7919,900000): 1,500 seconds_watched = 'NA', 900 = round(runtime*1.5), 400 = '0', and 3,000 clean rows repeated with their own event_id, giving 903,000. Then 700 rows on title_id 'T9999' with fresh ids E0900000-E0900699: 903,700.

GENERATOR CODE – RUN THIS FIRST, EXACTLY AS WRITTEN

This assumes MY_ID and VOL are already set, as on the second page of this document. Do not change the seed or any of the counts — your checks later depend on them.

```
OPT={'header':True,'ignoreLeadingWhiteSpace':False,'ignoreTrailingWhiteSpace':False}
def wr(d,p): d.write.mode('overwrite').options(**OPT).csv(f'{VOL}/raw/{p}')
GN="array('Drama','Comedy','Thriller','Romance','Action','Docu','Kids','Horror')"
CT="array('Chennai','Pune','Indore','Kochi','Patna','Surat','Nagpur','Jaipur')"
ti=spark.range(400).selectExpr("format_string('T%04d',id+1) title_id",
    "format_string('Title %03d',id+1) title_name","if(id%2=0,'Film','Episode') title_type",
    f"element_at({GN},cast(id%8+1 as int)) genre","if(id%2=0,5400,2400) runtime_seconds",
    "round(0.70+rand(73)*0.60,3) hook")
wr(ti,'titles')
wr(spark.range(30000).selectExpr("format_string('U%05d',id) user_id",
    "date_add(date'2023-01-01',cast(id%800 as int)) signup_date",
    "element_at(array('Basic','Standard','Premium'),cast(id%3+1 as int)) plan",
    f"element_at({CT},cast(id%8+1 as int)) city"),'users')
ev=spark.range(900000).selectExpr("pmod(id*7919,900000) kk","rand(76) r","rand(77) b",
    "rand(78) h","rand(79) g","format_string('E%07d',id) event_id",
    "format_string('U%05d',cast(rand(74)*30000 as int)) user_id",
    "format_string('T%04d',cast(rand(75)*400 as int)+1) title_id",
    "timestampadd(minute,cast(rand(80)*90720 as int),timestamp'2025-03-03') start_ts")
DV="case when r<0.55 then 'Mobile' when r<0.85 then 'TV' else 'Laptop' end"
MD="case when r<0.55 then 0.50 when r<0.85 then 0.85 else 0.65 end"
BP="case when r<0.55 then 0.30 when r<0.85 then 0.08 else 0.18 end"
FR=f"if(b<{BP},0.002+g*0.010,least(0.99,{MD}*hook*(0.4+h*1.3)))"
SW=("case when kk<1500 then 'NA' when kk<2400 then cast(round(runtime_seconds*1.5) as"
    " string) when kk<2800 then '0' else cast(round(runtime_seconds*"+FR+") as string) end")
ev2=ev.join(ti,'title_id').selectExpr("kk","event_id","user_id","title_id","start_ts",
    f"{DV} device",f"{SW} seconds_watched")
op=spark.range(700).selectExpr("cast(-1 as bigint) kk","format_string('E09%05d',id)"
    " event_id","format_string('U%05d',cast(rand(81)*30000 as int)) user_id",
    "'T9999' title_id","timestampadd(minute,cast(id*100 as int),timestamp'2025-03-03')"
    " start_ts","element_at(array('Mobile','TV','Laptop'),cast(id%3+1 as int)) device",
    "'900' seconds_watched")
wr(ev2.unionByName(ev2.filter('kk>=2800 and kk<5800')).unionByName(op).drop('kk'),
    'view_events')
```

BRONZE – LAND IT, CHANGE NOTHING

Land all three files with every column read as a string, `seconds_watched` included: 1,500 of its values are the text 'NA'. Add `_source_file`, `_ingested_at` and `_row_hash`. Do not cap the over-runtime rows and do not drop the zeros here - a player reporting 8,100 seconds of a 5,400 second film is a client bug engineering should see, and this table is your evidence. Bronze `view_events` = 903,700 rows, titles = 400, users = 30,000. Nothing in Gold needs the users file; it is the dimension the stretch asks for.

SILVER – THE DECISIONS YOU MUST MAKE, AND RECORD

Each decision below has a number attached to it. If your number does not match, stop and find out why before you go on — that gap is the finding, not a nuisance.

1. CAST: try `cast seconds_watched` to INT and `start_ts` to TIMESTAMP, never cast - under ANSI mode a plain cast throws on the first 'NA' and takes the notebook with it. Exactly 1,500 values become NULL. An event with no duration cannot feed a median, so move all 1,500 to `silver_rejects` with reason 'unparseable_seconds' rather than reading them as zero seconds watched.
2. DEDUPE: key `event_id`, 903,700 -> 900,700, and print both. A repeated play event is a finding on its own - the client fires the beacon twice, so every engagement number the team has published is inflated. The 700 'T9999' rows carry their own event ids, so dedupe leaves all 700 standing for the join to catch.
3. ORPHANS AND ZEROS: 700 rows carry `title_id` 'T9999', which appears in no titles row, so they have no runtime and no completion - reject with reason 'unknown_title'. 400 rows have `seconds_watched` = 0, a player that never started rather than a view - reject with reason 'zero_watch'. A 12-second event is a real person who left after the logo, and it stays.
4. JOIN AND CAP: join titles on `title_id` so every surviving event carries `runtime_seconds`, `title_type` and genre - Silver is where the runtime arrives. Exactly 900 rows exceed `runtime_seconds`; cap them at runtime, never drop them, and record the 900. $\text{completion_pct} = \text{seconds_watched} / \text{runtime_seconds}$ after capping, so those 900 are the only rows at 1.0. A completed view is $\text{completion_pct} \geq 0.90$, defined once, here.

GOLD – THE SHAPE THAT TRAVELS

One row per title per device per week: `title_id`, `title_name`, `genre`, `title_type`, `runtime_seconds`, `device`, `week_start`, `views`, `completed_views`, `completion_rate`, `avg_seconds_watched`, `p25_seconds`, `p50_seconds`, `p75_seconds`, `median_completion_pct`. `week_start` is the Monday of the event's week. Exactly $400 \times 3 \times 9 = 10,800$ rows - nine whole weeks by construction, and the thinnest cell still holds about 37 events, so no combination goes missing.

THE TRAP MOST PEOPLE FALL INTO

Answering question two with an average. A fifth of all plays end inside the first minute and pile up at the bottom of the range; on TV the mean completion sits about ten points below the median, so the average reports a viewer who does not exist. The median is the minute the audience halves. Second: the 900 uncapped over-runtime rows push a title's median completion above 1.0 - how did 108% get watched? Cap in Silver, before you divide. Third: deleting the bounces because they look like noise. They are the finding.

THE THREE QUESTIONS YOUR GOLD LAYER MUST ANSWER

1. Completion rate per title
2. The minute by which 50% of viewers have left, per title
3. Do mobile viewers leave earlier than TV viewers?

IN SNOWFLAKE – THE HARD ONE, WORKED

Tables this query needs in Snowflake: **SILVER_VIEW_EVENTS**

```
-- Q2/Q3: PERCENTILE_CONT is an ordered-set function. WITHIN GROUP (ORDER BY x) says
-- what is ranked, OVER (PARTITION BY ...) says the group - Snowflake allows neither an
-- ORDER BY nor a frame inside that OVER. CONT interpolates; DISC returns a real value.
SELECT DISTINCT title_id, device, runtime_seconds,
    COUNT(*) OVER (PARTITION BY title_id, device) AS views,
    ROUND(PERCENTILE_CONT(0.50) WITHIN GROUP (ORDER BY seconds_watched)
        OVER (PARTITION BY title_id, device) / 60.0, 1) AS median_minute,
    ROUND(PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY seconds_watched)
        OVER (PARTITION BY title_id, device) / 60.0, 1) AS q1_minute
FROM SILVER_VIEW_EVENTS
QUALIFY COUNT(*) OVER (PARTITION BY title_id, device) >= 200
ORDER BY title_id, device;
```

YOU ARE DONE WHEN

- ❑ Bronze view_events = 903,700. Silver = 903,700 - 3,000 repeats - 1,500 unparseable - 700 unknown titles - 400 zero-watch = 898,100. Print all five numbers and watch the subtraction close.
- ❑ Exactly 900 Silver rows sit at completion_pct = 1.0 and none is above it: the capped set, exactly 900 because the generator ceiling was 0.99.
- ❑ Gold = 10,800 rows exactly, SUM(views) = 898,100, and every row has p25_seconds <= p50_seconds <= p75_seconds.
- ❑ No Silver row lands between 66 and 335 seconds. Below that gap sit the bounces: 21% of all events and 30% of Mobile events. That empty band is the shape a mean cannot show you.
- ❑ median_completion_pct per title runs 0.25-0.52 on Mobile and 0.58-0.99 on TV. completion_rate is 0.00 for over half the titles on Mobile and never above 0.19; on TV it runs 0.14-0.66.

IF YOU FINISH EARLY

Build a retention curve: per title, the share of viewers still watching at each 5% mark of runtime, then find the steepest 5% bucket. It usually sits where the opening credits end. Join the users file and split the curve by plan.

THE SITUATION

Everyone believes attenders do better; the department needs to know where to spend attention. Attendance drives marks hard in one subject and barely at all in another, and until there is a number per subject the rule stays uniform for all six. And a student who falls two hundred places between semesters is worth finding in week three, not on results day.

WHERE THE DATA COMES FROM

UCI Student Performance (student-mat.csv, 395 rows; student-por.csv, 649) is real but semicolon-delimited, graded G1, G2, G3 on the Portuguese 0-20 scale, and its 'absences' column is a raw count with no total, so no attendance percentage can be computed from it. It also holds one subject per file, not six, and no three semesters of one cohort. Kaggle's 'Students Performance in Exams' has no attendance column at all. So you generate a cohort whose true answer you already know.

GENERATE IT – THE EXACT SPECIFICATION

Seed 84. students 1,200, S0001-S1200: branch CSE/ECE/MECH by id%3 (400 each), section SEC1-SEC4 by id%4 (300 each), diligence = round(0.50+rand*0.48,3), and a hidden penalty of 12.0 marks in semester 2 for the 150 students with id%8=3. subjects 6, with beta MATH 0.50, PHYS 0.30, PROG 0.55, DBMS 0.35, ENGL 0.10, WORK 0.05 - WORK is the workshop lab. marks 1,200 x 3 semesters x 6 subjects = 21,600 rows: classes_held 30 for WORK else 45; a hidden true attendance ap = clip(diligence*100 + randn*6, 35, 99); classes_attended = round(classes_held*ap/100); score = clip(round(52 + beta*(ap-75) + randn*9 - penalty), 0, 100); internal_marks = round(score*0.4) out of 40 and external_marks = score - internal out of 60. The file carries no total and no percentage - Silver derives both. Defects sit on disjoint classes of pmod(rn*7919,21600) where rn = id*18 + (semester-1)*6 + subject index: 300 internal_marks = 'AB', 200 classes_attended = classes_held+5, 150 external_marks = 70, 90 classes_held = 0, and 480 rows repeated, giving 22,080. Then 120 rows on unknown ids S9001-S9120, one row each, all keys distinct: 22,200.

GENERATOR CODE – RUN THIS FIRST, EXACTLY AS WRITTEN

This assumes MY_ID and VOL are already set, as on the second page of this document. Do not change the seed or any of the counts — your checks later depend on them.

```
OPT={'header':True,'ignoreLeadingWhiteSpace':False,'ignoreTrailingWhiteSpace':False}
def wr(d,p): d.write.mode('overwrite').options(**OPT).csv(f'{VOL}/raw/{p}')
SB=[(0,'MATH','Engineering Mathematics',0.50),(1,'PHYS','Applied Physics',0.30),
     (2,'PROG','Programming in C',0.55),(3,'DBMS','Database Systems',0.35),
     (4,'ENGL','Technical English',0.10),(5,'WORK','Workshop Practice',0.05)]
sub=spark.createDataFrame(SB,'sj int,subject_code string,subject_name string,beta double')
wr(sub.select('subject_code','subject_name'),'subjects')
stu=spark.range(1200).selectExpr("id","format_string('S%04d',id+1) student_id",
    "if(id%3=0,'CSE',if(id%3=1,'ECE','MECH')) branch",
    "format_string('SEC%d',id%4+1) section",
    "round(0.50+rand(84)*0.48,3) diligence","if(id%8=3,12.0,0.0) penalty")
wr(stu.selectExpr("student_id","branch","section'),'students')
m0=(stu.crossJoin(spark.range(1,4).withColumnRenamed('id','semester')).crossJoin(sub)
    .selectExpr("student_id","semester","subject_code","beta","penalty",
        "pmod((id*18+(semester-1)*6+sj)*7919,21600) kk","if(subject_code='WORK',30,45) hd",
        "round(least(99.0,greatest(35.0,diligence*100+randn(85)*6)),1) ap","randn(86)*9 nz"))
T="least(100.0,greatest(0.0,round(52+beta*(ap-75)+nz-if(semester=2,penalty,0.0))))"
m1=m0.selectExpr("student_id","semester","subject_code","kk","hd",
    "cast(round(hd*ap/100) as int) at",f"cast({T} as int) tm")
mk=m1.selectExpr("student_id","semester","subject_code","kk",
    "cast(if(kk>=650 and kk<740,0,hd) as string) classes_held",
    "cast(if(kk>=300 and kk<500,hd+5,at) as string) classes_attended",
    "if(kk<300,'AB',cast(cast(round(tm*0.4) as int) as string)) internal_marks",
    "cast(if(kk>=500 and kk<650,70,tm-cast(round(tm*0.4) as int)) as string) external_marks")
SC="element_at(array('MATH','PHYS','PROG','DBMS','ENGL','WORK'),cast(id%6+1 as int))"
op=spark.range(120).selectExpr("format_string('S9%03d',id+1) student_id",
    "cast(id%3+1 as bigint) semester",f"{SC} subject_code","cast(-1 as bigint) kk",
    "'45' classes_held","'38' classes_attended","'22' internal_marks","'34' external_marks")
wr(mk.unionByName(mk.filter('kk>=740 and kk<1220')).unionByName(op).drop('kk'),'marks')
```

BRONZE – LAND IT, CHANGE NOTHING

Land all three files with every column read as a string, internal_marks included: 300 of its values are the letters 'AB'. Add _source_file, _ingested_at and _row_hash. Do not compute an attendance percentage here and do not repair the impossible rows - a student marked present at 50 of 45 classes belongs in a report to the exam section, not in a silent UPDATE. Bronze marks = 22,200 rows, students = 1,200, subjects = 6.

SILVER – THE DECISIONS YOU MUST MAKE, AND RECORD

Each decision below has a number attached to it. If your number does not match, stop and find out why before you go on — that gap is the finding, not a nuisance.

1. CAST: try_cast internal_marks, external_marks, classes_held and classes_attended to INT, never cast - a plain cast throws on the first 'AB' under ANSI mode. Exactly 300 internal_marks become NULL. Write the rule down: 'AB' means absent for that test, and an absent student scored 0, not unknown. Convert those 300 to 0, then total_marks = internal_marks + external_marks.
2. DEDUPE and REJECTS: key (student_id, semester, subject_code); 22,200 -> 21,720. Deduping on student_id alone would keep 1 row of every 18. Then reject 120 rows on ids S9001-S9120, reason 'unknown_student', leaving 21,600; then 200 with classes_attended > classes_held as 'attendance_over_100', 90 with classes_held = 0 as 'no_classes_held', and 150 with external_marks > 60 as 'marks_over_max': 21,160.
3. ATTENDANCE: attendance_pct = classes_attended / NULLIF(classes_held,0) * 100. Band it BELOW_60 under 60.0, BAND_60_74 from 60.0 to under 75.0, BAND_75_84 from 75.0 to under 85.0, BAND_85_100 from 85.0 up. Band the unrounded percentage: 74.6 is BAND_60_74, and rounding it to 75 first moves hundreds of rows across the detention line.
4. SEMESTER TOTAL AND RANK: sem_total_marks sums a student's six total_marks in that semester. is_complete is true only when all six subject rows survived Silver; a student missing one is not ranked, because his total is not comparable. sem_rank = RANK() by semester on sem_total_marks DESC over the complete students only - RANK, not DENSE_RANK, so 'fell 20 places' means twenty students.

GOLD – THE SHAPE THAT TRAVELS

One row per student per semester per subject: student_id, branch, section, semester, subject_code, subject_name, classes_held, classes_attended, attendance_pct, attendance_band, internal_marks, external_marks, total_marks, sem_total_marks, is_complete, sem_rank. 21,160 rows. sem_total_marks, is_complete and sem_rank repeat across a student's six rows in a semester - that is expected, and it is why the query below starts with SELECT DISTINCT.

THE TRAP MOST PEOPLE FALL INTO

Partitioning the LAG by semester. RANK() OVER (PARTITION BY semester ...) is right; copying that same OVER clause onto the LAG leaves each student alone inside his semester, so prev_rank is NULL everywhere and the filter returns zero rows - a clean-looking, wrong finding. Second: leaving 'AB' as NULL. SUM skips a NULL, so a student short of one internal quietly loses 20 marks and drops three hundred places, and you will blame his attendance. Third: dividing by classes_held without NULLIF - 90 rows hold zero.

THE THREE QUESTIONS YOUR GOLD LAYER MUST ANSWER

1. Average marks by attendance band, per subject
2. Students whose rank fell 20+ places between semesters
3. Subjects where attendance predicts marks least

IN SNOWFLAKE – THE HARD ONE, WORKED

Tables this query needs in Snowflake: **GOLD_STUDENT_SUBJECT_SEM**

```
-- Q2: rank movement. RANK partitions by semester - it compares one student against
-- the others; LAG partitions by student - it compares him against himself. Copying the
-- first OVER clause onto the LAG is the whole bug, and it fails without an error.
WITH s AS (
  SELECT DISTINCT student_id, branch, semester, sem_total_marks
  FROM GOLD_STUDENT_SUBJECT_SEM WHERE is_complete = TRUE),
r AS (
  SELECT student_id, branch, semester, sem_total_marks,
         RANK() OVER (PARTITION BY semester ORDER BY sem_total_marks DESC) AS sem_rank
  FROM s)
SELECT student_id, branch, semester, sem_rank,
       LAG(sem_rank) OVER (PARTITION BY student_id ORDER BY semester) AS prev_rank,
       sem_rank - LAG(sem_rank) OVER (PARTITION BY student_id ORDER BY semester) AS chg
FROM r
QUALIFY sem_rank - LAG(sem_rank) OVER (PARTITION BY student_id ORDER BY semester) >= 20
ORDER BY chg DESC;
```

YOU ARE DONE WHEN

- Bronze marks = 22,200 = 21,600 + 480 repeats + 120 orphans, and compute 1,200 x 3 x 6 in code rather than typing 21,600.
- Silver = 22,200 - 480 repeats - 120 unknown students - 200 attendance-over-100 - 90 zero-classes - 150 marks-over-max = 21,160 rows in Gold. Confirm 300 'AB' internals became 0.
- Exactly 905 of the 1,200 students are is_complete in both semesters 1 and 2, and exactly 114 of the 150 penalty students (id%8=3) are among them. Print both, then say how many the query could not see.
- More than 90 of those 114 appear in the semester-2 fall list, and most of them again in a semester-3 rise list. A penalty student who was already near the bottom cannot fall 20 places.
- CORR(attendance_pct, total_marks) by subject_code: PROG and MATH above +0.55, DBMS and PHYS between +0.35 and +0.55, ENGL and WORK below +0.20. All six numbers were planted; if yours are flat, check the join.

IF YOU FINISH EARLY

Add a per-student attendance trend with LAG on average attendance_pct across semesters, and ask whether a semester-2 fall in attendance shows up in semester-2 marks or only in semester-3.

THE SITUATION

Batting average is runs divided by dismissals; across twenty overs that is close to meaningless, because fifty balls blocked for forty runs is a fine average and a lost match. Strike rate is better but still one number for three different jobs: surviving the powerplay, rotating through the middle, and swinging at the death. Build a phase-aware measure, then defend it against the minimum-balls cut-off that decides who appears in the list at all.

WHERE THE DATA COMES FROM

IPL ball-by-ball data is real, but the mirrors disagree. Kaggle ships deliveries.csv and matches.csv, where the batter column is 'batsman' in older copies and 'batter' in newer ones, the match key is 'id' in matches.csv but 'match_id' in deliveries.csv, season reads 2008 in one mirror and '2007/08' in another, the extras columns were split twice, and some copies stop several seasons short. Cricsheet ships nested JSON and YAML instead. Read the header and count rows per season before you trust any of it. This brief generates its own instead, so the answers are checkable.

GENERATE IT – THE EXACT SPECIFICATION

Seed 95. players 220, PL001-PL220, team TM01-TM10 by id%10, 22 each. matches 720 = 12 seasons 2014-2025 x 60, M0001-M0720; team1 = TM(id%10+1), team2 = TM((id*7+3)%10+1), never equal, so every team plays 12 matches a season. Each match has 2 innings of 120 legal balls: seq 1-120, over_no = (seq-1)/6+1, phase POWERPLAY overs 1-6, MIDDLE 7-15, DEATH 16-20. The bowler is slot cast(pow(rand,2.0)*8) of the fielding team, so ten lead bowlers get about a third of the overs; the batter is slot cast(pow(rand,1.6)*22) of the batting side. Runs per ball come from probabilities varying by phase and a per-batter aggression 0.70-1.30: P(4) 0.11/0.07/0.15 and P(6) 0.06/0.03/0.13 by phase, times aggression. Six bowlers - PL001, PL013, PL025, PL037, PL049, PL052 - get a tight death profile, roughly half the runs. is_wicket at p=0.035. Plus 18 extras a match, 12 wides and 6 no-balls, on seq 121-129, each reusing the over and ball number of a legal delivery: 720 x 258 = 185,760. Defects on disjoint classes of pmod(rn*7919,172800): 1,500 batter_runs = 'NA', 600 over_no = 21, 800 bowler_id = 'PL999', 2,400 rows repeated. 350 extras carry total_runs = 0 instead of 1. Bronze = 188,160.

GENERATOR CODE – RUN THIS FIRST, EXACTLY AS WRITTEN

This assumes MY_ID and VOL are already set, as on the second page of this document. Do not change the seed or any of the counts — your checks later depend on them.

```
def wr(d,p): d.write.mode('overwrite').option('header',True).csv(f'{VOL}/raw/{p}')
TM=lambda c: f"format_string('TM%02d',{c})"; PL=lambda c: f"format_string('PL%03d',{c}+1)"
E=lambda v: f"element_at(array({v}),pi)"; PHN="POWERPLAY','MIDDLE','DEATH'"
wr(spark.range(220).selectExpr(PL('id')+" player_id",TM('id%10+1')+" team",
    "format_string('Player %03d',id+1) player_name"),'players')
mt=spark.range(720).selectExpr("id m","format_string('M%04d',id+1) match_id","cast(id/60"
    " as int)+2014 season","cast(pmod(id,10)+1 as int) h","cast(pmod(id*7+3,10)+1 as int) a")
wr(mt.selectExpr("match_id","season",TM('h')+" team1",TM('a')+" team2"),'matches')
bs=mt.crossJoin(spark.range(1,3).withColumnRenamed('id','inning')).selectExpr("m",
    "match_id","season","inning","if(inning=1,h,a) bt","if(inning=1,a,h) bw")
BI="cast(pow(rand(96),2.0)*8 as int)*10+bw-1"; KI="cast(pow(rand(97),1.6)*22 as int)*10+bt-1"
SK="(0.70+pmod(ki*37,61)/100.0)"; TG="pi=3 and bi in (0,12,24,36,48,51)"
P=[f"if({TG},0.34,{E('0.40,0.48,0.34')})",f"if({TG},0.05,0.07)",
    f"if({TG},0.05,{E('0.11,0.07,0.15')})"*SK}],f"if({TG},0.03,{E('0.06,0.03,0.13')})"*SK)]
RV=("case when u<1-""-"".join(P)+" then 0 when u<1-""-"".join(P[1:])+"" then 1 when u<1-""
    +""-"".join(P[2:])+"" then 2 when u<1-""+P[3]+"" then 4 else 6 end")
lg=bs.crossJoin(spark.range(1,121).withColumnRenamed('id','seq')).selectExpr("match_id",
    "season","inning","seq","cast((seq-1)/6+1 as int) ov","rand(98) u",f"{BI} bi",f"{KI} ki",
    "cast(if(seq<=36,1,if(seq<=90,2,3)) as int) pi","if(rand(99)<0.035,1,0) is_wicket",
    "pmod(((m*2+inning-1)*120+seq-1)*7919,172800) kk")
lf=lg.selectExpr("match_id","season","inning","seq","kk",E(PHN)+" phase",
    "if(kk>=1500 and kk<2100,21,ov) over_no","cast(pmod(seq-1,6)+1 as int) ball_no",
    PL('ki')+" batter_id","'legal' extra_type","0 extra_runs","is_wicket",
    f"if(kk>=2100 and kk<2900,'PL999',{PL('bi')}) bowler_id",
    f"if(kk<1500,'NA',cast({RV} as string)) batter_runs",f"{RV} total_runs")
LS="(cast(pmod(hash(match_id,n),120)+1 as int))"
ex=bs.crossJoin(spark.range(1,10).withColumnRenamed('id','n')).selectExpr("match_id",
    "season","inning","120+n seq","cast(-1 as bigint) kk",PL(KI)+" batter_id",
    PL(BI)+" bowler_id","'0' batter_runs","1 extra_runs","0 is_wicket",
    "if(n<=6,'wide','noball') extra_type",f"cast(({LS}-1)/6+1 as int) over_no",
    f"cast(pmod(({LS}-1,6)+1 as int) ball_no",
    f"element_at(array({PHN}),if({LS}<=36,1,if({LS}<=90,2,3))) phase",
    f"if(pmod(((m*2+inning-1)*9+n-1)*7919,12960)<350,0,1) total_runs")
wr(lf.unionByName(lf.filter('kk>=2900 and kk<5300')).unionByName(ex).drop('kk'),'deliveries')
```

BRONZE – LAND IT, CHANGE NOTHING

Land all three files with every column read as a string: 1,500 batter_runs are the text 'NA' and 600 over_no read 21. Add _source_file, _ingested_at and _row_hash. Do not recompute total_runs and do not delete the over-21 rows here - that count is what you send back to the feed provider. extra_type holds exactly three values, 'legal', 'wide' and 'noball'; nothing else. Bronze deliveries = 188,160 rows, matches = 720, players = 220.

SILVER – THE DECISIONS YOU MUST MAKE, AND RECORD

Each decision below has a number attached to it. If your number does not match, stop and find out why before you go on — that gap is the finding, not a nuisance.

1. CAST: try_cast batter_runs, extra_runs, total_runs, over_no, ball_no and is_wicket to INT, never cast - under ANSI mode cast throws on the first 'NA'. Exactly 1,500 batter_runs become NULL. A delivery whose runs cannot be read counts for neither the batter nor the bowler, so move all 1,500 to silver_rejects with reason 'unparseable_runs'.
2. DEDUPE: key (match_id, inning, seq), not (match_id, inning, over_no, ball_no). A wide and the legal ball that follows it deliberately share an over and a ball number, so the second key deletes a real delivery from every over that had a wide. 188,160 -> 185,760.
3. REJECTS in this order: 800 rows carry bowler_id 'PL999', which is in no players row - 'unknown_bowler', 184,960. 600 rows have over_no = 21 - 'over_out_of_range', 184,360. Then the 1,500 unparseable, 182,860. Range-check the over between 1 and 20; do not hunt the literal 21, because the next feed will send 0 or 99. Then repair total_runs = batter_runs + extra_runs: exactly 350 rows disagree.
4. BALLS FACED and BALLS BOWLED, defined once, because cricket refuses to use one number for both. A wide is neither faced by the batter nor legal for the bowler; a no-ball IS faced but is NOT legal. So balls_faced = legal + no-balls = 169,900 + 4,320 = 174,220, while legal_balls_bowled = 169,900. runs_conceded is every run off that bowler, extras included; economy divides it by legal balls only.

GOLD – THE SHAPE THAT TRAVELS

One row per player per phase per season, built as a full grid so the zeros are visible: player_id, player_name, team, season, phase, balls_faced, runs_scored, fours, sixes, dismissals, strike_rate, boundary_pct, balls_bowled, runs_conceded, wickets, economy. Exactly $220 \times 3 \times 12 = 7,920$ rows. Only 80 players ever bowl, so balls_bowled is 0 on most rows; strike_rate and economy are NULL where the denominator is 0, never 0.

THE TRAP MOST PEOPLE FALL INTO

The minimum-balls rule written as WHERE instead of HAVING. Gold holds one row per season, so WHERE tests a season and not a career: a bowler with 45 death balls in each of twelve seasons has 540 career balls and still disappears, while a lead bowler keeps every row. Second: AVG(economy) across a player's season rows weighs a 24-ball season like a 240-ball one - sum the runs, sum the balls, divide once, at the end. Third: economy over balls_faced instead of balls_bowled quietly adds the no-balls back in.

THE THREE QUESTIONS YOUR GOLD LAYER MUST ANSWER

1. Strike rate by phase (powerplay, middle, death) per batter
2. Bowlers by death-overs economy, minimum 100 legal balls
3. Which rankings change most when the minimum-balls rule changes?

IN SNOWFLAKE – THE HARD ONE, WORKED

Tables this query needs in Snowflake: **GOLD_PLAYER_PHASE_SEASON**

```
-- Q2: death economy at a career minimum of 100 legal balls. The minimum is a HAVING on
-- the aggregate, not a WHERE on the rows - WHERE runs before GROUP BY, at a moment when
-- the career total does not exist yet. Economy = runs conceded per six legal balls.
SELECT player_id, player_name, team,
       SUM(balls_bowled) AS legal_balls,
       SUM(runs_conceded) AS runs,
       SUM(wickets) AS wkts,
       ROUND(SUM(runs_conceded) / NULLIF(SUM(balls_bowled) / 6.0, 0), 2) AS economy
FROM GOLD_PLAYER_PHASE_SEASON
WHERE phase = 'DEATH' AND balls_bowled > 0
GROUP BY player_id, player_name, team
HAVING SUM(balls_bowled) >= 100
ORDER BY economy ASC
LIMIT 20;
```

YOU ARE DONE WHEN

- Bronze deliveries = 188,160 = 185,760 + 2,400 repeats, and 185,760 = 720 x 258. Silver = 188,160 - 2,400 - 800 unknown bowlers - 600 over-21 - 1,500 unparseable = 182,860.
- Silver splits into 12,960 extras (8,640 wides + 4,320 no-balls) and 169,900 legal. Total balls_faced = 174,220 and legal balls bowled = 169,900; equal totals mean you counted no-balls as legal.
- Gold = 7,920 rows. Strike rate over all batters is near 135 in POWERPLAY, 108 in MIDDLE and 176 in DEATH. A flat profile means phase was read from the wrong column.
- The six best career death economies are PL001, PL013, PL025, PL037, PL049 and PL052, every one under 6.0, and the seventh name is above 9.5. Those six were planted; if they are not on top, the join lost the extras.
- Now swap HAVING SUM(balls_bowled) >= 100 for WHERE balls_bowled >= 100. Exactly ten players survive - PL001 to PL010 - against 80 under HAVING. Seventy names vanish because no single season reached 100 balls.

IF YOU FINISH EARLY

Rebuild the death list at minimums of 50, 100 and 200 career balls side by side, name every bowler who swings more than twenty places between them, and say which cut-off you would defend to a selection committee.

THE SITUATION

A training company must decide what to teach next year. 'Snowflake is in 31% of postings, up from 18% eighteen months ago' is a number a curriculum committee can act on. The skills sit in one free-text column typed by five hundred recruiters: three separators, no agreed capitalisation, some rows padded with spaces. One mistake splits a skill into four strings and drops it out of the top ten.

WHERE THE DATA COMES FROM

Kaggle job-posting scrapes (LinkedIn Job Postings and its mirrors). Column names differ between mirrors - read the header first. In some versions skills are not delimited text but a separate file keyed by job id and normalised to codes, which removes the exploding step this project is built around. No scrape has an even month spread either, so any growth you measure on real data can be a scrape-timing artefact rather than a hiring trend. The generator below is even by month, so the trend is real and checkable.

GENERATE IT – THE EXACT SPECIFICATION

Seed 26. companies.csv: 400 rows CMP000-CMP399, 8 sectors of 50, 10 cities. postings.csv: 54,000 = 18 months x 3,000 from 2024-01; m is the month 0-17 and k the row 0-2999 inside it; posted_date = the 1st + (id mod 28) days. Every posting carries 2-4 filler skills from a 16-word list (Python, SQL, Spark, Airflow, Kafka, Hive, AWS, Azure, GCP, Docker, Kubernetes, Terraform, Scala, Java, PowerBI, Tableau) joined with ';'. Four tracked skills are placed by k alone and never by the filler, so their counts are exact: Snowflake $k < 520 + 22m$ (12,726 rows, 18.1% -> 31.0% of a month); Databricks $1000 \leq k < 1400 + 18m$ (9,954, 13.9% -> 24.5%); Hadoop $1800 \leq k < 2660 - 38m$ (9,666, 29.9% -> 7.4%); dbt on even-k Snowflake rows (6,363, exactly half of them). Defects, none twice, on the skill-free band $k \geq 2700$ or on the scrambled key $ak = \text{pmod}((m * 2700 + k) * 7919, 48600)$: 900 empty skills_raw, 450 'Not specified', 540 company_id 'CMP999', 270 posted_date '0000-00-00', 1,350 duplicated rows (file 55,350), 1,000 joined with ',', 2,200 upper-cased, 1,700 joined with ' ; ' and wrapped in a leading and trailing space.

GENERATOR CODE – RUN THIS FIRST, EXACTLY AS WRITTEN

This assumes MY_ID and VOL are already set, as on the second page of this document. Do not change the seed or any of the counts — your checks later depend on them.

```
FL = ("Python','SQL','Spark','Airflow','Kafka','Hive','AWS','Azure','GCP','Docker',"
      "'Kubernetes','Terraform','Scala','Java','PowerBI','Tableau'")
SEC = "'BFSI','IT Services','Product','Retail','Health','Telecom','Energy','Logistics'"
CTY = ("Bengaluru','Hyderabad','Pune','Chennai','Mumbai','Delhi NCR','Noida',"
      "'Kolkata','Ahmedabad','Kochi'")
P = lambda x: f"element_at(array({FL}), cast(pmod({x}, 16) as int) + 1)"
def wr(d, p): # both whitespace flags off, or the padded rows lose their spaces
    (d.write.mode('overwrite').option('header', True)
     .option('ignoreLeadingWhiteSpace', False)
     .option('ignoreTrailingWhiteSpace', False).csv(f'{VOL}/raw/{p}'))
wr(spark.range(400).selectExpr("format_string('CMP%03d', id) company_id",
    "format_string('Company %03d', id + 1) company_name",
    f"element_at(array({SEC}), cast(id div 50 as int) + 1) sector",
    f"element_at(array({CTY}), cast(pmod(id, 10) as int) + 1) city"), 'companies')
b = spark.range(54000).selectExpr("id", "cast(id div 3000 as int) m",
    "cast(pmod(id, 3000) as int) k")
b = b.selectExpr("id", "m", "k", "pmod((m*2700 + k)*7919, 48600) ak",
    f"concat(array_distinct(array({P('id')}), {P('id div 16')}), {P('id div 256')}),"
    f" {P('id div 4096')}))", if(k < 520 + 22*m, array('Snowflake'), array()),"
    " if(k >= 1000 and k < 1400 + 18*m, array('Databricks'), array()),"
    " if(k >= 1800 and k < 2660 - 38*m, array('Hadoop'), array()),"
    " if(k < 520 + 22*m and pmod(k, 2) = 0, array('dbt'), array())) a")
b = b.selectExpr("k", "format_string('JP%06d', id) posting_id",
    "if(k >= 2775 and k < 2805, 'CMP999',"
    " format_string('CMP%03d', pmod(id, 400))) company_id",
    "if(k >= 2805 and k < 2820, '0000-00-00', date_format(date_add(add_months("
    "date'2024-01-01', m), pmod(id, 28)), 'yyyy-MM-dd')) posted_date",
    "case when k >= 2700 and k < 2750 then '"
    " when k >= 2750 and k < 2775 then 'Not specified'"
    " when k < 2700 and ak < 1000 then array_join(a, ',')"
    " when k < 2700 and ak < 3200 then upper(array_join(a, ';'))"
    " when k < 2700 and ak < 4900 then concat(' ', array_join(a, ' ; '), ' ')"
    " else array_join(a, ';') end skills_raw")
wr(b.unionByName(b.filter('k >= 2820 and k < 2895')).drop('k'), 'postings')
```

BRONZE – LAND IT, CHANGE NOTHING

Land both files with every column read as a string, and keep `skills_raw` exactly as written - no split, no trim, no lower. Bronze is the evidence that 1,000 recruiters used a comma and 2,200 shouted in capitals; strip that here and you can never report it. Add `_source_file`, `_ingested_at` and `_row_hash`. Bronze postings = exactly 55,350 rows and companies = 400. One posting is one row; the explode belongs in Silver.

SILVER – THE DECISIONS YOU MUST MAKE, AND RECORD

Each decision below has a number attached to it. If your number does not match, stop and find out why before you go on — that gap is the finding, not a nuisance.

1. DEDUPE BEFORE YOU EXPLODE, on `posting_id`: 55,350 rows fall to exactly 54,000, so 1,350 go. Print both. Explode first and you are deduplicating 235,000-odd skill rows against each other, which quietly deletes genuine repeats - the order of these two steps is itself the decision.
2. SPLIT ON A CHARACTER CLASS that accepts `';` or `'`, not a plain split on `'`. Exactly 1,000 postings used a comma; prove you handled them by counting split values that still contain a comma - it must be 0. Split on `'` alone and each of those 1,000 rows yields one junk string instead of four skills.
3. NORMALISE ONCE, trim then lower, on every exploded value. Distinct raw strings are exactly 57 and fall to exactly 20. Print both. 57, not 60, because SQL, AWS and GCP upper-case to themselves: 20 clean + 20 space-wrapped + 20 shouted, minus those 3 collisions. That is the argument for cleaning in Silver rather than in every query.
4. REJECT WITH REASONS: 900 empty `skills_raw` plus 450 'Not specified' = 1,350 rows, reason 'no_skills_listed', or 'not specified' walks into your top ten as a skill; 540 rows carry `company_id` 'CMP999', which matches no company - 'unknown_company'; 270 carry `posted_date` '0000-00-00', where `try_cast` to DATE gives NULL - 'unparseable_date'. Never cast; under ANSI mode cast throws.

GOLD – THE SHAPE THAT TRAVELS

Skill-month: `skill`, `month`, `postings_with_skill`, `postings_that_month`, `skill_share`, `rank_in_month` - exactly $20 \times 18 = 360$ rows, and `postings_that_month` is 2,880 in every one of the 18 months. Ordered pair-month: `skill_a`, `skill_b`, `month`, `pair_postings`, `postings_a`, `confidence_a_to_b`, `lift` - $20 \times 19 \times 18 = 6,840$ possible rows, of which exactly 6,660 have `pair_postings > 0`.

THE TRAP MOST PEOPLE FALL INTO

Splitting on `'` and stopping. It works on 53,000 postings, so the code looks right, while the 1,000 comma rows each become one invented skill such as 'Python,SQL,Spark,Snowflake' - seen once, never in the top ten, and roughly 4,400 real mentions vanish. Then capitalisation and padding: 'Snowflake', 'SNOWFLAKE' and ' Snowflake ' split those 12,726 mentions three ways, so Snowflake ranks below Docker and the committee is told the wrong thing.

THE THREE QUESTIONS YOUR GOLD LAYER MUST ANSWER

1. Top 10 skills by posting count, per month
2. Which skills are growing fastest quarter on quarter
3. Which skills appear together - given one skill, what else is on the same posting?

IN SNOWFLAKE – THE HARD ONE, WORKED

Tables this query needs in Snowflake: **SILVER_POSTING_SKILL**

```
-- Q3: companion skills, rolled up over all months. Confidence is directional, so there
-- is no a < b filter here - you want both directions of every pair.
WITH s AS (SELECT DISTINCT posting_id, skill FROM SILVER_POSTING_SKILL),
     n AS (SELECT skill, COUNT(*) AS c FROM s GROUP BY 1),
     t AS (SELECT COUNT(DISTINCT posting_id) AS total FROM s)
SELECT a.skill AS skill_a, b.skill AS skill_b, COUNT(*) AS pair_postings,
       ROUND(COUNT(*) / NULLIF(MAX(na.c), 0), 3) AS confidence_a_to_b,
       ROUND(COUNT(*) * MAX(t.total) / NULLIF(MAX(na.c) * MAX(nb.c), 0), 2) AS lift
FROM s a JOIN s b ON a.posting_id = b.posting_id AND a.skill <> b.skill
JOIN n na ON na.skill = a.skill JOIN n nb ON nb.skill = b.skill CROSS JOIN t
GROUP BY 1, 2
-- ORDER BY 3 inside OVER() is the literal 3, not the third column. Repeat the aggregate.
QUALIFY ROW_NUMBER() OVER (PARTITION BY a.skill ORDER BY COUNT(*) DESC) <= 3;
```

YOU ARE DONE WHEN

- ☐ Bronze 55,350. Silver = 55,350 - 1,350 duplicates - 1,350 no-skills - 540 unknown company - 270 bad date = 51,840, which is 18 months x 2,880. Print all five numbers; the subtraction must close.
- ☐ Bronze rows whose skills_raw contains 'nowflake' in any case = exactly 12,726. Distinct raw skill strings before normalising = exactly 57, after = exactly 20, none holding a comma, a leading space or a capital.
- ☐ Gold skill-month = 360 rows and pair-month = 6,660 rows with pair_postings > 0. The 180 empty pairs are the five skill pairs this feed never puts on one posting, not a broken join.
- ☐ Snowflake's monthly share rises 18.1% to 31.0%, Databricks 13.9% to 24.5%, Hadoop falls 29.9% to 7.4%. Print all 18 monthly shares for each.
- ☐ dbt is Snowflake's top companion at confidence exactly 0.500 and no other skill reaches 0.30. If dbt is not first, your explode lost rows.

IF YOU FINISH EARLY

Aggregate share by quarter, use LAG for the change in percentage points, and flag the first quarter in which a skill enters the top ten - the quarter the committee should have changed the syllabus.

THE SITUATION

Something broke last week: the dashboard went red at 11am, an engineer restarted a service, it went green again. Nobody can say when it actually started, because the only record is a two-million-line text file with no columns. The hard part of this project is at the front - turning lines into columns, and knowing exactly how many lines you failed to turn into columns.

WHERE THE DATA COMES FROM

NASA-HTTP access logs: two gzipped files, July and August 1995, about two million requests - verify that count, do not quote it. There is no header row, so read a few lines by eye first; the layout is close to Apache common log format but not guaranteed to match the textbook, and missing sizes are '-', not 0. Almost every response is 200, 304 or 404 with very few 5xx, so that file cannot answer question 1 or 2 at all. The generator below writes the same shape of text with an incident in it.

GENERATE IT – THE EXACT SPECIFICATION

Seed 27. One text file of Apache common-log lines, all times UTC, each ending in a 10-character request_id. Body 2,160,000 = 12 endpoints x 30 days x 24 hours x 250 requests, 2025-03-01 to 2025-03-30; e is the endpoint 0-11, h the hour 0-719, j the position 0-249 inside that endpoint-hour. /api/order/, /api/product/ and /api/user/ each take one of 9,000 five-digit ids, so 27,009 distinct raw paths in all. 5,000 client IPs: j<75 draws one of 50 crawler IPs, so 50 addresses take exactly 30% of hits. The 5xx count in an endpoint-hour is $1 + ((h+e) \bmod 3)$, which makes any 24-hour baseline exactly 48 errors in 6,000 requests, 0.8%; /api/checkout jumps to 30 an hour (12%) for the 18 hours from 2025-03-14 09:00, /api/search to 8 an hour (3.2%) from 2025-03-20 16:00 to month end. Defects, none twice, on dk = $\text{pmod}(\text{id} * 7919, 648000)$ over the crawler rows only, so no 5xx line is ever damaged: 4,000 truncated after the timestamp, 1,800 with the request unquoted, 300 missing the closing quote, 2,500 bytes '-', 1,200 timestamp '[--/--/----:--:--:-- +0000]', 6,000 duplicated lines; plus 900 blank lines. File = 2,166,900 lines.

GENERATOR CODE – RUN THIS FIRST, EXACTLY AS WRITTEN

This assumes MY_ID and VOL are already set, as on the second page of this document. Do not change the seed or any of the counts — your checks later depend on them.

```
EP = ("'/api/checkout','/api/search','/api/cart','/api/login','/api/pay','/api/ship',"
      "'/api/track','/health','/static/app.js','/api/order','/api/product','/api/user/'")
b = spark.range(2160000).selectExpr("id", "cast(pmod(id, 12) as int) e",
  "cast(pmod(id div 12, 720) as int) h", "cast(id div 8640 as int) j")
b = b.selectExpr("...", f"element_at(array({EP}), e + 1) ep",
  "cast(if(id < 648000, pmod(id*7919, 648000), 999999) as int) dk",
  "timestamp'2025-03-01 00:00:00' + make_dt_interval(0, h) ts")
b = b.selectExpr("...",
  "if(ep like '%/', concat(ep, cast(10000 + pmod(j*720 + h, 9000) as string)), ep) path",
  "if(e = 0 and h >= 321 and h <= 338, 30,"
  " if(e = 1 and h >= 472, 8, 1 + pmod(h + e, 3))) nerr")
b = b.selectExpr("id", "dk", "path", "cast(if(j >= 250 - nerr, 503, 200) as string) st",
  "if(j < 75, format_string('10.9.0.%d', pmod(h*12 + e, 50) + 1),"
  " format_string('10.20.%d.%d', pmod(id, 4950) div 225,"
  " pmod(pmod(id, 4950), 225) + 1)) ip",
  "concat('[', if(dk >= 8600 and dk < 9800, '--/---/-----:--:--',"
  " date_format(ts, 'dd/MMM/yyyy:HH:mm:ss')), ' +0000]'") tsb")
LN = ("if(dk < 4000, concat_ws(' ', ip, '- -', tsb), concat_ws(' ', ip, '- -', tsb),"
  " case when dk >= 4000 and dk < 5800 then concat('GET ', path, ' HTTP/1.1')"
  " when dk >= 5800 and dk < 6100 then concat(char(34), 'GET ', path, ' HTTP/1.1')"
  " else concat(char(34), 'GET ', path, ' HTTP/1.1', char(34)) end, st,"
  " if(dk >= 6100 and dk < 8600, '-', cast(500 + pmod(id, 9500) as string)), "
  " format_string('R%09d', id)) value")
lg = b.selectExpr(LN, "dk")
(lg.select('value').union(lg.filter('dk >= 9800 and dk < 15800').select('value'))
  .union(spark.range(900).selectExpr("' ' value")))
.write.mode('overwrite').text(f'{VOL}/raw/access_logs')
```

BRONZE – LAND IT, CHANGE NOTHING

`spark.read.text` gives you one column, value, holding the raw line - that is Bronze. Add `_source_file`, `_ingested_at` and `_line_length`. Do not parse here, and do not drop the blank or truncated lines. Bronze = exactly 2,166,900 rows, which must equal the file's line count. A pipeline that silently discards the lines it cannot parse is exactly how an outage goes unnoticed for a week.

SILVER – THE DECISIONS YOU MUST MAKE, AND RECORD

Each decision below has a number attached to it. If your number does not match, stop and find out why before you go on — that gap is the finding, not a nuisance.

1. PARSE with one anchored regex capturing ip, timestamp, method, path, protocol, status, bytes and request_id. Exactly 7,000 lines yield no status - 4,000 truncated, 1,800 whose request is not in quotes, 300 with an unbalanced quote, 900 blank - and every one of them goes to `bronze_unparsed` with its raw line. Parse rate $2,159,900 / 2,166,900 = 99.68\%$; publish that number.
2. DEDUPE ON request_id, which is why the generator writes one: 2,159,900 parsed rows fall to exactly 2,153,900, so 6,000 go. Print both. Do not dedupe on the raw line - two real requests from one IP in the same second to the same path are identical text, and both of them happened.
3. NULLS ARE ANSWERS. Exactly 2,500 lines carry bytes '-'; try_cast to BIGINT gives NULL, and a body-less response is not a response of size zero, so never coalesce it to 0. Exactly 1,200 timestamps are unparseable; try_cast with a format, reject those with reason 'bad_timestamp', and range-check 2025-03-01 to 2025-03-31 rather than hunting for the literal dashes.
4. COLLAPSE PATH TO ENDPOINT - the decision this whole project turns on. Replace any all-digit path segment with '{id}', so `/api/order/48213` becomes `/api/order/{id}` and 27,009 raw paths become exactly 12 endpoints. Store both columns. Define error once, here: `status >= 500` is the numerator; a 404 is a client asking for something that is not there, not your outage.

GOLD – THE SHAPE THAT TRAVELS

`gold_endpoint_hour`, one row per endpoint per hour: `endpoint`, `hour_ts`, `requests`, `errors_5xx`, `errors_4xx`, `error_rate_5xx`, `distinct_ips`, `bytes_sent`, `bytes_missing` - exactly $12 \times 30 \times 24 = 8,640$ rows, zero-error hours kept, because they are the evidence the incident had stopped. `gold_client_ip`, one row per address: `client_ip`, `requests`, `pct_of_total`, `endpoints_hit`, `first_seen`, `last_seen`, `is_top_1pct` - exactly 5,000 rows. Question 3 reads this second table, not the first.

THE TRAP MOST PEOPLE FALL INTO

Parsing by splitting on spaces. The first four fields line up, then the quoted request holds two spaces of its own and everything after it shifts one place: status comes out as 'HTTP/1.1' on every line, and the 5xx rate is zero everywhere, so the outage is invisible. Use an anchored regex. Second: forgetting to collapse numeric path segments, which makes `/api/order/48213` an endpoint of one request at a 100% error rate, and buries the twelve real endpoints under 27,000 fakes.

THE THREE QUESTIONS YOUR GOLD LAYER MUST ANSWER

1. 5xx error rate per endpoint per hour
2. The first hour each endpoint's error rate reached twice its own recent baseline
3. Which client IPs are the top 1% of traffic, and how much of the total is that?

IN SNOWFLAKE – THE HARD ONE, WORKED

Tables this query needs in Snowflake: **GOLD_ENDPOINT_HOUR**

```
-- Q2: the frame ends at 1 PRECEDING so the bad hour cannot inflate its own baseline.
-- Baseline here is 48 errors in 6,000 requests, 0.8%, for every endpoint.
WITH h AS (SELECT endpoint, hour_ts, requests, errors_5xx,
  errors_5xx / NULLIF(requests, 0) AS rate,
  SUM(errors_5xx) OVER (PARTITION BY endpoint ORDER BY hour_ts
    ROWS BETWEEN 24 PRECEDING AND 1 PRECEDING) AS e24,
  SUM(requests) OVER (PARTITION BY endpoint ORDER BY hour_ts
    ROWS BETWEEN 24 PRECEDING AND 1 PRECEDING) AS r24
  FROM GOLD_ENDPOINT_HOUR)
SELECT * FROM h
WHERE rate >= 2 * e24 / NULLIF(r24, 0) AND errors_5xx >= 5
QUALIFY ROW_NUMBER() OVER (PARTITION BY endpoint ORDER BY hour_ts) = 1
ORDER BY hour_ts;
```

YOU ARE DONE WHEN

- ☐ Bronze 2,166,900; parsed 2,159,900 after 7,000 unparsed; 2,153,900 after 6,000 duplicates; Silver 2,152,700 after 1,200 bad timestamps. Print every step and close the subtraction.
- ☐ Distinct endpoints in Silver is exactly 12 and distinct raw paths exactly 27,009. Endpoints in the thousands means the numeric segments were never collapsed.
- ☐ gold_endpoint_hour has exactly 8,640 rows and SUM(requests) over it equals 2,152,700, the Silver row count. Assert $12 \times 30 \times 24 = 8,640$ in code, not by eye.
- ☐ The query above returns exactly two rows: /api/checkout at 2025-03-14 09:00 and /api/search at 2025-03-20 16:00. A third row means your baseline window is wrong.
- ☐ gold_client_ip has exactly 5,000 rows; the top 50 by requests are the 50 crawler addresses and hold 29.8% of Silver requests. 50 is 1% of 5,000.

IF YOU FINISH EARLY

Gaps-and-islands - the difference between a row number and a dense rank is the run key - merges consecutive flagged hours into incidents with a start, end, duration and peak rate, so checkout reads as one 18-hour incident instead of 18 separate alerts.

THE SITUATION

The dashboard says average latency is 120 ms; the support inbox says the app is unusable. Both are true: the mean is dragged down by a million health checks, and the angry customers are the slowest 1%. An average cannot see them; a percentile can. The business also wants an error budget - a fixed monthly allowance of slow requests, spent down day by day until there is nothing left.

WHERE THE DATA COMES FROM

There is no reliable public dataset of per-request API latency carrying service, endpoint and timestamp. Real traces are internal telemetry; anything published as one is either synthetic already or has been aggregated to per-minute averages, which destroys exactly the percentiles this project is about. Do not go looking - the generator plants three different latency shapes, and the point of the exercise is that you can check your answer against them.

GENERATE IT – THE EXACT SPECIFICATION

Seed 28. services.csv: 15 rows SVC00-SVC14, 5 teams of 3, tier 'critical' below SVC05, slo_ms 500, slo_target 0.99. requests.csv: 1,800,000 = 15 services x 30 days x 4,000, 2025-04-01 to 2025-04-30; sv = id div 120000, dy is the day 0-29, q the position 0-3999 inside the day, and q*21.6 seconds is the time of day. latency_ms = round($45 * \exp(0.095 * sv) * \exp(\text{randn} * \text{sigma}) * \text{peak}$) with sigma 0.40, so the median runs 45 ms at SVC00 to 171 ms at SVC14 and p99 sits near 425 ms - inside the 500 ms SLO for every service. Three plants: SVC03 gets sigma 1.1 on dy 10-13, taking p99 to roughly 730 ms while its median does not move; SVC09 gets peak 3.5 for q 3167-3499, which is 19:00 to 21:00; and SVC09's first 80 peak rows each day write the word 'timeout' instead of a number, 2,400 in all. status_code is 500 on 1%. Defects, none twice, on dk = pmod(nk*7919, 1680000) over the non-SVC09 rows: 1,500 latency_ms '-1', 900 latency_ms '0', 700 service 'SVC99', 1,100 request_ts written day-first as 14-04-2025 08:15:00, and 3,600 duplicated rows, so the file is 1,803,600.

GENERATOR CODE – RUN THIS FIRST, EXACTLY AS WRITTEN

This assumes MY_ID and VOL are already set, as on the second page of this document. Do not change the seed or any of the counts — your checks later depend on them.

```
TEAM = "'payments','identity','catalog','fulfilment','platform'"
(spark.range(15).selectExpr("format_string('SVC%02d', id) service_id",
    f"element_at(array({TEAM}), cast(id div 3 as int) + 1) team",
    "if(id < 5, 'critical', 'standard') tier", "500 slo_ms", "0.99 slo_target"))
.write.mode('overwrite').option('header', True).csv(f'{VOL}/raw/services')
b = spark.range(1800000).selectExpr("id", "cast(id div 120000 as int) sv",
    "cast(pmod(id, 4000) as int) q", "cast(pmod(id div 4000, 30) as int) dy")
b = b.selectExpr("?", "if(sv = 9, 9999999, "
    " cast(pmod(if(id < 1080000, id, id - 120000)*7919, 1680000) as int)) dk",
    "timestamp'2025-04-01 00:00:00' + make_dt_interval(dy, 0, 0, q*21.6) ts",
    "if(sv = 3 and dy >= 10 and dy <= 13, 1.1, 0.40) sg",
    "if(sv = 9 and q >= 3167 and q < 3500, 3.5, 1.0) pk")
b = b.selectExpr("id", "sv", "q", "dk", "ts",
    "greatest(cast(round(45 * exp(0.095*sv) * exp(randn(28)*sg) * pk) as int), 1) lat")
b = b.selectExpr("dk", "format_string('RQ%08d', id) request_id",
    "if(dk >= 2400 and dk < 3100, 'SVC99', format_string('SVC%02d', sv)) service",
    "if(dk >= 3100 and dk < 4200, date_format(ts, 'dd-MM-yyyy HH:mm:ss'),"
    " date_format(ts, 'yyyy-MM-dd HH:mm:ss')) request_ts",
    "case when sv = 9 and q >= 3167 and q < 3247 then 'timeout'"
    " when dk < 1500 then '-1' when dk < 2400 then '0'"
    " else cast(lat as string) end latency_ms",
    "if(pmod(id, 100) = 7, 500, 200) status_code")
(b.unionByName(b.filter('dk >= 4200 and dk < 7800')).drop('dk'))
.write.mode('overwrite').option('header', True).csv(f'{VOL}/raw/requests')
```

BRONZE – LAND IT, CHANGE NOTHING

Land both files with every column read as a string: 2,400 latency values are the word 'timeout', and a typed read either throws under ANSI mode or nulls them without telling you. Add `_source_file`, `_ingested_at` and `_row_hash`. Bronze requests = exactly 1,803,600 rows and services = 15. Keep the -1 sentinels and the zeros exactly as they arrived - their count is a message you owe the instrumentation owner.

SILVER – THE DECISIONS YOU MUST MAKE, AND RECORD

Each decision below has a number attached to it. If your number does not match, stop and find out why before you go on — that gap is the finding, not a nuisance.

- 1. CAST AND DEDUPE.** `try_cast latency_ms to INT` and `request_ts to TIMESTAMP`, never cast - under ANSI mode cast throws on the first 'timeout'. Exactly 2,400 latencies and 1,100 timestamps come back NULL; reject the 1,100 as 'unparseable_timestamp'. Dedupe on `request_id`: 1,803,600 falls to exactly 1,800,000. Reject the 700 rows whose service is 'SVC99', an id in no services row.
- 2. A TIMEOUT IS NOT MISSING DATA**, it is the slowest possible request: those 2,400 rows hit the 30-second client ceiling. Set `latency_ms = 30000` and `measured = false` on exactly those 2,400, and keep them. They are all on SVC09; drop them and SVC09's p99 falls from 30,000 ms to about 540, which reads as a healthy service that quietly fails 80 requests a day.
- 3. A -1 IS MISSING AND A 0 IS IMPOSSIBLE.** Exactly 1,500 rows carry -1 and 900 carry 0, and neither is a very fast request. Reject all 2,400 with reasons 'latency_not_measured' and 'impossible_latency'. Left in, the 1,500 negatives sort to the bottom of every ORDER BY and drag p50 down while p99 does not move, so the median improves during an incident.
- 4. DEFINE SLOW ONCE:** `latency_ms > slo_ms` joined from the services master, strictly greater. The monthly error budget is $(1 - slo_target) \times \text{the month's planned requests}$: at $4,000 \times 30 = 120,000$ and a 0.99 target, exactly 1,200 slow requests per service. Store `slo_ms`, the budget and the `is_slow` flag in Silver so no two queries can disagree about it.

GOLD – THE SHAPE THAT TRAVELS

One row per service per day: `service`, `team`, `tier`, `day`, `requests`, `p50_ms`, `p95_ms`, `p99_ms`, `max_ms`, `slow_requests`, `slow_rate`, `breached` (`p99_ms > 500`), `budget_allowance`, `budget_used_cum`, `budget_remaining_pct`. Exactly $15 \times 30 = 450$ rows, read by a service owner over coffee every morning.

THE TRAP MOST PEOPLE FALL INTO

Rolling a monthly p99 up by averaging thirty daily p99s. Percentiles do not average; the answer can be out by a factor of two. A monthly p99 must be computed from the raw rows, and the same mistake kills any max or mean taken over hourly p99s. Second: leaving the 1,500 -1 rows in, after which p50 falls while p99 does not, so the median appears to improve in the middle of an incident.

THE THREE QUESTIONS YOUR GOLD LAYER MUST ANSWER

1. p50, p95 and p99 latency per service per day
2. The days on which p99 breached the 500 ms SLO, and for which services
3. How much of each service's monthly error budget is consumed, day by day?

IN SNOWFLAKE – THE HARD ONE, WORKED

Tables this query needs in Snowflake: **SILVER_REQUESTS**

```
-- Q1 and Q3: true percentiles from the raw rows, then the error budget as a running
-- sum against the fixed monthly allowance of 1,200 slow requests per service.
WITH d AS (SELECT service, DATE(request_ts) AS day, COUNT(*) AS requests,
  PERCENTILE_CONT(0.50) WITHIN GROUP (ORDER BY latency_ms) AS p50,
  PERCENTILE_CONT(0.95) WITHIN GROUP (ORDER BY latency_ms) AS p95,
  PERCENTILE_CONT(0.99) WITHIN GROUP (ORDER BY latency_ms) AS p99,
  COUNT_IF(latency_ms > 500) AS slow
FROM SILVER_REQUESTS GROUP BY 1, 2),
b AS (SELECT d.*, SUM(slow) OVER (PARTITION BY service ORDER BY day
  ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS used FROM d)
SELECT service, day, requests, p50, p95, p99, slow, p99 > 500 AS breached, used,
  ROUND(100 * (1 - used / 1200.0), 1) AS budget_left_pct
FROM b ORDER BY service, day;
```

YOU ARE DONE WHEN

- ☐ Bronze 1,803,600. Silver = 1,803,600 - 3,600 duplicates - 1,500 sentinels - 900 zeros - 700 SVC99 - 1,100 bad timestamps = 1,795,800, with the 2,400 timeouts still in it.
- ☐ Gold has exactly 450 rows, SUM(requests) over Gold equals 1,795,800, and SVC09's 30 rows each read exactly 4,000 requests because no defect was placed on that service.
- ☐ SVC03 breaches p99 500 ms on 2025-04-11 to 14 only, near 730 ms, while its median stays near 60 ms. SVC14 has the highest median, near 171 ms, and a p99 near 425 ms that never breaches.
- ☐ SVC09 breaches on all 30 days with a median near 110 ms - the shape an average hides. Its p99 is the 30,000 ms ceiling; recompute it without the timeout rows and it drops to about 540.
- ☐ Per service, budget_used on the last day equals SUM(slow_requests) over its 30 rows, and budget_left_pct = 100 x (1 - used/1200). SVC09 passes 1,200 around the ninth day and ends deeply negative; SVC14 ends near 70%. Negative is real, not a bug.

IF YOU FINISH EARLY

Add a burn rate: budget consumed in a rolling 6-hour window divided by what an even spend across the month would allow. Above 14.4 the whole month's budget is gone in about two days, which is the number that should page someone.

THE SITUATION

Finance gets one Snowflake bill; seven teams caused it. The bill is per warehouse per hour, the work is per query per person, so nobody can say whose it is. Every other project here analyses somebody else's data. This one analyses you: run a controlled workload on your own trial, then read your own account metadata and split the bill by it.

WHERE THE DATA COMES FROM

Your own trial. The INFORMATION_SCHEMA table functions are current within minutes and answer everything here, so they are the primary source - but they keep only 7 days, so snapshot.

ACCOUNT_USAGE.QUERY_HISTORY keeps a year and lags 45 minutes to about 3 hours: use it tomorrow as a cross-check. Reading it needs ACCOUNTADMIN, which a trial owner already has. The workload runs on SNOWFLAKE_SAMPLE_DATA.TPCH_SF1 - confirm it with SHOW DATABASES, and DESCRIBE a view before trusting a column name.

GENERATE IT – THE EXACT SPECIFICATION

No file: you generate the history by running a controlled workload. WH_XS (X-Small) and WH_SM (Small), AUTO_SUSPEND 60, INITIALLY_SUSPENDED TRUE, 60-second minimum charge on every resume. Eight TPCH_SF1 queries, one team each: Q1 finance LINEITEM aggregate; Q2 marketing ORDERS plus CUSTOMER plus NATION; Q3 ops two-table join over one year; Q4 risk a global ROW_NUMBER over six million rows, built to spill on X-Small; Q5 risk an ORDERS self-join; Q6 growth a single-key lookup; Q7 data_eng COUNT(*) on LINEITEM; Q8 ml ranked customer spend. Set USE_CACHED_RESULT FALSE first or a repeat costs 0 credits in 30 ms. Tag every query 'team=<team>|qid=<Qn>|size=<XSMALL|SMALL>'. Run the sheet five times on WH_XS, swap two tokens, run it five more on WH_SM: $8 \times 5 \times 2 = 80$ runs, two to six metered warehouse-hours, roughly one credit. Watch Q7 - Snowflake answers COUNT(*) from metadata without starting a warehouse, so its ten runs return in milliseconds with WAREHOUSE_SIZE NULL and cost nothing. Real runs, not cache hits.

GENERATOR CODE – RUN THIS FIRST, EXACTLY AS WRITTEN

This assumes MY_ID and VOL are already set, as on the second page of this document. Do not change the seed or any of the counts — your checks later depend on them.

```
-- Snowflake SQL, not PySpark: paste this into a Snowflake worksheet, not a
-- notebook. Pass 1: run the whole sheet 5 times exactly as written. Pass 2:
-- replace WH_XS with WH_SM and XSMALL with SMALL, run it 5 more times. 80 runs.
CREATE WAREHOUSE IF NOT EXISTS WH_XS WAREHOUSE_SIZE=XSMALL
  AUTO_SUSPEND=60 INITIALLY_SUSPENDED=TRUE;
CREATE WAREHOUSE IF NOT EXISTS WH_SM WAREHOUSE_SIZE=SMALL
  AUTO_SUSPEND=60 INITIALLY_SUSPENDED=TRUE;
USE WAREHOUSE WH_XS;
USE SCHEMA SNOWFLAKE_SAMPLE_DATA.TPCH_SF1;
ALTER SESSION SET USE_CACHED_RESULT=FALSE;
ALTER SESSION SET QUERY_TAG='team=finance|qid=Q1|size=XSMALL';
SELECT l_returnflag, l_linestatus, SUM(l_quantity) q FROM LINEITEM GROUP BY 1,2;
ALTER SESSION SET QUERY_TAG='team=marketing|qid=Q2|size=XSMALL';
SELECT n_name, SUM(o_totalprice) t FROM ORDERS o JOIN CUSTOMER c
  ON o.o_custkey=c.c_custkey JOIN NATION n ON c.c_nationkey=n.n_nationkey GROUP BY 1;
ALTER SESSION SET QUERY_TAG='team=ops|qid=Q3|size=XSMALL';
SELECT o_orderpriority, COUNT(*) c FROM ORDERS o JOIN LINEITEM l
  ON o.o_orderkey=l.l_orderkey WHERE YEAR(o_orderdate)=1996 GROUP BY 1;
ALTER SESSION SET QUERY_TAG='team=risk|qid=Q4|size=XSMALL';
SELECT MAX(rn) FROM (SELECT ROW_NUMBER() OVER (ORDER BY l_extendedprice) rn
  FROM LINEITEM);
ALTER SESSION SET QUERY_TAG='team=risk|qid=Q5|size=XSMALL';
SELECT COUNT(*) FROM ORDERS a JOIN ORDERS b ON a.o_custkey=b.o_custkey;
ALTER SESSION SET QUERY_TAG='team=growth|qid=Q6|size=XSMALL';
SELECT o_orderdate, o_totalprice FROM ORDERS WHERE o_orderkey=1;
ALTER SESSION SET QUERY_TAG='team=data_eng|qid=Q7|size=XSMALL';
SELECT COUNT(*) FROM LINEITEM;
ALTER SESSION SET QUERY_TAG='team=ml|qid=Q8|size=XSMALL';
SELECT c_custkey, SUM(o_totalprice) s, RANK() OVER (ORDER BY SUM(o_totalprice) DESC) r
  FROM CUSTOMER JOIN ORDERS ON c_custkey=o_custkey GROUP BY 1 ORDER BY 2 DESC LIMIT 20;
ALTER SESSION SET QUERY_TAG='';
-- After BOTH passes, snapshot Bronze, then:
-- ALTER WAREHOUSE WH_XS SUSPEND; ALTER WAREHOUSE WH_SM SUSPEND;
```

BRONZE – LAND IT, CHANGE NOTHING

Bronze is a snapshot, not optional: the INFORMATION_SCHEMA functions keep 7 days and ACCOUNT_USAGE has not caught up yet. The moment the last run finishes: `CREATE TABLE BRONZE_QUERY_HISTORY AS SELECT *, CURRENT_TIMESTAMP() AS _snapshot_at FROM TABLE(INFORMATION_SCHEMA.QUERY_HISTORY(RESET_LIMIT => 10000))` - that argument matters, the default is 100 rows and would swallow most of your 80. Then BRONZE_WAREHOUSE_METERING the same way from WAREHOUSE_METERING_HISTORY(`DATE_RANGE_START => DATEADD('day', -1, CURRENT_DATE())`). Transform nothing.

SILVER – THE DECISIONS YOU MUST MAKE, AND RECORD

Each decision below has a number attached to it. If your number does not match, stop and find out why before you go on — that gap is the finding, not a nuisance.

1. KEEP ONLY BILLABLE WORK: `EXECUTION_STATUS = 'SUCCESS'`, and drop `QUERY_TYPE` 'USE', 'SHOW', 'DESCRIBE', 'ALTER_SESSION' and 'CREATE_WAREHOUSE' - none of them touch a warehouse. All 80 tagged runs survive as 'SELECT'. A cached run also survives, it just returns in about 20 ms with `BYTES_SCANNED 0`; if more than the ten Q7 runs look like that, the session reset `USE_CACHED_RESULT` and that pass has to be run again.
2. PARSE THE TAG, NOT THE SQL TEXT: split `QUERY_TAG` on '|' then on '=' for team, qid and size. Exactly 80 rows yield all three; everything else you ran in the window is team 'unattributed', and print that count - quietly filing your own exploration under a team is how the report loses its argument. Take the warehouse from the tag's size too: the ten Q7 rows can come back with `WAREHOUSE_NAME` and `WAREHOUSE_SIZE` both NULL.
3. ATTRIBUTE CREDITS AT THE WAREHOUSE RATE, NOT AS A SHARE OF THE HOUR: credits for one run = $\text{EXECUTION_TIME} / 1000 / 3600 \times \text{rate}$, where X-Small is 1 credit an hour, Small 2, doubling per size up. Dividing an hour's metered credits proportionally among its queries is the tempting version and it is wrong: it hands out every credit, so idle comes out zero and the biggest item on the bill vanishes.
4. IDLE IS REAL MONEY: for each warehouse-hour, idle = metered `CREDITS_USED` minus the sum of that hour's query credits, floored at zero because concurrent queries can out-sum the wall clock. It is the 60-second resume minimum plus the auto-suspend runway, usually over half the bill. It gets its own bucket - never spread it across teams - and is reported as a percentage of the total.

GOLD – THE SHAPE THAT TRAVELS

gold_team_day, one row per warehouse per team per date: `usage_date`, `warehouse_name`, `warehouse_size`, `team`, `queries`, `total_execution_ms`, `credits_attributed`, `bytes_scanned`, `spill_local_bytes`, `queued_ms`. Seven teams plus 'unattributed' plus 'idle', two warehouses, one date: exactly 18 rows. gold_query_cost, one row per run - what questions 2 and 3 read: `query_id`, `qid`, `team`, `warehouse_name`, `size_tag`, `start_time`, `elapsed_ms`, `execution_ms`, `credits_attributed`, `bytes_scanned`, `spill_local_bytes`, `queued_ms`.

THE TRAP MOST PEOPLE FALL INTO

Splitting the bill by query count. Growth ran 10 queries and risk ran 10, so each gets half - except one Q6 run took 40 ms and one Q4 took four minutes. Weight by `EXECUTION_TIME`, converted to credits at the warehouse's own rate. Second: deleting the ten Q7 rows because `WAREHOUSE_SIZE` is NULL and calling them cache hits - they are real runs that cost nothing. Third: scaling teams up until they sum to the metered credits, which buries the finding that most of the bill bought idle.

THE THREE QUESTIONS YOUR GOLD LAYER MUST ANSWER

1. Credits by warehouse by day, and how much of that was idle
2. The 10 most expensive queries, and which team ran them
3. Which queries would a bigger warehouse NOT have helped?

IN SNOWFLAKE – THE HARD ONE, WORKED

Tables this query needs in Snowflake: **GOLD_QUERY_COST**

```
-- Q3: the same eight queries at two sizes. A bigger warehouse only helps a
-- query that spilled or queued. Size comes from the tag, never from the
-- column: Q7 never used a warehouse at all, so its size column is NULL.
WITH q AS (
  SELECT qid, size_tag,
         MEDIAN(elapsed_ms)          AS ms,
         MAX(spill_local_bytes)      AS spill,
         MAX(queued_ms)              AS queued
  FROM GOLD_QUERY_COST
  WHERE team <> 'unattributed'
  GROUP BY 1, 2)
SELECT qid,
       MAX(IFF(size_tag = 'XSMALL', ms, NULL)) AS xs_ms,
       MAX(IFF(size_tag = 'SMALL', ms, NULL)) AS sm_ms,
       ROUND(MAX(IFF(size_tag = 'XSMALL', ms, NULL))
             / NULLIF(MAX(IFF(size_tag = 'SMALL', ms, NULL)), 0), 2) AS speedup,
       MAX(spill) AS spill_bytes, MAX(queued) AS queued_ms
FROM q
GROUP BY 1
ORDER BY 4 DESC;
```

YOU ARE DONE WHEN

- ☐ Grouping gold_query_cost by qid and size_tag gives exactly 16 groups of 5 runs, 80 in all. Fewer than 5 in a group means the worksheet reconnected and USE_CACHED_RESULT went back to TRUE - re-run that pass.
- ☐ Exactly 10 of the 80 are Q7. They return in milliseconds with WAREHOUSE_SIZE NULL and BYTES_SCANNED 0 because Snowflake answered from metadata without a warehouse. They cost 0 credits and still belong in the 80.
- ☐ For each warehouse, SUM(credits_attributed) over the seven teams plus 'unattributed' plus 'idle' equals SUM(CREDITS_USED) for that warehouse in the metering snapshot, to six decimals. gold_team_day is exactly 18 rows.
- ☐ Idle is above 40% of total credits. The whole exercise should land near one credit; several credits means a warehouse never suspended, so check AUTO_SUSPEND is 60.
- ☐ Q6 and Q7 show zero spill, zero queue and a speedup under 1.3. If neither Q4 nor Q5 spilled on X-Small, write that down: a query that never spilled is the answer to question 3, not a broken lab.

IF YOU FINISH EARLY

Recompute the bill at AUTO_SUSPEND 60, 300 and 600 seconds from the real gaps between your query start times, then say which setting you would pick and what it saves. A longer runway buys fewer resume minimums and more paid idle.

THE SITUATION

Every pipeline so far accepted whatever arrived: 'unknown' became a NULL nobody counted, an empty file produced no rows and no complaint, an order pointing at a customer who does not exist was dropped by a LEFT JOIN. This one builds the watcher - named rules on every load, a table recording what each caught, a freshness check, a schedule that runs the lot. The generator builds the whole history from nothing, so no earlier work is needed, and you know every defect count before you write a single rule.

WHERE THE DATA COMES FROM

No public dataset, and you do not need one: you inject the corruption yourself, so you know how many rows each rule must catch before you run it. The generator writes ten loaded order files and ten customer files as history, then six new order files - one clean control, then one file per defect. Nothing depends on tables you built earlier; if you still have your own medallion tables, point the same rules at them afterwards as a second run.

GENERATE IT – THE EXACT SPECIFICATION

Seed 30, every value derived from the row number, so counts are exact. Loaded history: cust_day_01 to cust_day_10, 300 customers each, 3,000 in all, ids C00000 to C02999, signup dates 2025-06-01 to 2025-06-10; and day_01 to day_10, 5,000 orders each, 50,000 in all, ORD000001 to ORD050000. Contract, in this order: order_id, customer_id, order_ts, quantity, unit_price, channel, status. Every customer_id in an order file exists in the master, quantity is 1 to 5 written as text, four channels, every 50th order CANCELLED. Then six new files: day_11, 5,000 clean; day_12, 5,000 with quantity 'unknown' on the first 40; day_13, 4,850 new ids plus 150 re-sent from the loaded history, 5,000 lines; day_14, 5,000 valid rows carrying the same seven column names in a different order; day_15, a header line and no rows; day_16, 5,000 of which 220 carry customer_id 'C99999' and a separate 90 are dated 2035-06-16. 25,000 new rows, 500 of them bad, and no row is bad twice.

GENERATOR CODE – RUN THIS FIRST, EXACTLY AS WRITTEN

This assumes MY_ID and VOL are already set, as on the second page of this document. Do not change the seed or any of the counts — your checks later depend on them.

```
C = ['order_id','customer_id','order_ts','quantity','unit_price','channel','status']
CTY = "'Chennai','Pune','Kochi','Indore','Jaipur','Bhopal','Nagpur','Surat'"; R = f'{VOL}/raw'
def wr(df, path, cols):          # header on, whitespace preserved on write
    (df.selectExpr(*cols).write.mode('overwrite').option('header', True)
     .option('ignoreLeadingWhiteSpace', False)
     .option('ignoreTrailingWhiteSpace', False).csv(path))
def day(d, n, first, q="cast(1 + pmod(id, 5) as string)", c=None, t=None):
    return spark.range(n).selectExpr(
        f"format_string('ORD%06d', id + {first}) order_id",
        c or "format_string('C%05d', cast(pmod(id*7919, 3000) as int)) customer_id",
        t or f"cast(date_add(date'2025-06-01', {d-1}) as string) order_ts",
        f"{q} quantity",
        "round(20 + pmod(id*37, 18000)/100.0, 2) unit_price",
        "concat('CH', cast(pmod(id, 4) as string)) channel",
        "if(pmod(id, 50) = 0, 'CANCELLED', 'PLACED') status")
for d in range(1, 12):          # ten loaded files plus the clean control file
    wr(day(d, 5000, (d-1)*5000 + 1), f'{R}/orders/day_{d:02d}', C)
for d in range(1, 11):          # 300 customers a file, 3,000 in all
    wr(spark.range(300).selectExpr(
        f"format_string('C%05d', cast(id + {(d-1)*300} as int)) customer_id",
        f"element_at(array({CTY}), cast(pmod(id + {(d-1)*300}, 8) as int) + 1) city",
        f"cast(date_add(date'2025-06-01', {d-1}) as string) signup_date",
        f'{R}/customers/cust_day_{d:02d}', ['customer_id','city','signup_date'])
    wr(day(12, 5000, 55001, q="if(id < 40, 'unknown', cast(1+pmod(id,5) as string))",
        f'{R}/orders/day_12', C)          # 40 quantities not numeric
    wr(day(13, 4850, 60001).union(day(1, 150, 1)), f'{R}/orders/day_13', C)
    wr(day(14, 5000, 64851), f'{R}/orders/day_14',          # same names, moved order
        ['order_id','order_ts','unit_price','quantity','status','channel','customer_id'])
    spark.createDataFrame([('','').join(C,)], 'value string') \
        .write.mode('overwrite').text(f'{R}/orders/day_15')    # header line, no rows
    wr(day(16, 5000, 69851, c="if(id < 220, 'C99999', format_string('C%05d',"
        " cast(pmod(id*7919, 3000) as int))) customer_id",
        t="if(id >= 220 and id < 310, '2035-06-16', '2025-06-16') order_ts",
        f'{R}/orders/day_16', C)
```

BRONZE – LAND IT, CHANGE NOTHING

Land every file as it arrives, all columns as strings, plus `_source_file`, `_ingested_at`, `_load_id` and `_row_hash`. Read each file twice: once with `header=True` and no schema, to record the column names exactly as they arrived, and once with the declared schema for the data. Then `bronze_load_manifest`, one row per file: `load_id`, `table_name`, `source_file`, `rows_in_file`, `column_list` as one comma-joined string, `ingested_at`. File-scope rules read the manifest, not the rows - `day_15` has no rows to look at.

SILVER – THE DECISIONS YOU MUST MAKE, AND RECORD

Each decision below has a number attached to it. If your number does not match, stop and find out why before you go on — that gap is the finding, not a nuisance.

- 1. READ BY DECLARED SCHEMA**, never `inferSchema` and never by position. `day_14` then loads correctly by name, and R6 still flags it, because the manifest's `column_list` does not match the contract string. Read positionally and `unit_price` lands in `quantity` on all 5,000 rows with no rule noticing, since every value is still a number. R6 must compare the ordered list: compare the two as sets and `day_14` sails through.
- 2. CAST AND DEDUPE**. `try_cast` quantity to INT, never cast, which throws here: exactly 40 rows go NULL, all from `day_12`, rejected as R3 'quantity_not_numeric'. Then R2 - anti-join or MERGE `order_id` against `silver_orders` as it already stands, so exactly 150 rows from `day_13` are rejected as 'duplicate_order_id'. Every reject row carries `load_id`, `order_id`, `rule_id` and the raw row it came from.
- 3. REFERENTIAL AND RANGE**. R4: exactly 220 rows carry `customer_id` 'C99999', which has no row in the customer master, rejected as 'unknown_customer'. R5: a separate 90 rows carry `order_ts` 2035-06-16, rejected as 'order_ts_out_of_range'. Both come from `day_16` and the two sets never overlap. Range-check 2024-01-01 to `current_date` rather than hunting for the literal year 2035.
- 4. FILE-SCOPE RULES RUN WITH ZERO ROWS**. R6 compares the arrived column list with the contract and fails exactly one file, `day_14`. R7 compares `rows_in_file` with the median of that table's last seven loads, 5,000, and fails exactly one file, `day_15`, where 0 sits far outside a 40% band. Every row-scope rule passes `day_15`, 0 of 0 - which is the entire reason file-scope rules exist.

GOLD – THE SHAPE THAT TRAVELS

`gold_dq_results`, one row per table, load and rule: `table_name`, `load_id`, `rule_id`, `rule_name`, `rule_scope` 'row' or 'file', `rows_checked`, `rows_failed`, `fail_rate`, `rule_status` 'pass' or 'fail'. Row scope: R1 `null_key`, R2 `duplicate_order_id`, R3 `quantity_not_numeric`, R4 `unknown_customer`, R5 `order_ts_out_of_range`. File scope: R6 `column_contract`, R7 `row_count_vs_median`. Orders runs all seven, customers only R1 and R7: 132 rows. `gold_freshness`, one row per table: `check_ts`, `last_business_ts`, `hours_behind`, `expected_interval_hours`, `freshness_status`.

THE TRAP MOST PEOPLE FALL INTO

Deduplicating inside the incoming batch. `dropDuplicates` on `day_13` reports zero, because all 150 repeated `order_ids` collide with rows loaded earlier, not with each other. Uniqueness belongs to the table, so R2 has to anti-join or MERGE against the target. Second, `day_15`: every row-scope rule scores 0 of 0 and the dashboard turns green on the one day nothing arrived. Third, measuring freshness on `_ingested_at`: you loaded every file tonight, so both tables look minutes old and the stale customer table never shows up.

THE THREE QUESTIONS YOUR GOLD LAYER MUST ANSWER

1. Rows rejected per load, per rule, per day
2. How many hours behind the newest data is each table, and is that too long?
3. Which rule fires most - and is the rule wrong rather than the data?

IN SNOWFLAKE – THE HARD ONE, WORKED

Tables this query needs in Snowflake: **GOLD_DQ_RESULTS**

```
-- Q3: a rule that fires a little on every load is describing the data.
-- A rule that fires all at once, on one load, is describing one broken file.
SELECT rule_id, rule_name,
       SUM(rows_failed)          AS total_failed,
       COUNT_IF(rows_failed > 0) AS loads_fired,
       COUNT(*)                  AS loads_checked,
       MAX(rows_failed)          AS worst_single_load,
       ROUND(MAX(rows_failed) / NULLIF(SUM(rows_failed), 0), 3) AS concentration,
       ROUND(SUM(rows_failed) / NULLIF(SUM(rows_checked), 0), 5) AS fail_rate
FROM GOLD_DQ_RESULTS
WHERE rule_scope = 'row' AND table_name = 'orders'
GROUP BY 1, 2
ORDER BY 3 DESC;
```

YOU ARE DONE WHEN

- ☐ The six new files hold 5,000, 5,000, 5,000, 5,000, 0 and 5,000 rows. Assert that the fifth is a file with 0 rows and not a missing file, and that the six sum to 25,000.
- ☐ $40 + 150 + 220 + 90 = 500$ rejects, with no row counted twice. silver_orders grows from 50,000 to exactly 74,500 and silver_rejects gains exactly 500. Close that subtraction in code, not by eye.
- ☐ Rule fires: R1 0, R2 150, R3 40, R4 220, R5 90; R6 fails one file, day_14; R7 fails one file, day_15. day_11 is the control and must fire nothing at all.
- ☐ gold_dq_results is exactly 132 rows - 16 order loads x 7 rules plus 10 customer loads x R1 and R7 - and SUM(rows_failed) across the row-scope orders rows is exactly 500.
- ☐ gold_freshness has 2 rows: customers 144 hours behind against an expected 24, status 'fail'; orders 0 hours, 'pass'. It must read silver, or the 90 rows dated 2035 keep orders looking fresh forever.

IF YOU FINISH EARLY

Make the schedule the alarm: have the quality notebook raise an exception when any rule returns 'fail', so the Job itself goes red instead of writing a red row into a table nobody opens. Then add a rule of your own and prove it fires on a file you break yourself.

THE SITUATION

The circle report says 1.82 percent of calls dropped last fortnight and nobody argues with it. Underneath, 40 of the 120 towers fail about one call in eight. They hide because together they carry only 7.69 percent of the traffic, so the healthy towers drown them in the average. The people living under those 40 towers are the ones who change operator. A second trap sits on top: a drop rate is a ratio of sums. Average the per-hour rates instead and the same fortnight reads 4.04 percent.

WHERE THE DATA COMES FROM

There is no public call detail record file. Real CDRs identify people, so operators and the regulator publish aggregates only, never per-call rows with tower_id, start time and end cause. Anything offered as a real CDR dump is either synthetic already or should not be on your laptop. The generator below is the only path, and every number quoted in this brief comes back out of it exactly, on any cluster, in any order.

GENERATE IT – THE EXACT SPECIFICATION

No random draw; every value follows from row position, so it reproduces exactly. towers 120 rows: tower_id T001-T120, site_name, city (6 cities by n%6), district (D1-D4 by n%4), capacity_channels 10 for T001-T080 and 2 for T081-T120, commissioned_on. calls: 120 towers x 15 dates from 2025-06-01 x 24 hours x cnt calls, cnt = reps x w. reps is 6 for T001-T080 and 1 for T081-T120. w is 3 in the busy hours 9-12 and 17-20, 2 in 6-8, 13-16 and 21, 1 elsewhere. That is 4,320 calls per healthy tower and 720 per weak tower, 374,400 in all. Within an hour the cnt calls start evenly, 3600/cnt seconds apart. duration_seconds is 45+(k%4)*15 in quiet hours, 240+(k%4)*120 in medium hours, 420+(k%4)*120 in busy hours. end_cause is DROPPED on every md-th row of a tower-and-w block: md is 360, 180, 72 for healthy towers at w of 1, 2, 3 and 20, 10, 6 for weak towers, giving 40 drops per healthy tower and exactly 90 per weak tower. Then, over rows numbered by call_id, inject on disjoint classes: 1,200 duration_seconds set to 'NA', 900 end_cause padded with spaces and lower-cased, 600 tower_id overwritten to 'T999', 450 duration_seconds set to -1, and 1,800 whole rows repeated, for a file of 376,200.

GENERATOR CODE – RUN THIS FIRST, EXACTLY AS WRITTEN

This assumes MY_ID and VOL are already set, as on the second page of this document. Do not change the seed or any of the counts — your checks later depend on them.

```
import pyspark.sql.functions as F; from pyspark.sql import Window
CI = "element_at(array('Pune','Kochi','Indore','Jaipur','Nagpur','Surat'),cast(id%6 as int)+1)"
PK, MD = '(9,10,11,12,17,18,19,20)', '(6,7,8,13,14,15,16,21)'
t = spark.range(120).selectExpr('id n', "format_string('T%03d',id+1) tower_id",
    f"concat({CI},' Site ',format_string('%03d',id+1)) site_name", f'{CI} city',
    "format_string('D%d',cast(id%4 as int)+1) district", 'if(id<80,10,2) capacity_channels',
    "date_add(date'2019-01-01',cast(id*11 as int)) commissioned_on")
t.drop('n').write.mode('overwrite').option('header',True).csv(f'{VOL}/raw/towers')
c = (t.withColumn('call_date', F.explode(F.expr(
    "sequence(date'2025-06-01',date'2025-06-15')"))))
    .withColumn('hour_of_day', F.explode(F.expr('sequence(0,23)'))))
    .selectExpr('n', 'tower_id', 'call_date', 'hour_of_day', 'if(n<80,6,1) reps',
        f'case when hour_of_day in {PK} then 3 when hour_of_day in {MD} then 2'
        ' else 1 end w')).selectExpr('*', 'w*reps cnt',
    'if(n<80,element_at(array(360,180,72),w),element_at(array(20,10,6),w)) md')
c = (c.withColumn('k', F.explode(F.expr('sequence(1,cnt)'))).selectExpr('*',
    'hour_of_day*3600+(k-1)*cast(3600/cnt as int) ss',
    'element_at(array(45,240,420),w)+(k%4)*element_at(array(15,120,120),w) dur',
    "concat_ws('-',tower_id,string(call_date),string(hour_of_day),string(k)) call_id")
    .withColumn('rb', F.row_number().over(Window.partitionBy('tower_id','w')
        .orderBy('call_date','hour_of_day','k'))))
c = c.selectExpr('*', "case when rb%md=0 then 'DROPPED' when rb%17=1 then 'BUSY'"
    " when rb%23=2 then 'NO_ANSWER' else 'NORMAL' end cause").withColumn(
    'rn', F.row_number().over(Window.orderBy('call_id')))
raw = c.selectExpr('rn', 'call_id',
    "format_string('SUB%05d',cast(pmod(k*37+hour_of_day*11+n*7,60000) as int)) subscriber_id",
    "if(rn%624=2,'T999',tower_id) tower_id",
    "cast(timestampadd(second,ss,cast(call_date as timestamp)) as string) start_ts",
    "case when rn%312=0 then 'NA' when rn%832=3 then '-1' else string(dur) end duration_seconds",
    "if(rn%416=1,concat(' ',lower(cause),' '),cause) end_cause",
    "if(k%2=0,'MT','MO') direction", "if(n%3=0,'5G','4G') technology")
(raw.unionByName(raw.filter('rn%208=5')).drop('rn').write.mode('overwrite')
    .option('header',True).option('ignoreLeadingWhiteSpace',False)
    .option('ignoreTrailingWhiteSpace',False).csv(f'{VOL}/raw/calls'))
```

BRONZE – LAND IT, CHANGE NOTHING

Land towers and calls exactly as written, every column a string. calls must read back as 376,200 rows and towers as 120. Add `_source_file` from `input_file_name()`, `_ingested_at` from `current_timestamp()` and `_row_hash` over the concatenated raw columns. Do not trim `end_cause`, do not cast `duration_seconds`, do not drop the 'T999' rows. How many broken rows the collector sent is a fact about the collector, lost the moment you clean here.

SILVER – THE DECISIONS YOU MUST MAKE, AND RECORD

Each decision below has a number attached to it. If your number does not match, stop and find out why before you go on — that gap is the finding, not a nuisance.

1. CAST with `try_cast`, never cast: `duration_seconds` to INT, `start_ts` to TIMESTAMP. ANSI mode is on, so cast throws on the first bad value and the notebook stops. Exactly 1,200 rows hold the string 'NA' in `duration_seconds` and go NULL. Send those 1,200 to `silver_rejects` with reason 'duration_not_numeric'. A call with no duration is unknown, not a zero-second call, so it leaves both numerator and denominator.
2. DEDUPE on `call_id`. The collector re-sent one slice of its file. 376,200 rows in, 374,400 out, exactly 1,800 removed: ROW_NUMBER over `call_id` ordered by `_ingested_at`, keep rank 1. Check both counts and the difference before going on. Removing more than 1,800 means your key is wrong; removing none means you deduped on something already unique, such as the whole row.
3. ORPHANS and RANGE. Exactly 600 rows carry `tower_id` 'T999', which matches no tower master row, so they can be blamed on nobody: reject, reason 'unknown_tower'. Exactly 450 rows carry `duration_seconds` -1. Reject those with a rule - duration outside 0 to 7200 seconds - and reason 'duration_out_of_range', not by hunting for -1, because the next collector bug sends a different wrong number. Silver = 372,150 rows.
4. END CAUSE, defined once. Exactly 900 rows hold the cause padded with spaces and in lower case, so a raw GROUP BY shows eight causes instead of four and the drop count comes up short. trim then upper before anything else, then set `is_dropped` = (`end_cause` = 'DROPPED') here and nowhere else. Derive `end_ts` = `start_ts` + `duration_seconds` here too; the concurrency work needs it.

GOLD – THE SHAPE THAT TRAVELS

One row per tower per calendar date per hour of day: `tower_id`, `site_name`, `city`, `district`, `capacity_channels`, `call_date`, `hour_of_day`, `calls`, `dropped_calls`, `drop_rate`, `peak_concurrent`, `utilisation`, `avg_duration_seconds`. $120 \times 15 \times 24 = 43,200$ cells, less 58 whose only call was rejected, so exactly 43,142 rows. `peak_concurrent` is the most calls open at once in that cell: emit +1 at `start_ts` and -1 at `end_ts`, running SUM over time, take the max. `utilisation` = `peak_concurrent` / `capacity_channels`.

THE TRAP MOST PEOPLE FALL INTO

Writing `AVG(drop_rate)` over the Gold rows, or a rate per tower-hour averaged up to the tower. That is an average of ratios, and it gives a quiet hour with 6 calls and 1 drop the same vote as a busy hour with 18 calls and none. Here it reads 4.04 percent instead of 1.82, more than double, and it reshuffles the order inside the worst 40. Carry `SUM(dropped_calls)` and `SUM(calls)` as two columns to the final query and divide once, at the end, with NULLIF on the denominator. Same rule per day in question 2.

THE THREE QUESTIONS YOUR GOLD LAYER MUST ANSWER

1. Drop rate per tower per hour of day, and the hour each tower is worst
2. Which towers exceed a 5 percent drop rate on more than 10 of the 15 days?
3. Does drop rate rise with concurrent calls on the same tower, or is it flat?

IN SNOWFLAKE – THE HARD ONE, WORKED

Tables this query needs in Snowflake: **GOLD_TOWER_HOUR**

```
-- Q1. Drop rate is SUM(dropped) / SUM(calls): a ratio of sums.
-- The window SUMs put each tower's own overall rate beside its worst hour.
WITH th AS (
  SELECT tower_id, hour_of_day,
         SUM(calls) AS calls, SUM(dropped_calls) AS drops
  FROM   GOLD_TOWER_HOUR
  GROUP BY tower_id, hour_of_day)
SELECT tower_id, hour_of_day, calls, drops,
       ROUND(100.0 * drops / NULLIF(calls, 0), 2) AS worst_hour_pct,
       ROUND(100.0 * SUM(drops) OVER (PARTITION BY tower_id
                                     / NULLIF(SUM(calls) OVER (PARTITION BY tower_id), 0), 2) AS tower_pct
  FROM   th
  QUALIFY ROW_NUMBER() OVER (PARTITION BY tower_id
                             ORDER BY drops / NULLIF(calls, 0) DESC, hour_of_day) = 1
  ORDER BY worst_hour_pct DESC;
```

YOU ARE DONE WHEN

- ☐ Bronze calls = 376,200 rows and Bronze towers = 120. Count the file you wrote, not the DataFrame you think you wrote. Reading back 374,400 means the repeated slice never landed.
- ☐ Silver = 376,200 - 1,800 repeated call_id - 1,200 'NA' durations - 600 'T999' - 450 negative durations = 372,150. Print all five numbers and check the subtraction closes.
- ☐ Gold = 43,142 rows, SUM(calls) = 372,150, SUM(dropped_calls) = 6,792, circle rate 1.82 percent. Ranked by ratio of sums the worst 40 tower_ids are exactly T081 to T120, each near 12.5 percent.
- ☐ peak_concurrent is 4 in busy hours on T001-T080, 2 in medium hours, 1 otherwise, and 1 everywhere on T081-T120. Drop rate by utilisation rises 0.27, 0.56, 1.40, 12.57 percent across utilisation 0.1, 0.2, 0.4, 0.5.

IF YOU FINISH EARLY

Add a 7-day trailing drop rate per tower with ROWS BETWEEN 6 PRECEDING AND CURRENT ROW over the daily sums, and flag the first date each tower crosses 5 percent. Then bucket by raw peak_concurrent instead of utilisation and watch it invert: 4.37, 0.56, 1.40 percent. Explain why before believing either number.

THE SITUATION

A referral scheme has 600 members in six cities. The leaderboard names the top referrer: one member brought in 34 people. Look one step further and that reach stops dead, because those 34 already knew each other - only 34 more members sit at two hops. Five other members have about a dozen contacts each and reach more than fifty at two hops, because each is the only link between two cities. Remove one and the network breaks apart. Nobody has noticed, because no column says 'bridge'. You have to join the edge list to itself to find them.

WHERE THE DATA COMES FROM

There is no public referral-graph file of this shape. Real referral tables belong to one company and identify people, and published network datasets carry none of referrer, referred, date, channel and status together. So the generator below is the only path. One thing to settle before you start: GraphFrames is NOT available on Databricks Free Edition serverless and no third-party library can be installed, so do not go looking for it. Everything here is self-joins on the edge table in plain PySpark.

GENERATE IT — THE EXACT SPECIFICATION

Determined by row position, no random draw. members 600 rows: member_id M001-M600, city (6 cities, M001-M100 in the first), joined_on, segment by id%3. Write m as 0-599, city $c = m/100$, local $l = m\%100$. Edges come in four blocks, numbered in that order. ring: every member refers $(l+s)\%100$ in its own city, s in [1,2,3,5,8,13] when $l\%4=0$, [1,2,3,5] when 1, [1,2,3] when 2, [1,2] when 3 - 2,250 rows. hub: the member at $l=0$ in each city refers 30 more, s = 21 to 50 - 180 rows. mutual: for every $l\%10=0$ the member at $l+1$ refers back - 60 reciprocal pairs. bridge: the five members at $l=50$ in the first five cities each refer 6 members at $l=50$ to 55 in the next city - 30 rows, the only edges that leave a city. Total 2,520. referred_at spreads over 2025-01 to 2025-06 by step, so the network grows month on month. status is JOINED except on ring rows with $s>1$, where every 25th is REJECTED and the next PENDING. Then, over rows numbered by build order, inject on disjoint classes: 45 self-referrals (referred_id set to referrer_id), 60 rows with referred_id 'M999' matching no member, 90 rows with status padded and lower-cased, 72 with referred_at 'NA', and 120 whole rows repeated - a file of 2,640.

GENERATOR CODE – RUN THIS FIRST, EXACTLY AS WRITTEN

This assumes MY_ID and VOL are already set, as on the second page of this document. Do not change the seed or any of the counts — your checks later depend on them.

```
import pyspark.sql.functions as F; from pyspark.sql import Window
CI = "element_at(array('Pune','Kochi','Indore','Jaipur','Nagpur','Surat'),id div 100+1)"
ST = "element_at(array(array(1,2,3,5,8,13),array(1,2,3,5),array(1,2,3),array(1,2)),l%4+1)"
m = spark.range(600).selectExpr("format_string('M%03d',id+1) member_id", f'{CI} city',
    "date_add(date'2024-06-01',cast(id%180 as int)) joined_on",
    "element_at(array('STUDENT','PRO','SMB'),cast(id%3 as int)+1) segment",
    'id mno', 'id div 100 c', 'cast(id%100 as int) l')
m.drop('mno','c','l').write.mode('overwrite').option('header',True).csv(f'{VOL}/raw/members')
ring = m.withColumn('s', F.explode(F.expr(ST))).selectExpr('mno src',
    'c*100+(l+s)%100 dst', 'element_at(map(1,6,2,5,3,4,5,3,8,2,13,1),s) mon', '0 grp', 's sk')
hub = (m.filter('l=0').withColumn('s', F.expr('explode(sequence(21,50))')))
    .selectExpr('mno src', 'c*100+(l+s)%100 dst', '4+s%3 mon', '1 grp', 's sk'))
mut = m.filter('l%10=0').selectExpr('c*100+(l+1)%100 src', 'mno dst', '5 mon',
    '2 grp', '0 sk')
brg = (m.filter('mno in (50,150,250,350,450)').withColumn('j', F.expr('explode(sequence(0,5))')))
    .selectExpr('mno src', '(c+1)*100+50+j dst', '6 mon', '3 grp', 'j sk'))
e = (ring.unionByName(hub).unionByName(mut).unionByName(brg)
    .withColumn('rn', F.row_number().over(Window.orderBy('grp', 'src', 'sk'))))
    .selectExpr('*', "case when grp=0 and sk>1 and rn%25=0 then 'REJECTED'"
    " when grp=0 and sk>1 and rn%25=1 then 'PENDING' else 'JOINED' end st0"))
raw = e.selectExpr('rn', "format_string('R%05d',rn) referral_id",
    "format_string('M%03d',src+1) referrer_id",
    "case when rn%56=0 then format_string('M%03d',src+1) when rn%42=1 then 'M999'"
    " else format_string('M%03d',dst+1) end referred_id",
    "case when rn%35=3 then 'NA' else string(date_add("
    "add_months(date'2025-01-01',mon-1),cast(rn%28 as int))) end referred_at",
    "element_at(array('APP','SMS','EMAIL','EVENT'),cast(rn%4 as int)+1) channel",
    "if(rn%28=2,concat(' ',lower(st0),' '),st0) status")
(raw.unionByName(raw.filter('rn%21=4')).drop('rn')).write.mode('overwrite')
    .option('header',True).option('ignoreLeadingWhiteSpace',False)
    .option('ignoreTrailingWhiteSpace',False).csv(f'{VOL}/raw/referrals'))
```

BRONZE – LAND IT, CHANGE NOTHING

Land members and referrals as written, every column a string. referrals must read back as 2,640 rows and members as 600. Add `_source_file`, `_ingested_at` and `_row_hash`. Say it once now: GraphFrames is not installable on Free Edition and you will not use it. Every question below is answered by joining the edge table to itself - one join for mutual pairs and two-hop, three for triangles - so your job in Silver is an edge table clean enough that a self-join cannot lie to you.

SILVER – THE DECISIONS YOU MUST MAKE, AND RECORD

Each decision below has a number attached to it. If your number does not match, stop and find out why before you go on — that gap is the finding, not a nuisance.

1. CAST `referred_at` to DATE with `try_cast`, never `cast`, because ANSI mode makes cast throw. Exactly 72 rows hold 'NA' and go NULL: reject with reason 'no_referral_date'. Question 1 needs the month, and an edge with no date cannot be placed in one, so it must not be quietly kept as month zero.
2. DEDUPE on `referral_id`. 2,640 rows in, 2,520 out, exactly 120 removed. Use `referral_id`, not the pair (`referrer_id`, `referred_id`): the injected self-referrals and 'M999' rows carry their own `referral_id` and have to survive dedupe so you can count and reject them properly in the next step.
3. SELF-REFERRALS and UNKNOWN MEMBERS. Exactly 45 rows have `referrer_id` = `referred_id` - reject, reason 'self_referral'; a loop is not a link and it shows up later as a fake triangle. Exactly 60 rows have `referred_id` = 'M999', which matches no member - reject, reason 'unknown_member', found by an anti-join against members rather than a hardcoded 'M999'. Silver = 2,520 - 72 - 45 - 60 = 2,343 rows.
4. STATUS once, then the link table. Exactly 90 rows hold status padded with spaces and lower-cased, so trim then upper first. Keep only status = 'JOINED': 2,227 rows, setting aside 87 REJECTED and 29 PENDING, because a pending invite is not a relationship. Then build LINKS, one row per undirected pair: `least(referrer_id, referred_id) AS a_id, greatest(...) AS b_id, DISTINCT` - 2,175 rows.

GOLD – THE SHAPE THAT TRAVELS

Two tables. `gold_member_month`, one row per member per month, dense so zero months are present: `member_id`, `city`, `segment`, `month`, `out_edges`, `in_edges`, `cum_out_degree`, `cum_in_degree` - 600 x 6 = exactly 3,600 rows, with `SUM(out_edges) = SUM(in_edges) = 2,227`. `gold_member_graph`, one row per member: `member_id`, `city`, `segment`, `out_degree`, `in_degree`, `undirected_degree`, `mutual_pairs`, `triangles`, `one_hop`, `two_hop_exact`, `cities_within_two` - exactly 600 rows, none missing, none isolated.

THE TRAP MOST PEOPLE FALL INTO

Counting each undirected pair twice. A referral row is directed, so a self-join returns (A,B) and (B,A), every pair counts double and every triangle six times. Fix it once in Silver: build LINKS with `least()/greatest()` and `DISTINCT`, then keep an ordering predicate `a.x < b.x` in every self-join after. Two more: two-hop without an anti-join against the one-hop set and against self counts direct contacts twice and every member as reaching itself; and keeping PENDING and REJECTED invents 116 links nobody accepted.

THE THREE QUESTIONS YOUR GOLD LAYER MUST ANSWER

1. Out-degree and in-degree per member, and how each changed month by month
2. Mutual pairs and closed triangles - which members sit in the most?
3. Two-hop reach: how many members are reachable in exactly two steps, not one?

IN SNOWFLAKE – THE HARD ONE, WORKED

Tables this query needs in Snowflake: **LINKS, GOLD_MEMBER_GRAPH**

```
-- Q2. Triangles by joining the edge list to itself three times.
-- LINKS already holds each undirected pair once with a_id < b_id, so
-- forcing x < y < z counts each triangle once instead of six times.
WITH t AS (
  SELECT e1.a_id AS x, e1.b_id AS y, e2.b_id AS z
  FROM   LINKS e1
  JOIN   LINKS e2 ON e2.a_id = e1.b_id AND e2.b_id > e1.b_id
  JOIN   LINKS e3 ON e3.a_id = e1.a_id AND e3.b_id = e2.b_id),
p AS (SELECT x AS member_id FROM t
      UNION ALL SELECT y FROM t
      UNION ALL SELECT z FROM t)
SELECT p.member_id, g.city, g.undirected_degree AS degree,
       COUNT(*) AS triangles
FROM   p JOIN GOLD_MEMBER_GRAPH g ON g.member_id = p.member_id
GROUP BY p.member_id, g.city, g.undirected_degree
QUALIFY RANK() OVER (ORDER BY COUNT(*) DESC) <= 10
ORDER BY triangles DESC, p.member_id;
```

YOU ARE DONE WHEN

- ☐ Bronze referrals = 2,640 and members = 600. After dedupe on referral_id, 2,520: exactly 120 removed. Count the written file, not the DataFrame.
- ☐ Silver = 2,520 - 72 unreadable dates - 45 self-referrals - 60 unknown members = 2,343. Of those 2,227 are JOINED, 87 REJECTED, 29 PENDING, and $2,227 + 87 + 29 = 2,343$.
- ☐ LINKS = 2,175 undirected pairs and mutual pairs = 52, so $2,175 + 52 = 2,227$ joined edges: each reciprocal pair is two edges folded into one link. Triangles = 2,164, and $\text{SUM}(\text{triangles})$ over the 600 members = $6,492 = 3 \times 2,164$.
- ☐ gold_member_graph = 600 rows, $\text{SUM}(\text{undirected_degree}) = 4,350 = 2 \times 2,175$, and no member has one_hop = 0. Exactly 5 members have two_hop_exact of 52 or more - M051, M151, M251, M351, M451 - while the four largest out-degrees, 34 each, reach only 33 to 36.

IF YOU FINISH EARLY

Snowflake's recursive CTE reaches depth beyond two - three, four, or the whole component - which self-joins cannot do without one join per hop. It is not needed for these three questions, but write it and compare hop 2 with your PySpark answer. Then drop the 30 bridge edges and recount two_hop_exact.

THE SITUATION

Nothing alerted. The firewall did its job all morning and denied what it was told to deny. A scan is not one strange packet - it is one source touching many ports in a short window, and it reads as a busy client until you count distinct destination ports per source per minute. In this three hours, 27.5% of every denied connection comes from three addresses that were never once allowed through. Three sources cross 100 distinct ports inside a five-minute window, and one of the three is your own backup server.

WHERE THE DATA COMES FROM

Raw firewall text of this shape is hard to get honestly. The captures people name - CICIDS2017, UNSW-NB15, the LANL 2017 release - are flow records or labelled feature tables, not firewall log lines; several sit behind a registration form, the files are large, and the column layout changes between releases. Do not quote a schema for one of them here without opening it yourself first. There is no small public file with the line shape this project needs, so the generator below is the primary path and the only one whose numbers are checkable.

GENERATE IT – THE EXACT SPECIFICATION

Seed 33. One text file, 233,200 lines, all 2025-04-08, 180 one-minute buckets from 06:00. Line: 'YYYY-MM-DD HH:MM:SS fw-edge-01 <proto> <ACTION> <src>:<sport> > <dst>:<dport> bytes=<n|-> eid=E<9 digits>'. Body 224,700 events, each with its own eid. Chatter: 200 sources 10.30.0.1-10.30.3.50, 6 events a minute each, 216,000 rows, dst ports from a fixed set of 8 so each shows 6 distinct ports a minute; DENY where $(6s+r) \bmod 25 = 0$, exactly 8,640; sources 10.30.0.31-10.30.0.38 send 1 ICMP event a minute with both ports written '-', exactly 1,440; 10.30.0.5, 10.30.0.9 and 10.30.0.13 write accept in lower case, exactly 2,700, every accept they ever send. Scanners, all DENY, never an accept: 203.0.113.45 hits 10.30.9.7 on ports 1-1200 over minutes 30-39, 120 a minute; 198.51.100.22 hits ports 1000-1899 over minutes 20-109, 10 a minute; 10.30.7.99 hits 20 hosts on ports 1024-2223 over minutes 100-114, 80 a minute. Backup 10.30.8.20 streams to 8 storage hosts on ports 20000-20999, 30 a minute for all 180 minutes: 5,400 rows, 60 DENY, 5,340 ACCEPT. Every DENY writes bytes='-'. Defects: chatter minutes 0-4 repeated, 6,000 lines; 2,000 truncated after the protocol; 500 blank.

GENERATOR CODE – RUN THIS FIRST, EXACTLY AS WRITTEN

This assumes MY_ID and VOL are already set, as on the second page of this document. Do not change the seed or any of the counts — your checks later depend on them.

```
P = "array(22,80,443,3306,53,123,8080,9000)"
a = spark.range(216000).selectExpr("id", "cast(id div 1200 as int) m",
    "cast(pmod(id div 6, 200) as int) s", "cast(pmod(id, 6) as int) r")
a = a.selectExpr("m", "format_string('10.30.%d.%d', s div 50, pmod(s, 50) + 1) src",
    "format_string('10.30.9.%d', pmod(id, 20) + 1) dst",
    f"if(s between 30 and 37 and r = 0, '-', cast(element_at({P}, "
    " pmod(6*s + r, 8) + 1) as string)) dps",
    "if(s between 30 and 37 and r = 0, 'icmp', 'tcp') pr",
    "if(pmod(6*s + r, 25) = 0, 'DENY', if(s in (4,8,12), 'accept', 'ACCEPT')) act",
    "if(pmod(6*s + r, 25) = 0, '-', cast(200 + pmod(id, 9000) as string)) by",
    "format_string('E%09d', id) eid")
sc = spark.range(3300).selectExpr(
    "cast(if(id < 1200, 30 + id div 120, if(id < 2100, 20 + (id - 1200) div 10, "
    " 100 + (id - 2100) div 80)) as int) m",
    "if(id < 1200, '203.0.113.45', if(id < 2100, '198.51.100.22', '10.30.7.99')) src",
    "if(id < 1200, '10.30.9.7', format_string('10.30.50.%d', pmod(id, 20) + 1)) dst",
    "cast(if(id < 1200, 1 + id, if(id < 2100, id - 200, id - 1076)) as string) dps",
    "'tcp' pr", "'DENY' act", "'-' by", "format_string('E%09d', 1000000 + id) eid")
bk = spark.range(5400).selectExpr("cast(id div 30 as int) m", "'10.30.8.20' src",
    "format_string('10.30.50.%d', pmod(id, 8) + 1) dst",
    "cast(20000 + pmod(id, 1000) as string) dps", "'tcp' pr",
    "if(pmod(id, 90) = 0, 'DENY', 'ACCEPT') act",
    "if(pmod(id, 90) = 0, '-', cast(9000 + pmod(id, 900) as string)) by",
    "format_string('E%09d', 2000000 + id) eid")
LN = ("concat_ws(' ', date_format(timestamp'2025-04-08 06:00:00' + make_dt_interval("
    "0, 0, m, cast(pmod(crc32(eid), 60) as decimal(8,6))), 'yyyy-MM-dd HH:mm:ss'),"
    " 'fw-edge-01', pr, act, concat(src, ':', if(pr = 'icmp', '-', "
    " cast(40000 + pmod(crc32(eid), 9000) as string))), '>',"
    " concat(dst, ':', dps), concat('bytes=', by), concat('eid=', eid)) value")
tr = spark.range(2000).selectExpr("concat('2025-04-08 06:', "
    " lpad(cast(pmod(id, 60) as string), 2, '0'), ':00 fw-edge-01 tcp') value")
(a.union(sc).union(bk).selectExpr(LN).union(a.filter('m < 5').selectExpr(LN))
    .union(tr).union(spark.range(500).selectExpr("' ' value"))
    .write.mode('overwrite').text(f'{VOL}/raw/fw_logs'))
```

BRONZE – LAND IT, CHANGE NOTHING

`spark.read.text` gives one column, value, holding the raw line - that is Bronze. Add `_source_file`, `_ingested_at` and `_line_length` and stop. Do not parse, do not drop the blank or truncated lines, do not lower-case anything. Bronze is exactly 233,200 rows and that number must equal the line count of the file. A scan hides in the rows a pipeline threw away before anyone looked, so the count of lines you could not read is a published number, not a silent filter.

SILVER – THE DECISIONS YOU MUST MAKE, AND RECORD

Each decision below has a number attached to it. If your number does not match, stop and find out why before you go on — that gap is the finding, not a nuisance.

1. PARSE with one anchored regex capturing timestamp, device, protocol, action, `src_ip`, `src_port`, `dst_ip`, `dst_port`, bytes and `eid`. Exactly 2,500 lines yield nothing - 2,000 truncated after the protocol and 500 blank - and every one goes to a reject table with its raw line. Parse rate $230,700 / 233,200 = 98.93\%$. Publish that number.
2. DEDUPE ON `eid`, which is why the generator writes one. 230,700 parsed rows fall to exactly 224,700, so 6,000 go, and the DENY count falls from 12,240 to 12,000. Print both. Never dedupe on the raw line: two real connections from one host in the same second to the same port are identical text and both of them happened.
3. PORTS AND BYTES ARE NULL, NOT ZERO. Exactly 1,440 ICMP events write '-' for both ports; `try_cast` to INT gives NULL, and `coalesce` to 0 invents a port 0 that adds a fake distinct port to eight sources. Exactly 12,000 DENY rows write `bytes='-'`; a refused packet carried no bytes, which is not the same as carrying zero.
4. UPPER-CASE THE ACTION ONCE, HERE. Exactly 2,700 lines write accept in lower case, and they are every accept ever sent by 10.30.0.5, 10.30.0.9 and 10.30.0.13. Compare case-sensitively and those three look like sources that were never allowed through: 540 extra denies land in question 3 and 27.5% becomes 32.0%.

GOLD – THE SHAPE THAT TRAVELS

`gold_source_minute`, one row per source IP per minute: `src_ip`, `minute_ts`, events, `distinct_dst_ports`, `distinct_dst_ips`, denies, accepts, `deny_ratio`, `is_internal` - exactly 36,295 rows. That is 200 chatter sources x 180 minutes, plus 180 for the backup server, plus 10, 90 and 15 for the three scanners, which only exist while they are scanning. Keep minutes with one event; `SUM(events)` over the table is 224,700.

THE TRAP MOST PEOPLE FALL INTO

Calling 10.30.8.20 a scanner. It clears 100 distinct ports in 177 five-minute windows, more than either real scanner, because a backup agent opens a fresh high port per stream. But 5,340 of its 5,400 connections were accepted, and a port is only evidence when the far side refused it. The rule that survives is denied with no accept ever, not port count. That threshold also misses 198.51.100.22: 900 ports over 90 minutes is 50 in any five-minute window, under any threshold you would pick, and it is a scanner.

THE THREE QUESTIONS YOUR GOLD LAYER MUST ANSWER

1. Distinct destination ports per source IP per minute, and the top offenders
2. Which sources hit more than 100 distinct ports in any 5-minute window?
3. What share of denied traffic comes from sources that never had a single accept?

IN SNOWFLAKE – THE HARD ONE, WORKED

Tables this query needs in Snowflake: **SILVER_FW**, **GOLD_SOURCE_MINUTE**

```
-- COUNT(DISTINCT ...) is not allowed as a window function in Snowflake, so a
-- sliding five-minute distinct-port count is two steps: reduce to one row per
-- source, minute and port, then re-count inside each window with a self-join.
WITH sp AS (SELECT DISTINCT src_ip, minute_ts, dst_port
  FROM SILVER_FW WHERE dst_port IS NOT NULL),
sm AS (SELECT DISTINCT src_ip, minute_ts FROM sp),
w AS (SELECT a.src_ip, a.minute_ts AS win_start,
  COUNT(DISTINCT b.dst_port) AS ports_5m
  FROM sm a JOIN sp b ON b.src_ip = a.src_ip
  AND b.minute_ts >= a.minute_ts
  AND b.minute_ts < DATEADD(minute, 5, a.minute_ts)
  GROUP BY 1, 2)
SELECT w.src_ip, MAX(w.ports_5m) AS peak_ports_5m,
  COUNT_IF(w.ports_5m > 100) AS windows_over_100,
  MAX(t.accepts) AS accepts_all_day
FROM w JOIN (SELECT src_ip, SUM(accepts) AS accepts
  FROM GOLD_SOURCE_MINUTE GROUP BY 1) t ON t.src_ip = w.src_ip
GROUP BY 1
HAVING MAX(w.ports_5m) > 100
ORDER BY peak_ports_5m DESC;
```

YOU ARE DONE WHEN

- ☐ Bronze 233,200 equals the file's line count; 230,700 after 2,500 unparsed; 224,700 after 6,000 duplicate eids. Print all four numbers and close the subtraction in code.
- ☐ Silver has exactly 204 distinct src_ip. DENY 12,000 plus ACCEPT 212,700 equals 224,700, and exactly 1,440 rows carry a NULL dst_port.
- ☐ gold_source_minute has exactly 36,295 rows and SUM(events) = 224,700. The peak distinct_dst_ports is 120 for 203.0.113.45, then 80, then 30; every chatter source peaks at 6, and the eight ICMP senders at 5.
- ☐ The query returns exactly 3 rows: 203.0.113.45 at 600 ports, 10.30.7.99 at 400, 10.30.8.20 at 150 - and only the last has a non-zero accepts_all_day, 5,340. Question 3 is 3,300 / 12,000 = 27.5%.

IF YOU FINISH EARLY

Add a per-source day table: distinct ports over the whole three hours, denies, accepts, first and last seen. The slow scanner that no five-minute threshold catches appears immediately at 900 distinct ports and zero accepts.

THE SITUATION

The capacity report says the busiest circuit in the region averages 4.6 Mbps on a 10 Mbps link, so nobody orders an upgrade. It does not. A router interface counter does not report bandwidth - it reports a total that only climbs, until it passes 4,294,967,296 and starts again at zero, and the difference between two polls becomes about minus four billion bytes. A pipeline that drops negative rows throws away 992 of the day's samples, and they are the fastest ones. Corrected, six links sit above 80% of rated for 2,875 minutes. Dropped, they report 2,645.

WHERE THE DATA COMES FROM

There is no dependable public file of raw 32-bit interface counters. Vendor MIBs do define ifInOctets as a Counter32, so the wrap is real and every network engineer has met it, but the public archives are RRD or MRTG exports that already converted counters to rates and repaired the wraps before writing - the defect this project is about is gone before you can see it. Do not quote columns or row counts for a telemetry dump you have not opened. The generator below is the primary path.

GENERATE IT – THE EXACT SPECIFICATION

Seed 34. Two CSVs with headers. if_polls: 40 interfaces, RTR-01..RTR-10 x ge-0/0/1..4, 288 polls of 300 s on 2025-05-06: 11,520 rows plus 2,000 dups = 13,520. Columns device_id, interface_id, poll_ts, in_octets, out_octets, if_speed_mbps, admin_status, sys_uptime_s, poll_id. Rated by index: 10, 10, 50, 100 Mbps. Peak = polls 121-216. ge-0/0/1 on RTR-01..RTR-06 runs 90% of rated in peak, 25% off; all others 40%/15%. Per-poll octets: rated x 1e6 x util / 8 x 300; out_octets by half. Counters are 32-bit: stored value = running total mod 4,294,967,296, so in_octets wraps exactly 992 times and out_octets 479; 573 of the 992 fall on the ten 100 Mbps ports, 284 on the ten 50, 135 on the twenty 10. Exactly 4 reboots: RTR-03 at poll 150, RTR-06 at 60, RTR-07 at 200, RTR-09 at 250; each resets that device's 4 counters to a tenth of an interval and sys_uptime_s to 30: 16 reset rows. RTR-08 is admin down for polls 96-119: 96 rows, status 'down', frozen counters, if_speed_mbps 0. if_master: 40 rows - device_id, interface_id, site, rated_mbps, role; the 4 RTR-05 rows pad device_id with a trailing space. Duplicates: polls 0-49 repeated for all 40 interfaces, 2,000 rows, each keeping its poll_id.

GENERATOR CODE – RUN THIS FIRST, EXACTLY AS WRITTEN

This assumes MY_ID and VOL are already set, as on the second page of this document. Do not change the seed or any of the counts — your checks later depend on them.

```
W = 4294967296
PT = "partition by device_id, interface_id"
UB = f"{PT} order by p rows between unbounded preceding and current row"
i = spark.range(40).selectExpr("cast(1 + id div 4 as int) dv",
    "cast(pmod(id, 4) as int) ix",
    "format_string('RTR-%02d', 1 + id div 4) device_id",
    "format_string('ge-0/0/%d', pmod(id, 4) + 1) interface_id",
    "format_string('SITE-%02d', 1 + id div 4) site",
    "element_at(array(10, 10, 50, 100), pmod(id, 4) + 1) rated_mbps",
    "element_at(array('access', 'access', 'edge', 'core'), pmod(id, 4) + 1) role")
i.selectExpr("if(dv = 5, concat(device_id, ' '), device_id) device_id", "interface_id",
    "site", "rated_mbps", "role").write.mode('overwrite').options(header=True,
    ignoreLeadingWhiteSpace=False, ignoreTrailingWhiteSpace=False).csv(f'{VOL}/raw/if_master')
p = i.crossJoin(spark.range(288).withColumnRenamed('id', 'p'))
p = p.selectExpr("*(dv = 8 and p between 96 and 119) dn",
    "case dv when 3 then 150 when 6 then 60 when 7 then 200 when 9 then 250 end pr",
    "cast(rated_mbps * 1000000 * if(ix = 0 and dv <= 6,
    " if(p between 121 and 216, 90, 25), if(p between 121 and 216, 40, 15))"
    " / 800 * 300 as bigint) d0")
p = p.selectExpr("*(sum(if(dn, 0, d0)) over ({UB}) ci",
    "sum(if(dn, 0, d0) div 2) over ({UB}) co")
p = p.selectExpr("*(max(if(p = pr, ci, null)) over ({PT}) ri",
    "max(if(p = pr, co, null)) over ({PT}) ro", "max(if(p = pr, d0, null)) over ({PT}) rd")
p = p.selectExpr("device_id", "interface_id",
    "timestamp'2025-05-06 00:00:00' + make_dt_interval(0, 0, cast(5 * p as int)) poll_ts",
    "pmod(if(pr is not null and p >= pr, ci - ri + rd div 10, ci), {W}) in_octets",
    "pmod(if(pr is not null and p >= pr, co - ro + (rd div 2) div 10, co), {W})"
    " out_octets",
    "if(dn, 0, rated_mbps) if_speed_mbps", "if(dn, 'down', 'up') admin_status",
    "if(pr is not null and p >= pr, 30 + 300 * (p - pr), 100000 + dv * 1000 + 300 * p)"
    " sys_uptime_s", "format_string('PL%02d%d%03d', dv, ix, p) poll_id", "p")
(p.drop('p').union(p.filter('p < 50').drop('p')).write.mode('overwrite')
    .options(header=True, ignoreLeadingWhiteSpace=False, ignoreTrailingWhiteSpace=False)
    .csv(f'{VOL}/raw/if_polls'))
```

BRONZE – LAND IT, CHANGE NOTHING

Read both folders with header=True and every column as STRING, then stop. in_octets runs to 4,294,967,295, which does not fit an INT, so cast it in Silver with try_cast to BIGINT rather than letting an inferred schema decide. Bronze polls is exactly 13,520 rows and bronze master exactly 40. Do not join the two, do not dedupe, do not sort, and do not trim - the trailing space in four master rows is data you have to be able to see later.

SILVER – THE DECISIONS YOU MUST MAKE, AND RECORD

Each decision below has a number attached to it. If your number does not match, stop and find out why before you go on — that gap is the finding, not a nuisance.

1. DEDUPE ON poll_id BEFORE ANY LAG. 13,520 rows fall to exactly 11,520, so 2,000 go, and 40 interfaces x 288 polls = 11,520 closes it. Order matters here: leave the duplicates in and LAG compares a row with its own copy at the same poll_ts, the delta comes out 0, and polls 0-49 report about half the real throughput on every interface.
2. A NEGATIVE DELTA IS A WRAP, NOT A ROW TO DROP. Exactly 992 in_octets deltas and 479 out_octets deltas come out negative because the counter passed 4,294,967,296; add 4,294,967,296 to each and keep the row. The check that the fix is right: no corrected delta exceeds 1,500,000,000, which is the largest one interval can hold here.
3. A REBOOT IS NOT A WRAP. Exactly 16 rows - 4 devices x 4 interfaces - have a sys_uptime_s lower than the previous poll's. There the counter restarted near zero, so the interval is genuinely unknown: set the delta NULL and keep the row. Add 2^{32} there instead and you invent up to 4.29 GB in five minutes, a false 114 Mbps.
4. TRIM THE JOIN KEY AND DISTRUST if_speed_mbps. Exactly 4 master rows, RTR-05's, carry a trailing space in device_id, so an untrimmed join silently drops 4 interfaces - one of them RTR-05 ge-0/0/1, which is one of the six links question 2 is about, so the answer comes back 5. And 96 polls report if_speed_mbps 0 while RTR-08 is down; take rated_mbps from the master and wrap it in NULLIF.

GOLD – THE SHAPE THAT TRAVELS

gold_iface_interval, one row per device_id, interface_id and 5-minute interval, keyed on the interval's starting poll: device_id, interface_id, site, role, rated_mbps, bucket_start, in_octets_delta, out_octets_delta, in_mbps, out_mbps, util_pct, wrap_flag, reboot_flag - exactly 11,480 rows, 40 interfaces x 287 intervals. The first poll of each interface has no predecessor and makes no row. 16 rows carry reboot_flag true with NULL deltas; 992 carry wrap_flag true.

THE TRAP MOST PEOPLE FALL INTO

WHERE delta > 0. It reads like hygiene and it is a 992-row hole in exactly the wrong places, because the busiest intervals are the ones that wrap. Each of the six 90%-loaded links then reports 440 minutes above 80% instead of 480, and its daily mean falls 2.2%; the 100 Mbps links lose 6.8%. The other half is correcting every negative: 16 of the 1,008 are reboots, where the counter restarted rather than rolled over, and adding 2^{32} there books 4.29 GB in five minutes - a false 114 Mbps on a 10 Mbps circuit.

THE THREE QUESTIONS YOUR GOLD LAYER MUST ANSWER

1. Throughput in Mbps per interface per 5-minute bucket
2. Which interfaces run above 80% of their rated speed, and for how many minutes a day?
3. Which interfaces show the most counter resets, and are those the busiest ones?

IN SNOWFLAKE – THE HARD ONE, WORKED

Tables this query needs in Snowflake: **SILVER_POLL**, **DIM_INTERFACE**

```
-- LAG gives the raw delta. A negative one is a wrap, UNLESS sys_uptime_s fell:
-- then the device rebooted and the interval is unknown, not 2^32 larger.
WITH d AS (SELECT device_id, interface_id, poll_ts,
  in_octets - LAG(in_octets) OVER (PARTITION BY device_id, interface_id
    ORDER BY poll_ts) AS raw_d,
  sys_uptime_s - LAG(sys_uptime_s) OVER (PARTITION BY device_id, interface_id
    ORDER BY poll_ts) AS up_d
FROM SILVER_POLL),
c AS (SELECT d.*, CASE WHEN up_d < 0 THEN NULL
  WHEN raw_d < 0 THEN raw_d + 4294967296 ELSE raw_d END AS octets FROM d)
SELECT c.device_id, c.interface_id, m.rated_mbps,
  COUNT_IF(c.raw_d < 0 AND c.up_d >= 0) AS wraps,
  COUNT_IF(c.up_d < 0) AS reboots,
  ROUND(MAX(c.octets * 8.0 / 300 / 1e6 / NULLIF(m.rated_mbps, 0)) * 100, 1) AS peak_pct
FROM c JOIN DIM_INTERFACE m
  ON TRIM(m.device_id) = c.device_id AND TRIM(m.interface_id) = c.interface_id
GROUP BY 1, 2, 3
ORDER BY wraps DESC, peak_pct DESC;
```

YOU ARE DONE WHEN

- ☐ Bronze polls 13,520, Silver 11,520 after exactly 2,000 duplicate poll_ids, and 40 x 288 = 11,520 closes it. The master has 40 rows and 40 distinct trimmed device_id plus interface_id pairs.
- ☐ Negative in_octets deltas number exactly 1,008: 992 wraps plus 16 reboots. After the correction no delta is negative, none exceeds 1,500,000,000, and none implies more than 100% of rated speed.
- ☐ gold_iface_interval has exactly 11,480 rows, 16 of them with a NULL in_mbps. Intervals above 80% of rated total exactly 575 - 96 on each of five links and 95 on RTR-03 ge-0/0/1, which loses one to its reboot - which is 2,875 minutes.
- ☐ The query returns 40 rows. The ten 100 Mbps core interfaces lead on wraps with 55 to 58 each; the six links running at 90% of rated show only 10 or 11. Wraps count bytes, not utilisation - that is the answer to question 3, and it is counter-intuitive.

IF YOU FINISH EARLY

Add one data-quality column: the implied Mbps of every corrected delta against rated_mbps, flagged above 100%. A correct wrap fix never produces one, so an empty flag list is the cheapest proof the arithmetic held.